

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
кафедра інформаційних систем та мереж

Григорович Віктор

Програмування та командна робота
Динамічні та рекурсивні структури даних

Інструкції до лабораторних робіт

2020

Григорович Віктор Геннадійович

Програмування та командна робота. Динамічні та рекурсивні структури даних.
Інструкції до лабораторних робіт.

Дисципліна «Програмування та командна робота» вивчається на першому курсі, – цією дисципліною продовжується цикл предметів, що стосуються програмування та розробки програмного забезпечення.

Наведено основні теоретичні відомості, приклади, методичні вказівки з їх розв’язування, варіанти лабораторних завдань з теми «Динамічні та рекурсивні структури даних».

Розглядаються такі роботи лабораторного практикуму:

Лабораторна робота № 3.1. Формування і знищення динамічної мережної структури даних

Лабораторна робота № 3.2. Опрацювання лінійного однонаправленого списку

Лабораторна робота № 3.3. Опрацювання лінійного двонаправленого списку

Лабораторна робота № 3.4. Опрацювання кільцевого однонаправленого списку

Лабораторна робота № 3.5. Стеки

Лабораторна робота № 3.6. Черги

Лабораторна робота № 3.7. Однозв’язні списки

Лабораторна робота № 3.8. Двозв’язні списки

Лабораторна робота № 3.9. Опрацювання бінарних дерев

Лабораторна робота № 3.10. Деревя

Лабораторна робота № 3.11. Опрацювання динамічних структур даних – 1

Лабораторна робота № 3.12. Опрацювання динамічних структур даних – 2

Лабораторна робота № 3.13. Опрацювання префіксних дерев та хеш-таблиць

В посібнику наведено рекомендований набір лабораторних робіт та приклади їх виконання.

Відповідальний за випуск – Григорович В.Г.

Зміст

Вступ.....	4
Тема 3. Динамічні та рекурсивні структури даних	5
Основні теоретичні відомості	5
Загальні поняття	5
Класифікація та термінологія динамічних рекурсивних структур.....	7
Списки	9
Дерева	18
Вказівники на функції та функціональні типи даних	21
Лабораторний практикум	23
Оформлення звітів про виконання лабораторних робіт	23
Лабораторна робота № 3.1. Формування і знищення динамічної мережної структури даних.....	25
Лабораторна робота № 3.2. Опрацювання лінійного однонаправленого списку	38
Лабораторна робота № 3.3. Опрацювання лінійного двонаправленого списку.....	44
Лабораторна робота № 3.4. Опрацювання кільцевого однонаправленого списку	50
Лабораторна робота № 3.5. Стеки	56
Лабораторна робота № 3.6. Черги	64
Лабораторна робота № 3.7. Однозв'язні списки	72
Лабораторна робота № 3.8. Двозв'язні списки.....	80
Лабораторна робота № 3.9. Опрацювання бінарних дерев	89
Лабораторна робота № 3.10. Дерева.....	97
Лабораторна робота № 3.11. Опрацювання динамічних структур даних – 1	105
Лабораторна робота № 3.12. Опрацювання динамічних структур даних – 2	123
Лабораторна робота № 3.13. Опрацювання префіксних дерев та хеш-таблиць	138
Предметний покажчик	153
Література	154

Вступ

Дисципліна «Програмування та командна робота» вивчається на першому курсі, – цією дисципліною продовжується цикл предметів, що стосуються програмування та розробки програмного забезпечення.

Тема «Динамічні та рекурсивні структури даних» продовжує розгляд питань, що стосуються роботи з динамічною пам'яттю, розпочатий при вивченні теми «Одновимірні масиви» в розділах «Вказівники та динамічні змінні» та «Внутрішнє представлення та реалізація масивів». Тема «Динамічні та рекурсивні структури даних» охоплює такі розділи:

Наведено основні теоретичні відомості, приклади, методичні вказівки з їх розв'язування; наведено рекомендований набір лабораторних робіт з теми «Динамічні та рекурсивні структури даних», приклади їх виконання та прийнятий стиль оформлення C/C++ програм.

Тема 3. Динамічні та рекурсивні структури даних

Основні теоретичні відомості

Загальні поняття

Деталі основи роботи з вказівниками див. посібник з Теми 6: Одновимірні масиви.

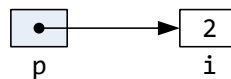
Доступ до змінної за іменем та за вказівником.

Розіменування вказівника

Розглянемо приклад:

```
int i = 2;      // 1
int *p = &i;    // 2
i += 1;         // 3
*p += 1;        // 4
```

Після виконання команд, помічених коментарями `// 1` та `// 2`, структура зв'язків між змінними буде такою:



Команда `// 1` оголошує змінну цілого типу `i` та присвоює їй початкове значення 2.

Команда `// 2` оголошує вказівник `p` на ціле та присвоює йому початкове значення адресу змінної `i`.

Команди `// 3` та `// 4` – еквівалентні: якщо в команді `// 3` значення змінної `i` збільшується на 1 за допомогою звертання до цієї змінної за її іменем, то в команді `// 4` це виконується за допомогою вказівника `p` на змінну `i`.

Конструкція `*p` називається «розіменування вказівника», означає доступ до тої змінної, адреса якої міститься у вказівнику, тобто – переміщення на діаграмі за напрямом стрілки, – до тої змінної, на яку налаштований вказівник.

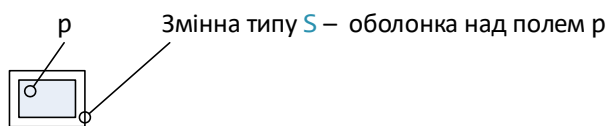
Приклад та поняття рекурсивних динамічних структур даних

Допускаються такі структурні типи, що містять поля, тип яких – *вказівники* на величини таких же структурних типів:

```
struct S      // правильне визначення структурного типу
{
    S *p;     // поле p - вказівник на дані такого ж структурного типу
```

};

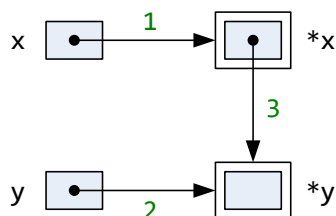
Структурний тип **S** описує множину певних значень. Кожне значення такого типу – це «оболонка» над полем **p**:



В такому випадку поле **p** можна використовувати для того, щоб пов'язати одну змінну структурного типу **S** з іншою змінною цього ж типу, наприклад:

```
S *x, *y;  
  
x = new S; // 1  
y = new S; // 2  
  
x->p = y; // 3
```

Структура зв'язків між змінними показана на рисунку:



Спочатку командою **S** *x, *y; створюються два вказівники **x** та **y**.

Команда, помічена коментарем // 1, створює динамічну змінну структурного типу **S** та налаштовує вказівник **x** на цю змінну. Доступ до вказаної динамічної змінної можливий за допомогою конструкції розіменування вказівника (*x).

Команда **y = new S**; // 2 аналогічна до попередньої.

Команда **x->p = y**; // 3 встановлює зв'язок між двома динамічними змінними: полем **p** тої динамічної змінної, на яку налаштований вказівник **x** присвоюється значення адреси, яка зберігається у вказівнику **y**, тобто, значення адреси динамічної змінної *y.

Конструкція **x->p** означає «поле **p** тої структури, на яку налаштований вказівник **x**», тобто, те саме, що і конструкція (*x).p.

Зауважимо: якщо структурний тип має лише одне поле типу «вказівник на такий же структурний тип», то змінну такого типу можна пов'язати лише з однією змінною такого ж типу. Таке поле будемо називати *полем зв'язку* або *зв'язковим полем*.

Класифікація та термінологія динамічних рекурсивних структур

Список – це структура даних, яка складається із елементів одного типу, послідовно пов’язаних між собою за допомогою вказівників.

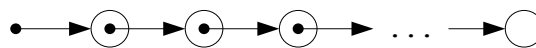
За топологією розрізняють *лінійні* та *кільцеві* списки.

За кількістю зв’язків (тобто, кількістю елементів, з якими пов’язаний кожний елемент) розрізняють *однонаправлені* та *двонаправлені* списки.

За однорідністю елементів розрізняють *списки без елемента-заголовку* (всі елементи мають однаковий зміст) та *списки з елементом-заголовком* (перший елемент списку за змістом відрізняється від інших – він містить деяку службову інформацію, яка характеризує весь список).

Приклади:

Лінійний однонаправлений список без заголовку:



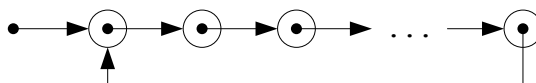
Лінійний однонаправлений список з заголовком:



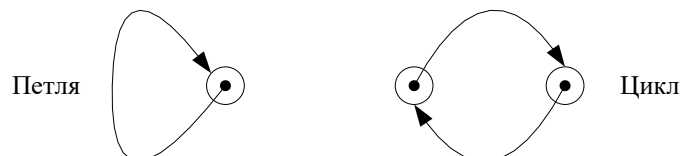
Лінійний двонаправлений список (без заголовку):



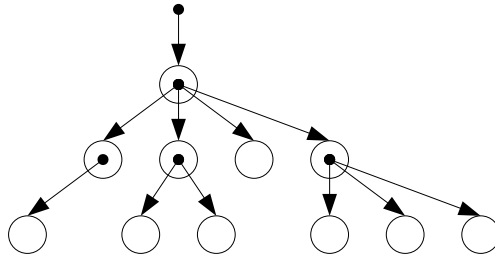
Кільцевий однонаправлений список (без заголовку) :



Дерево – це мережна структура, яка не містить петель та циклів.



Приклад дерева:



Терміни, які стосуються дерев: *рівень ієрархії*, *глибина дерева*, *корінь* (кореневий вузол), *листок* (листовий вузол), *дочірній вузол*, *батьківський вузол*.

Кожне дерево має вузол, з якого починається шлях до будь-якого іншого вузла – виходячи з цього вузла та переміщуючись вздовж стрілок, можна потрапити до будь-якого іншого вузла. Такий вузол називається *коренем* дерева або *кореневим вузлом*. На діаграмах дерево рисують так, що його кореневий вузол зазвичай буде вгорі.

Рівень ієрархії вузла – це кількість переходів, які потрібно зробити, переміщуючись вздовж стрілок, щоб потрапити із кореневого вузла до поточного. Кореневий вузол має нульовий рівень ієрархії.

Глибина дерева (або ще кажуть: *висота дерева*) – це найбільший серед рівнів ієрархії всіх вузлів дерева.

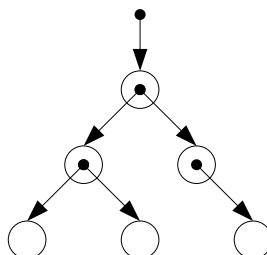
Розглянемо пов'язані (стрілкою) вузли, що знаходяться на двох сусідніх рівнях ієрархії. Той вузол, який розташований вище, на меншому рівні ієрархії, з якого виходить стрілка – називається *батьківським* вузлом. Той вузол, який розташований нижче, на більшому рівні ієрархії, до якого приходить стрілка – називається *дочірнім* вузлом. Кожний вузол може мати не більше одного батьківського вузла, і може мати довільну кількість дочірніх вузлів.

Кореневий вузол не має батьківського.

Якщо деякий вузол не має дочірніх вузлів, то такий вузол називається *лиском* або *листовим вузлом*.

Лінійний однонаправлений список – це різновид дерева (*вироджене* дерево з максимально можливою глибиною для заданої кількості вузлів).

Якщо кожен вузол дерева має не більше двох дочірніх вузлів, то таке дерево називається *бінарним*:



Списки

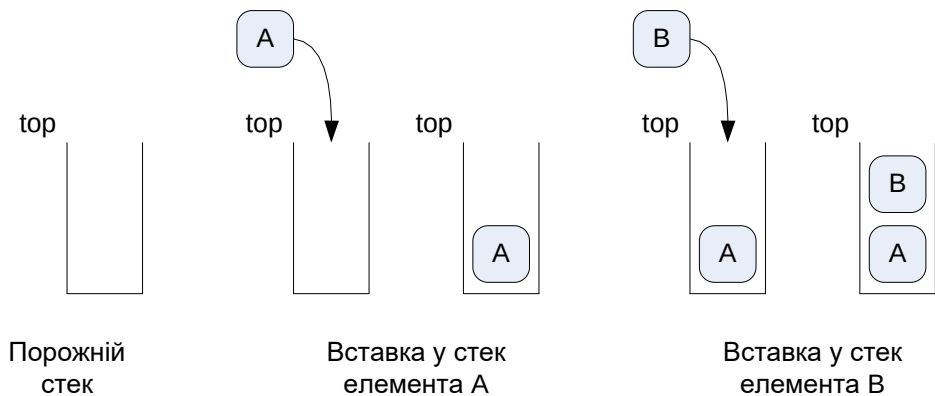
Стек

Стек (англ. *stack* – «пачка», «стос», «стопка») – структура даних, що являє собою список елементів, організованих за принципом LIFO (англ. *last in – first out*, «останнім прийшов – першим вийшов»).

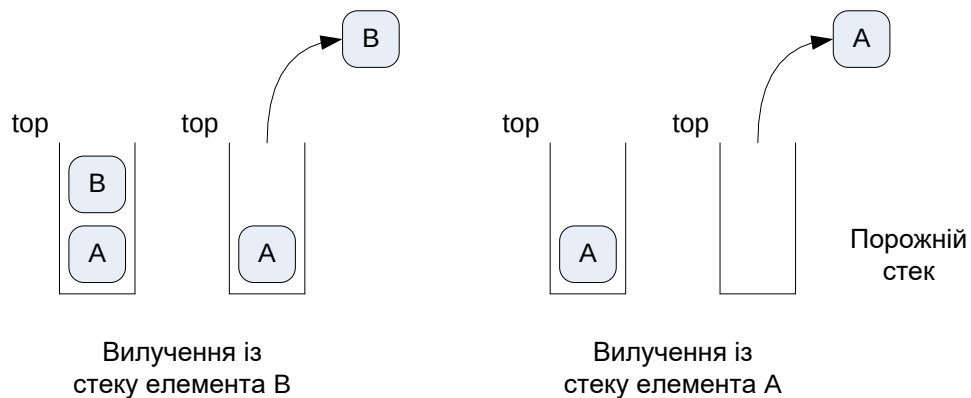
Найчастіше принцип роботи стеку порівнюють із стопкою тарілок: щоб взяти другу зверху, потрібно зняти верхню.

Стек – це лінійний однонаправлений список з однією точкою доступу, яка використовується і для додавання і для вилучення елементів. Ця єдина точка доступу називається вершина стеку – англ. *top* (of stack):

Схему додавання елементів у стек можна зобразити таким чином:



Тоді схема вилучення елементів із стеку виглядатиме таким чином:



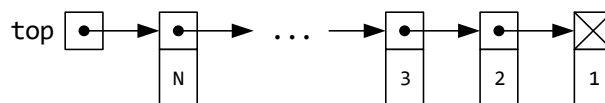
Структура стеку

Структуру стеку можна зобразити таким чином:



– кожен елемент пов'язаний з сусіднім, а вказівник *top* налаштований на вершину стеку.

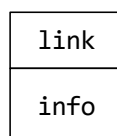
Для такої організації стеку достатньо, щоб кожний елемент містив лише одне поле зв'язку, тому більш детальний вигляд структури стеку можна зобразити так:



Згідно принципу LIFO, вказівник вершини стеку `top` завжди налаштований на той елемент, який був внесений до стеку останнім. Поле зв'язку кожного елемента налаштоване на попередній елемент стеку, поле зв'язку найпершого елемента має значення нульового вказівника.

Опис елементу стеку

Будемо позначати: `link` – поле зв'язку, `info` – інформаційне поле елемента стеку:



Тоді опис елементу стеку у випадку, коли інформаційне поле – цілого типу, матиме вигляд:

```
struct Elem
{
    Elem *link;
    int   info;
};
```

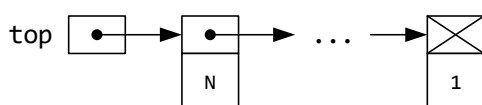
Опис та ініціалізація стеку

Для опису стеку достатньо описати вказівник `top` на його вершину. На початку роботи стек – порожній, тобто вказівник `top` має значення `NULL`:

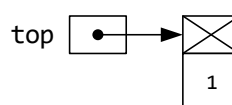
```
Elem *top = NULL;
```

Додавлення елемента (формування стеку)

Напишемо функцію, яка буде добавляти елементи до стеку. Функція має вірно опрацьовувати всі випадки: (1) коли новий елемент добавляється до довільного стеку, (2) коли новий елемент добавляється до стеку, який містить лише один елемент та (3) коли новий елемент добавляється до порожнього стеку:



(1)

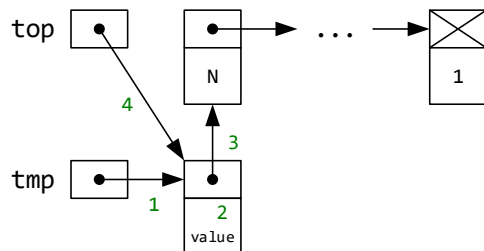


(2)

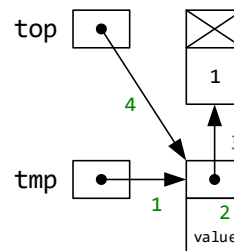


(3)

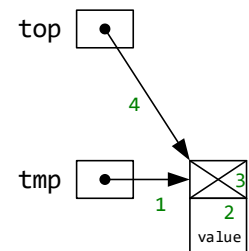
Традиційно для структури даних «стек» така функція називається *push* (англ. «запихати»). Вона має приймати вказівник *top* як посилання – бо в результаті її виконання вказівник *top* зміниться, він стане налаштований на новий елемент, добавлений до стеку, – і потрібно, щоб нове значення вказівника було доступне за межами цієї функції. Функція має також приймати значення *value* типу *Info* – значення інформаційного поля нового елемента стеку.



(1)



(2)

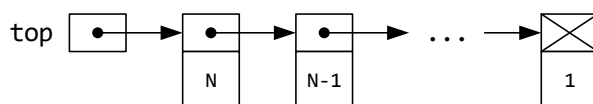


(3)

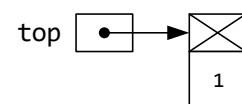
```
void push(Elem* &top, Info value)
{
    Elem *tmp = new Elem; // 1
    tmp->info = value;     // 2
    tmp->link = top;       // 3
    top = tmp;            // 4
}
```

Вилучення елемента (знищення стеку)

Напишемо функцію, яка буде вилучати елементи із стеку. Функція має вірно опрацьовувати випадки: (1) коли вилучається елемент із довільного стеку, (2) коли вилучається останній елемент – тобто, із стеку, який містить лише один елемент. Випадок, коли робиться спроба вилучити елемент із порожнього стеку, функція опрацьовувати не буде, вважаємо це завданням того, хто використовує нашу функцію. Таким чином, функція має вірно працювати, якщо стек містить хоча би один елемент:

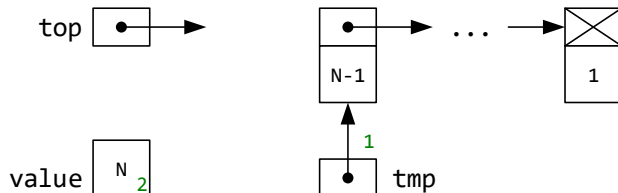
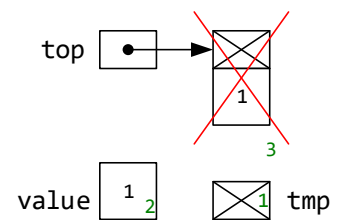
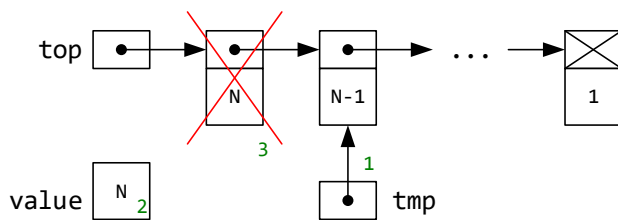


(1)

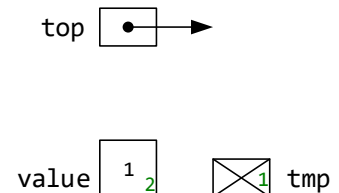


(2)

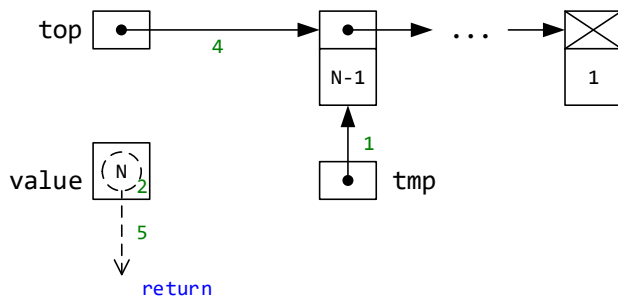
Традиційно для структури даних «стек» така функція називається *pop* (англ. «вистрілювати»). Вона має приймати вказівник *top* як посилання – бо в результаті її виконання вказівник *top* зміниться, він стане налаштований на передостанній елемент (для вилучення елемента із довільного стеку), або отримає значення *NULL* (для вилучення єдиного елемента), – і потрібно, щоб нове значення вказівника було доступне за межами цієї функції. Функція має повертати як результат значення *value* типу *Info* – значення інформаційного поля того елемента стеку, який буде вилучений.



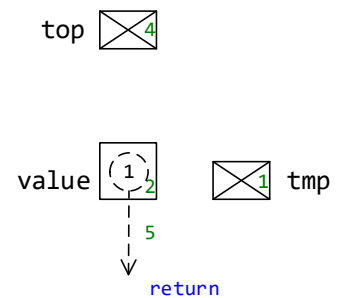
(1)



(2)



(1)



(2)

Остаточно, код функції:

```
Info pop(Elem* &top)
{
    Elem *tmp = top->link; // 1
    Info value = top->info; // 2
    delete top;           // 3
    top = tmp;              // 4
    return value;          // 5
}
```

Черга

Черга (англ. *queue*) – структура даних, що являє собою список елементів, організованих за принципом FIFO (англ. *first in – first out*, «першим прийшов – першим вийшов»).

Додавання елемента (цю операцію називають *enqueue* – поставити в чергу) можливе лише в кінець черги, вибірка – лише з початку, при цьому вибраний елемент видаляється із черги (таку операцію називають *dequeue* – забрати з черги).

Отже, *черга* – це лінійний однонаправлений список з двома точками доступу, одна з яких використовується лише для додавання, а інша – лише для вилучення елементів. Ці точки доступу будемо позначати *first* – перший та *last* – останній (елемент).

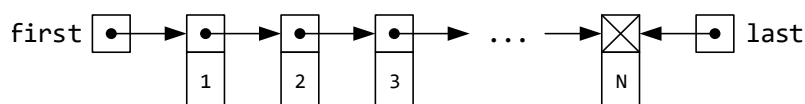
Структура черги

Структуру черги можна зобразити таким чином:



– кожен елемент пов’язаний з сусіднім, вказівник *first* налаштований на початок черги – на перший елемент, а вказівник *last* налаштований на кінець черги – на останній елемент.

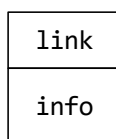
Для такої організації черги достатньо, щоб кожен елемент містив лише одне поле зв’язку, тому більш детальний вигляд структури черги можна зобразити так:



Згідно принципу FIFO, вказівник початку черги *first* завжди налаштований на той елемент, який був внесений до черги першим, а вказівник кінця черги *last* – завжди налаштований на той елемент, який був внесений до черги останнім. Поле зв’язку кожного елемента налаштоване на наступний елемент черги, поле зв’язку останнього елемента має значення нульового вказівника.

Опис елемента черги

Будемо позначати: *link* – поле зв’язку, *info* – інформаційне поле елемента черги:



Опис елемента черги у випадку, коли інформаційне поле – цілого типу, має вигляд:

```

struct Elem
{
    Elem *link;
    int   info;
};
  
```

Опис та ініціалізація черги

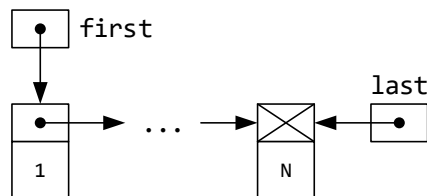
Для опису черги достатньо описати вказівники *first* та *last* на її початок та кінець відповідно. На початку роботи черга – порожня, тобто вказівники *first* та *last* мають значення *NULL*:

```

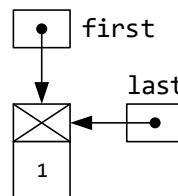
Elem *first = NULL,
      *last  = NULL;
  
```

Додавання елементу (формування черги)

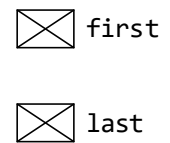
Напишемо функцію, яка буде добавляти елементи до черги. Функція має вірно опрацьовувати всі випадки: (1) коли новий елемент добавляється до довільної черги, (2) коли новий елемент добавляється до черги, яка містить лише один елемент та (3) коли новий елемент добавляється до порожньої черги:



(1)

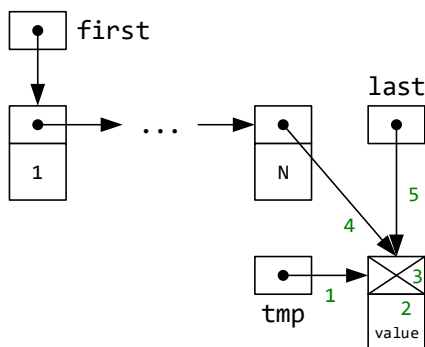


(2)

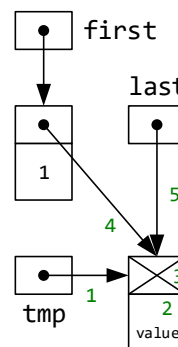


(3)

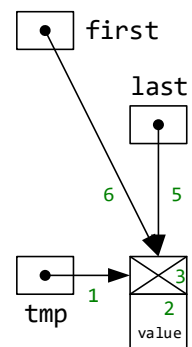
Традиційно для структури даних «черга» така функція називається *enqueue* (англ. «поставити в чергу»). Вона має приймати вказівники *first* та *last* як посилання – бо в результаті її виконання ці вказівники зміняться, – і потрібно, щоб нові значення вказівників були доступними за межами цієї функції. Функція має також приймати значення *value* типу *Info* – значення інформаційного поля нового елемента черги.



(1)



(2)



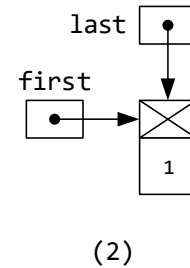
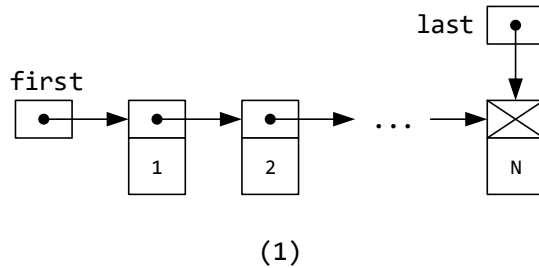
(3)

```
void enqueue(Elem* &first, Elem* &last, Info value)
{
    Elem *tmp = new Elem;    // 1
    tmp->info = value;        // 2
    tmp->link = NULL;         // 3
    if (last != NULL)        // 4
        last->link = tmp;    // 5
    last = tmp;              // 6
    if (first == NULL)
        first = tmp;
}
```

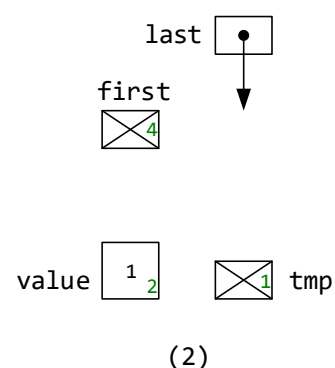
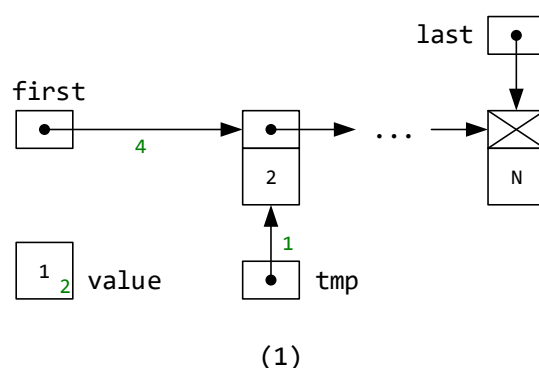
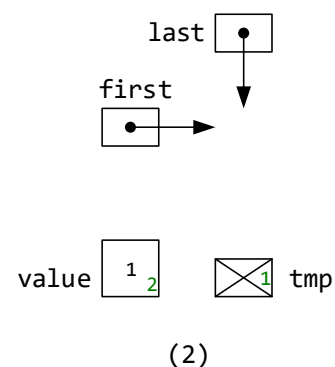
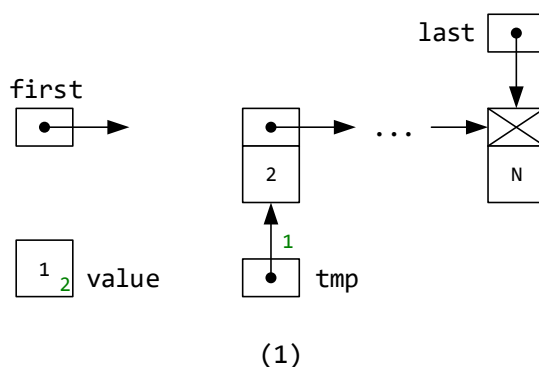
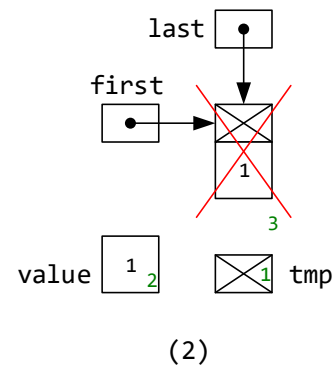
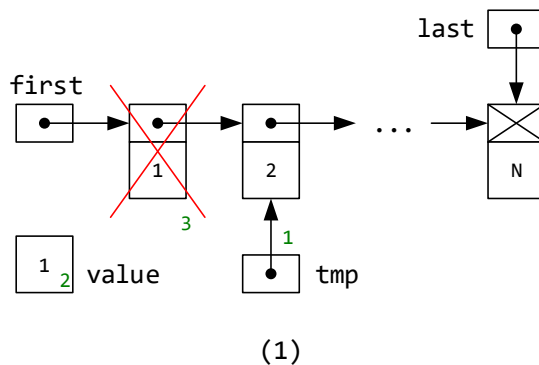
Вилучення елементу (знищення черги)

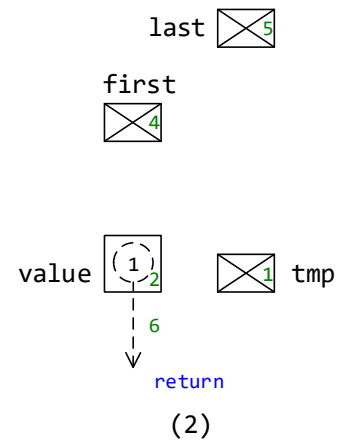
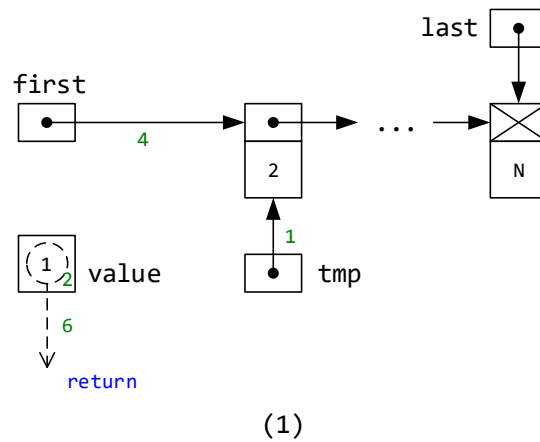
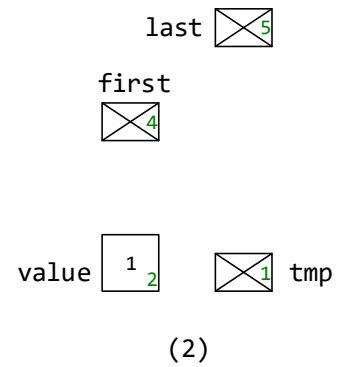
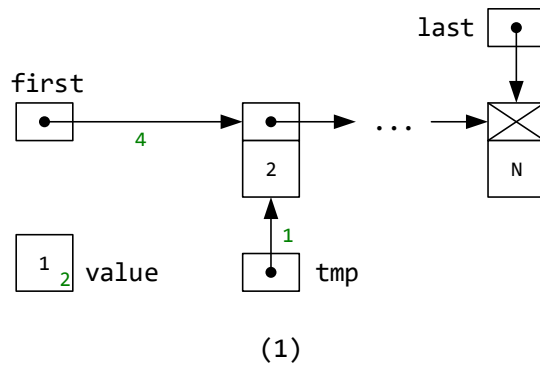
Напишемо функцію, яка буде вилучати елементи із черги. Функція має вірно опрацьовувати випадки: (1) коли вилучається елемент із довільної черги, (2) коли вилучається останній елемент – тобто, із черги, який містить лише один елемент. Випадок,

коли робиться спроба вилучити елемент із порожньої черги, функція опрацьовувати не буде, вважаємо це завданням того, хто використовує нашу функцію. Таким чином, функція має вірно працювати, якщо черга містить хоча би один елемент:



Традиційно для структури даних «черга» така функція називається *dequeue* (англ. «забрати з черги»). Вона має приймати вказівники *first* та *last* як посилання – бо в результаті її виконання ці вказівники зміняться, – і потрібно, щоб нові значення вказівників були доступними за межами цієї функції. Функція має повертати як результат значення *value* типу *Info* – значення інформаційного поля того елемента черги, який буде вилучений.





```
Info dequeue(Elem* &first, Elem* &last)
{
    Elem *tmp = first->link; // 1
    Info value = first->info; // 2
    delete first; // 3
    first = tmp; // 4
    if (first == NULL)
        last = NULL; // 5
    return value; // 6
}
```

Сканування списку

Сканування лінійного однонаправленого списку розглянемо на прикладі функції, яка обчислює кількість його елементів.

Ітераційний спосіб

Функція обчислення кількості елементів списку:

```
int Count(Elem* L)
{
    int k=0;
    while (L != NULL)
    {
        k++;
        L = L->link;
    }
    return k;
}
```


При виклику цієї функції необхідно вказівник на початок списку задати як аргумент; якщо список формується у вигляді черги, то – вказівник на перший її елемент:

```
// кількість елементів
cout << Count(first) << endl;
```

В тілі функції для зберігання обчисленого значення кількості елементів списку використовується локальна змінна *k*, яка отримує початкове значення 0:

```
int k=0;
```

Потім в циклі з умовою «поки вказівник на поточний елемент – не нульовий»

```
while (L != NULL)
```

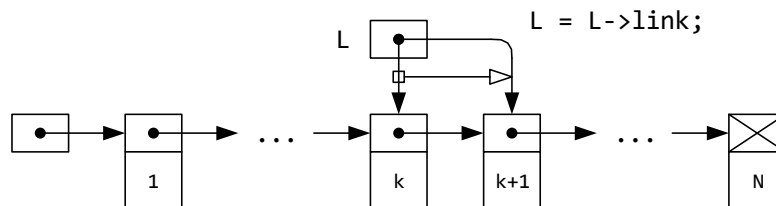
збільшується кількість порахованих елементів

```
k++;
```

та вказівник переходить від поточного до наступного елементу:

```
L = L->link;
```

Ця команда – дуже важлива, саме вона забезпечує переміщення по списку:



Типова помилка – забути про команду переміщення вказівника до наступного елементу при скануванні списку. Без цієї команди ми отримаємо «вічний» цикл.

Рекурсивний спосіб

Рекурсивна функція обчислення кількості елементів списку:

```
int Count(Elem* L)
{
    if (L != NULL)
        return 1 + Count(L->link);
    else
        return 0;
}
```

При виклику цієї функції необхідно вказівник на початок списку задати як аргумент; якщо список формується у вигляді черги, то – вказівник на перший її елемент:

```
// кількість елементів
cout << Count(first) << endl;
```

В тілі функції перевіряється умова «якщо вказівник на поточний елемент – не нульовий»; якщо ця умова – істинна, то функція повертає значення збільшеного на одиницю результату рекурсивного виклику. При рекурсивному виклику функції вказується команда переміщення вказівника від поточного до наступного елемента списку: *L->link*.

Ця команда – дуже важлива, саме вона забезпечує переміщення по списку.

Типова помилка – забути про команду переміщення вказівника до наступного елементу при скануванні списку. Без неї ми отримаємо «вічну» рекурсію.

Якщо умова продовження рекурсії – хибна, тобто, вказівник на поточний елемент списку має нульове значення, то функція повертає 0.

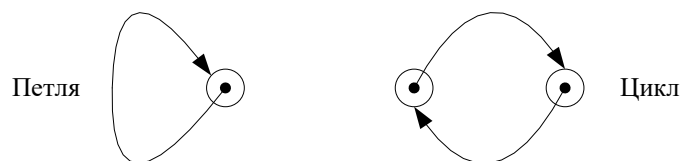
Дерева

Дерево (в теорії графів) – це зв'язний граф без циклів.

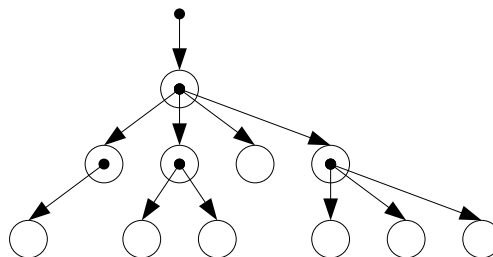
Дерево (англ. *tree*) – в інформатиці та програмуванні одна з найпоширеніших структур даних. Формально дерево визначається як скінченна множина T з однієї або більше вершин (вузлів, *nodes*), яка задовольняє таким вимогам:

1. існує один відокремлений вузол – корінь (*root*) дерева
2. інші вузли (за виключенням кореня) розподілені серед $m \geq 0$ непересічних множин $T_1 \dots T_m$ і кожна з цих множин в свою чергу є деревом. Дерева $T_1 \dots T_m$ мають назву під-дерев (*subtrees*) цього кореня.

Іншими словами, *дерево* – це мережна структура, яка не містить петель та циклів:



Приклад дерева:



Терміни, які стосуються дерев: *рівень ієрархії*, *глибина дерева*, *корінь* (*кореневий вузол*), *листок* (*листовий вузол*), *дочірній вузол*, *батьківський вузол*.

Кожне дерево має вузол, з якого починається шлях до будь-якого іншого вузла – виходячи з цього вузла та переміщуючись вздовж стрілок, можна потрапити до будь-якого іншого вузла. Такий вузол називається *коренем* дерева або *кореневим вузлом*. На діаграмах дерево рисують так, що його кореневий вузол зазвичай буде вгорі.

Рівень ієрархії вузла – це кількість переходів, які потрібно зробити, переміщуючись вздовж стрілок, щоб потрапити із кореневого вузла до поточного. Кореневий вузол має нульовий рівень ієрархії.

Глибина дерева (або ще кажуть: *висота дерева*) – це найбільший серед рівнів ієрархії всіх вузлів дерева.

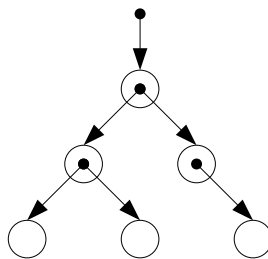
Розглянемо пов'язані (стрілкою) вузли, що знаходяться на двох сусідніх рівнях ієрархії. Той вузол, який розташований вище, на меншому рівні ієрархії, з якого виходить стрілка – називається *батьківським* вузлом (*предком*) . Той вузол, який розташований нижче, на більшому рівні ієрархії, до якого приходить стрілка – називається *дочірнім* вузлом (*нащадком*) . Кожний вузол може мати не більше одного батьківського вузла, і може мати довільну кількість дочірніх вузлів.

Кореневий вузол не має батьківського.

Якщо деякий вузол не має дочірніх вузлів, то такий вузол називається *листом* або *листовим вузлом*.

Лінійний однонаправлений список – це різновид дерева (*вироджене* дерево з максимально можливою глибиною для заданої кількості вузлів).

Якщо кожен вузол дерева має не більше двох дочірніх вузлів, то таке дерево називається *бінарним*:



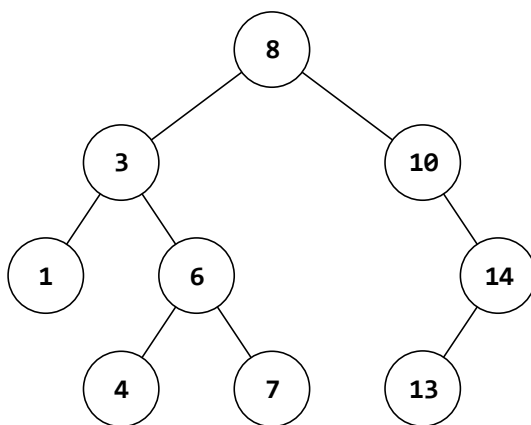
У цьому розділі будемо розглядати лише бінарні дерева.

Якщо вузли бінарного дерева впорядковані певним чином, то, в залежності від способу впорядкування, отримаємо різні види бінарних дерев.

Назвемо значення того інформаційного поля, за яким впорядковуються вузли бінарного дерева, *ключем* вузла.

Якщо x – вузол бінарного дерева з ключем k , і всі вузли в лівому під-дереві мають ключі, не більші k , а в правому під-дереві – не менші k , – то таке бінарне дерево називається *бінарним деревом пошуку*.

Приклад бінарного дерева пошуку:



Бінарні дерева

Структура бінарного дерева

Дерево – це мережна структура без петель та циклів. Елемент такої структури називають *вузлом* (англ. *node*).

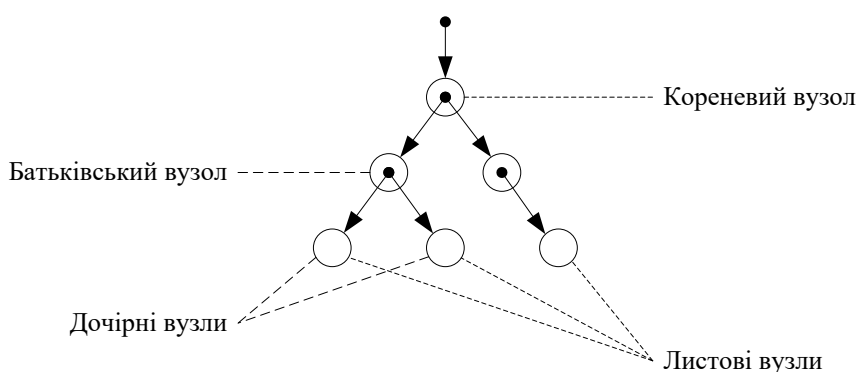
В *бінарному дереві* кожний вузол має не більше двох дочірніх вузлів.

Кожне дерево має вузол, з якого починається шлях до будь-якого іншого вузла – виходячи з цього вузла та переміщуючись вздовж стрілок, можна потрапити до будь-якого іншого вузла. Такий вузол називається *коренем* дерева або *кореневим вузлом*. На діаграмах дерево рисують так, що його кореневий вузол зазвичай буде вгорі.

Кожні два пов'язаних між собою вузла утворюють пару *батьківський вузол* – *дочірній вузол*: одне з полів зв'язку батьківського вузла містить вказівник на дочірній вузол.

Кореневий вузол не має батьківського.

Якщо деякий вузол не має дочірніх вузлів, то такий вузол називається *листом* або *листовим вузлом*.



Для такої організації бінарного дерева достатньо, щоб кожний вузол містив лише два поля зв'язку, тому більш детальний вигляд структури наведеного бінарного дерева можна зобразити так:

Можливі лише дві операції з функціями: виклик та отримання адреси. Вказівник, отриманий за допомогою операції отримання адреси, можна використовувати для виклику функції. Наприклад:

```
...

void f(char *p) { /* ... */ }    // функція

void (*pf)(char*);              // вказівник на функцію

int main()
{
    pf = &f;                    // pf налаштований на функцію f
    (*pf)("abcd");              // виклик функції f через вказівник pf
    ...
}
```

Для виклику функції за допомогою вказівника (pf в нашому прикладі) спочатку потрібно виконати операцію розіменування: *pf. Оскільки пріоритет операції виклику () – вищий, чим пріоритет операції розіменування *, не можна просто записати *pf("abcd"). Це буде означати *(pf("abcd")), що є помилкою. З тієї ж причини дужки потрібні і при оголошенні вказівника на функцію. Проте, записувати просто pf("abcd") – можна, оскільки компілятор розуміє, що pf – це вказівник на функцію, і створює команди, які викликають потрібну функцію.

Формальні параметри у вказівниках на функції описуються так само, як і в звичайних функціях. При присвоєнні вказівнику на функцію потрібна точна відповідність типу функції і типу значення, яке присвоюється. Наприклад:

```
...

void (*pf)(const char*);        // вказівник на void(char*)
void f1(const char*) {}         // void(char*);
int f2(const char*) {...}       // int(char*);
void f3(int*) {}                // void(int*);

void f()
{
    pf = &f1;                    // нормально
    pf = &f2;                    // помилка: не той тип результату
    pf = &f3;                    // помилка: не той тип параметру
    (*pf)("asdf");              // нормально
    (*pf)(1);                   // помилка: не той тип параметру
    int i = (*pf)("qwer");      // помилка: void присвоюється int
}

...
```

Правила передачі параметрів – однакові і для звичайного виклику, і для виклику за допомогою вказівника.

Лабораторний практикум

Оформлення звітів про виконання лабораторних робіт

Вимоги до оформлення звіту про виконання лабораторних робіт №№ 3.1–3.13

Звіти про виконання лабораторних робіт №№ 3.1–3.13 мають містити такі елементи:

1) заголовок;

2) умову завдання;

Умова завдання має бути вставлена у звіт як фрагмент зображення (скрін) сторінки посібника.

Умова завдання – це: спільна частина для всіх варіантів та умова свого варіанту (вона має містити номер варіанту, вставлений як зображення разом з завданням варіанту).

3) структурну схему програми;

Структурна схема програми зображує взаємозв'язки програми та всіх її програмних одиниць: схему вкладеності та охоплення підпрограм, програми та модулів; а також схему звертання одних програмних одиниць до інших.

4) діаграму стану пам'яті;

Діаграма стану пам'яті зображує зв'язки між елементами динамічної структури даних при виконанні ключового фрагменту програми.

5) текст програми;

Текст програми має бути правильно відформатований: відступами і порожніми рядками слід відображати логічну структуру програми; програма має містити необхідні коментарі – про призначення підпрограм, змінних та параметрів – якщо їх імена не значущі, та про призначення окремих змістовних фрагментів програми. Текст програми слід подавати моноширинним шрифтом (Courier New розміром 10 пт. або Consolas розміром 9,5 пт.) з одинарним міжрядковим інтервалом;

6) посилання на git-репозиторій з проектом;

7) хоча б для одної функції, яка повертає результат (як результат функції чи як параметр-посилання) – результати unit-тесту: текст програми unit-тесту та скрін результатів її виконання (див. хід виконання Лабораторної роботи № 5.0);

8) висновки.

Зразок оформлення звіту про виконання лабораторних робіт №№ 3.1–3.13

ЗВІТ

про виконання лабораторної роботи № < номер >

« назва теми лабораторної роботи »

з дисципліни

«Програмування та командна робота»

студента(ки) групи КН-16

< Прізвище Ім'я По_батькові >

Умова завдання:

...

Структурна схема програми:

...

Діаграма стану пам'яті:

...

Текст програми:

...

Посилання на git-репозиторій з проектом:

...

Результати unit-тесту:

...

Висновки:

...

Лабораторна робота № 3.1. Формування і знищення динамічної мережної структури даних

Мета роботи

Навчитися опрацьовувати динамічні структури даних.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення вказівникових типів та змінних-вказівників.
- 2) Загальний синтаксис оголошення динамічних змінних.
- 3) Доступ до змінних за допомогою вказівника.
- 4) Дії з вказівниками та динамічними змінними.
- 5) Внутрішня реалізація вказівників, динамічних та статичних змінних.
- 6) Передавання та опрацьовання вказівників та динамічних змінних у підпрограмах.

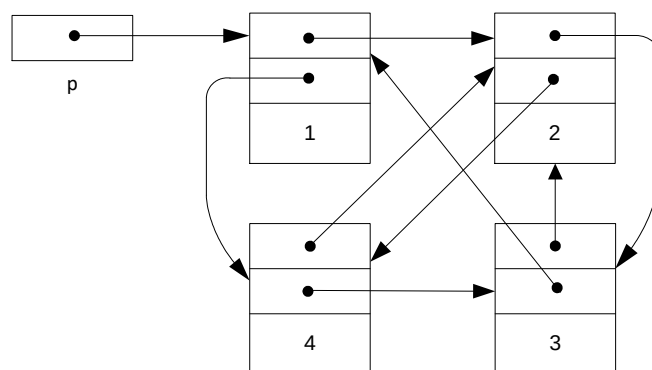
Оформлення звіту

Вимоги та зразок оформлення звіту наведені у вступі до лабораторного практикуму.

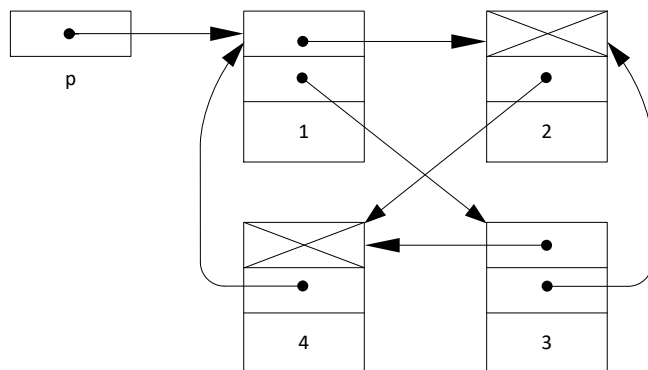
Варіанти лабораторних завдань

Описати, побудувати, а потім знищити наведену структуру зв'язаних динамічних даних (перекреслений блок означає «нульовий вказівник», тобто «вказівник в нікуди»).

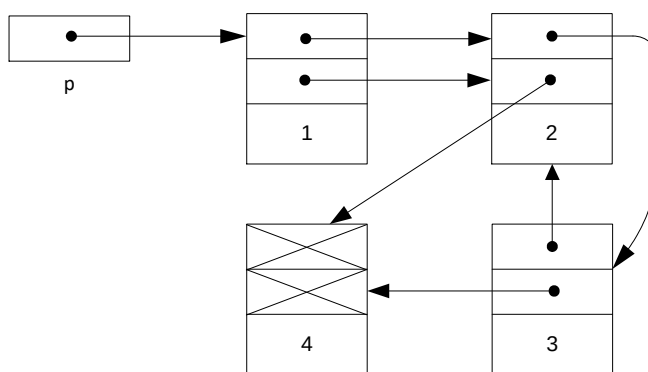
Варіант 1.



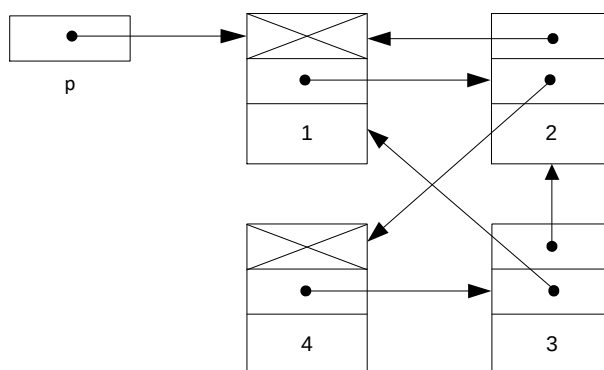
Варіант 2.



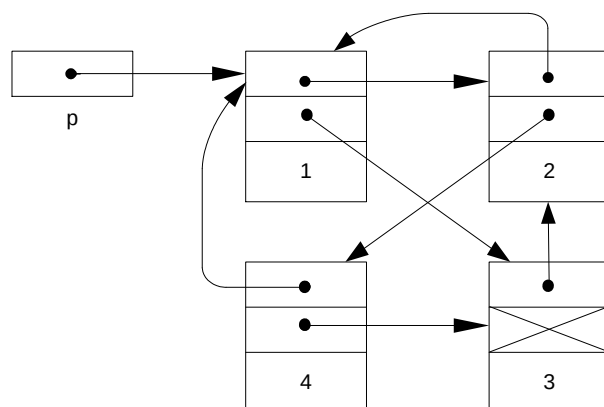
Варіант 3.



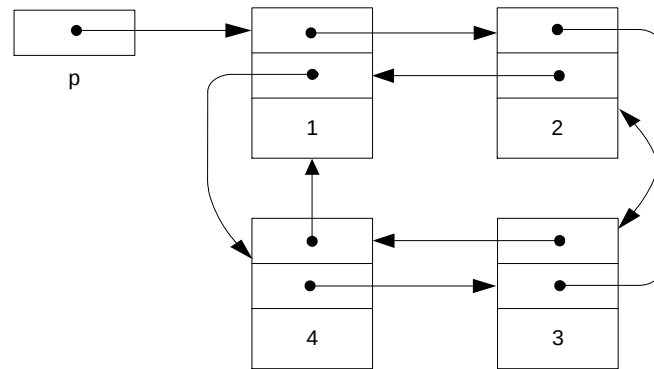
Варіант 4.



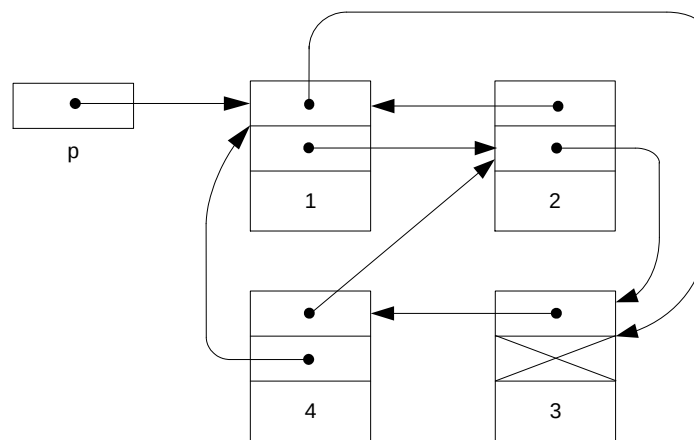
Варіант 5.



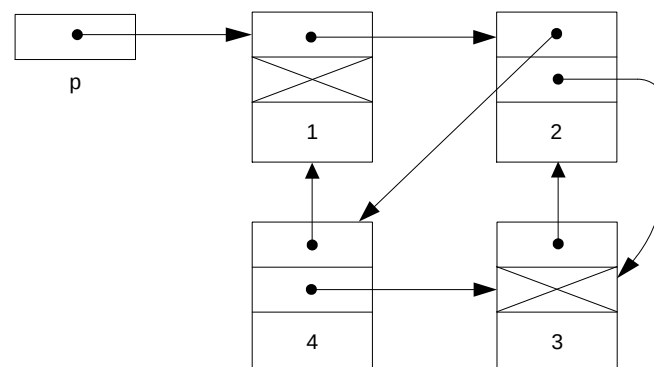
Варіант 6.



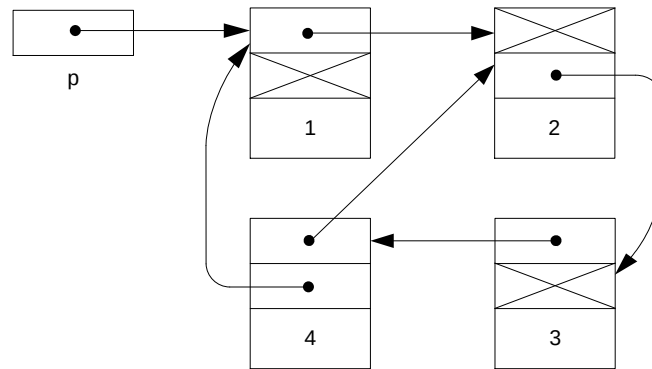
Варіант 7.



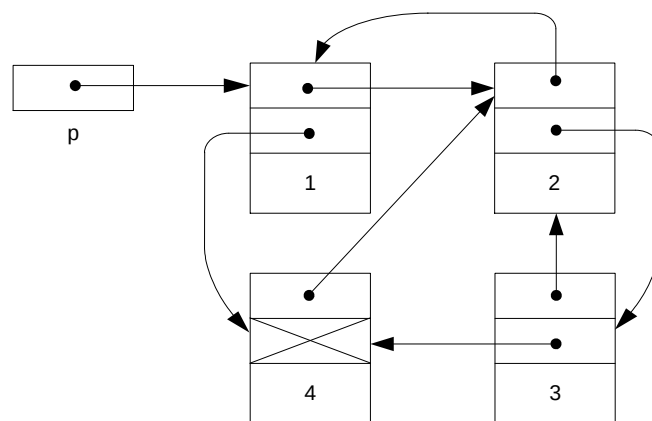
Варіант 8.



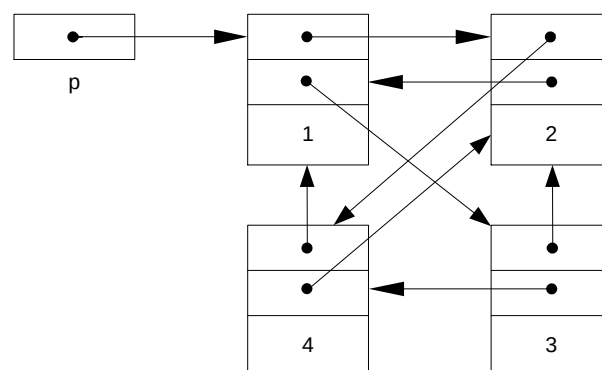
Варіант 9.



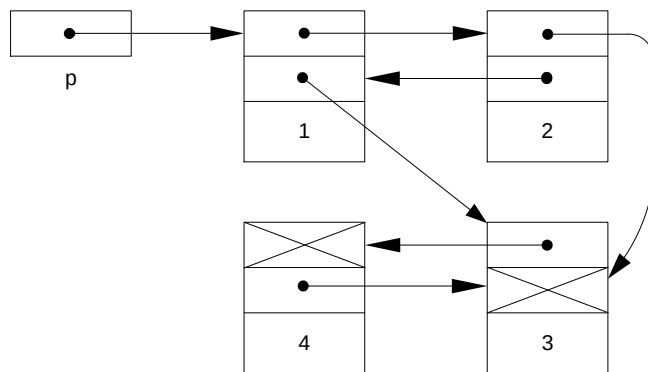
Варіант 10.



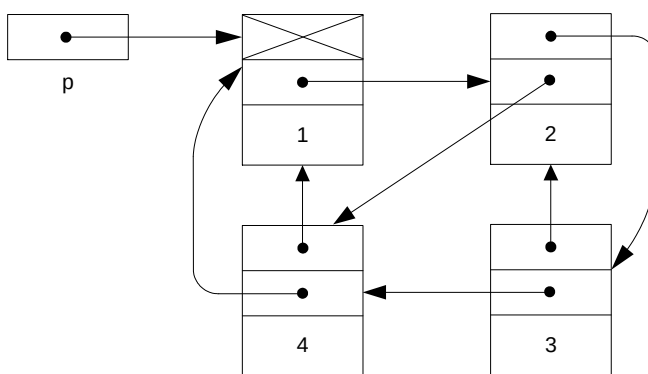
Варіант 11.



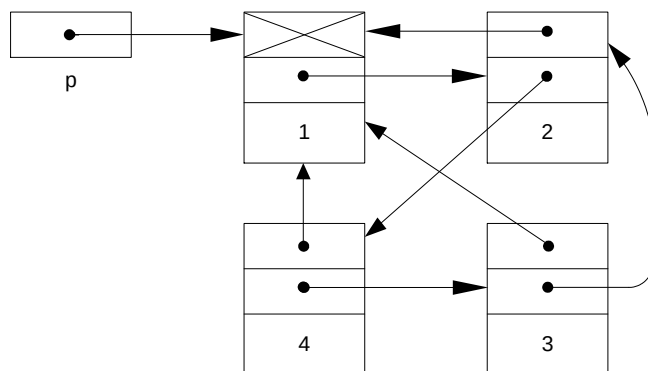
Варіант 12.



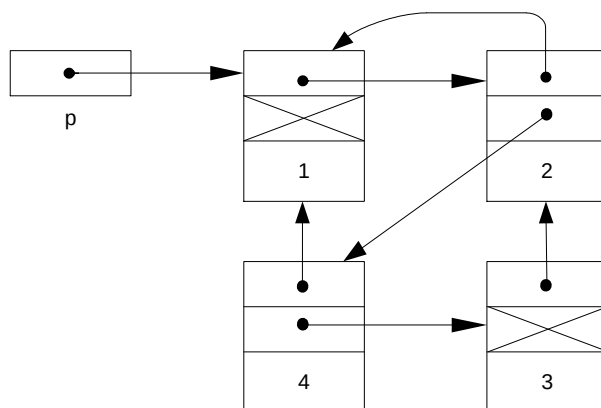
Варіант 13.



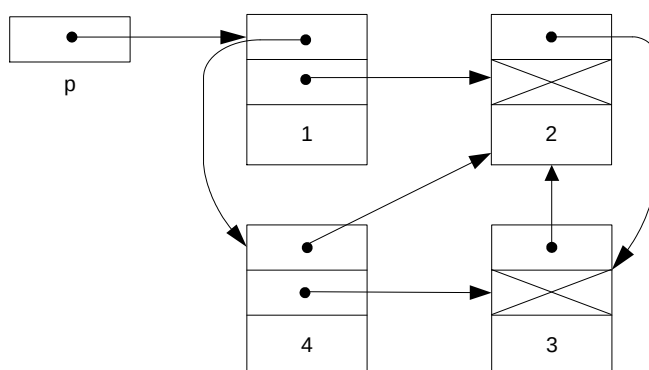
Варіант 14.



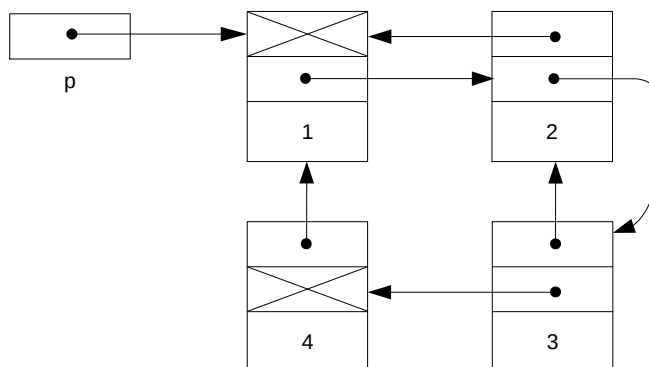
Варіант 15.



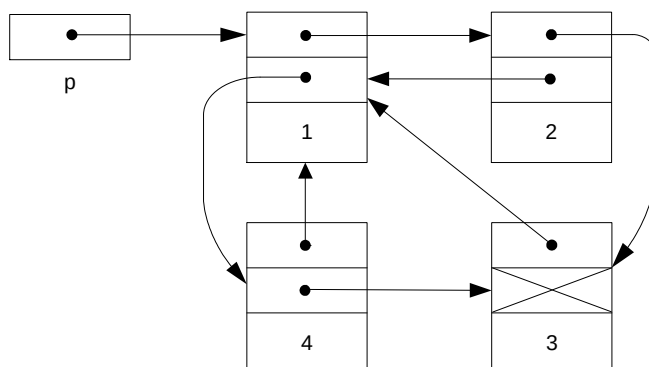
Варіант 16.



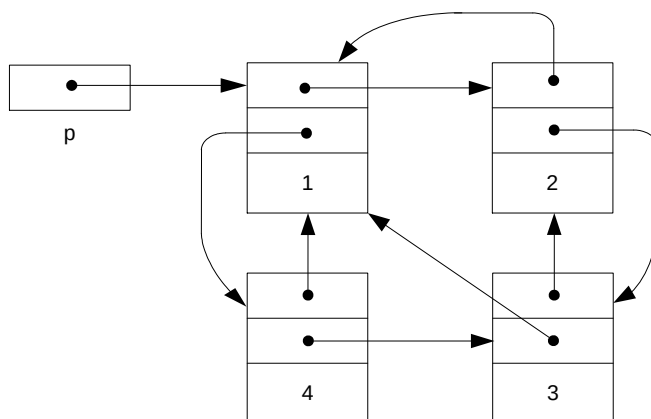
Варіант 17.



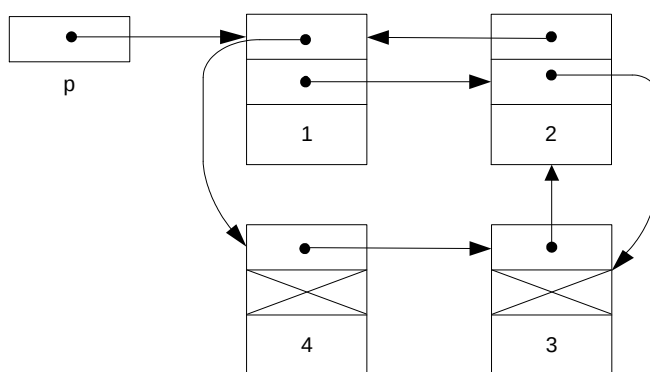
Варіант 18.



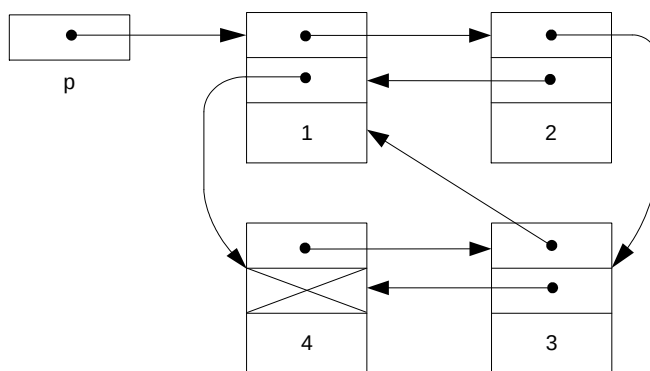
Варіант 19.



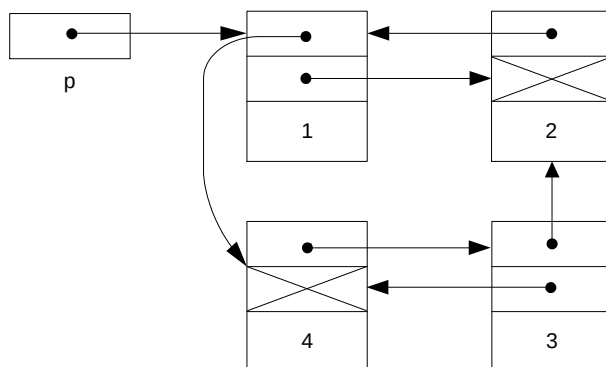
Варіант 20.



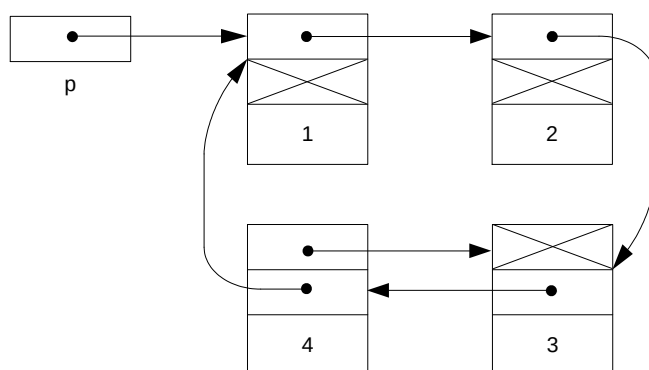
Варіант 21.



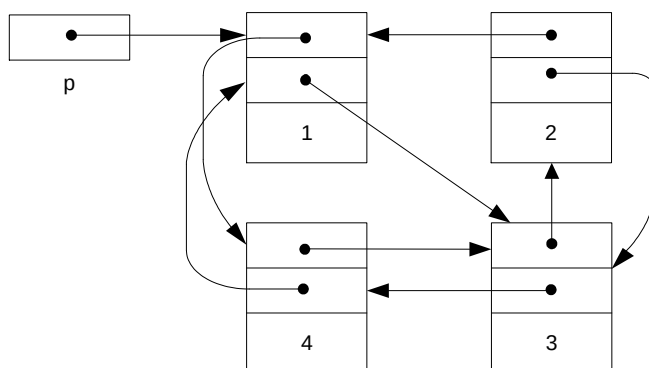
Варіант 22.



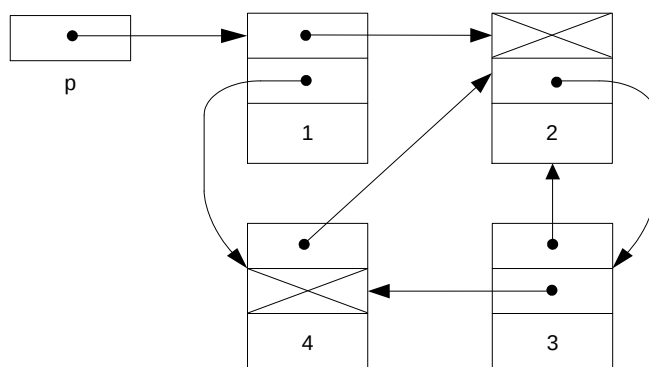
Варіант 23.



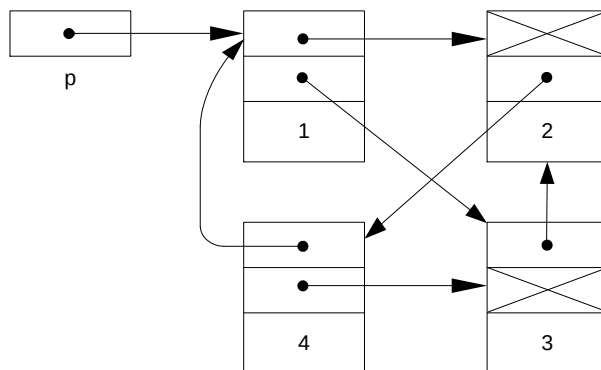
Варіант 24.



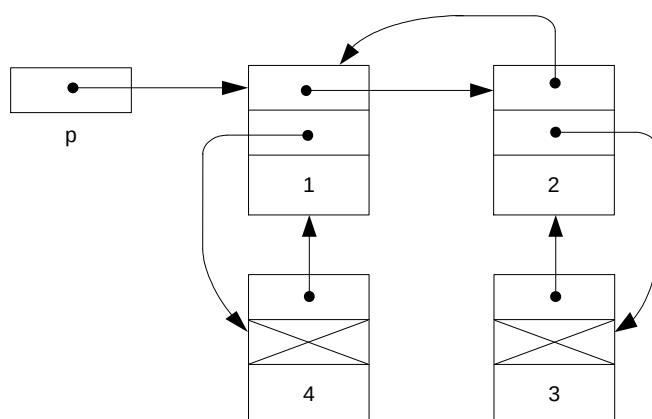
Варіант 25.



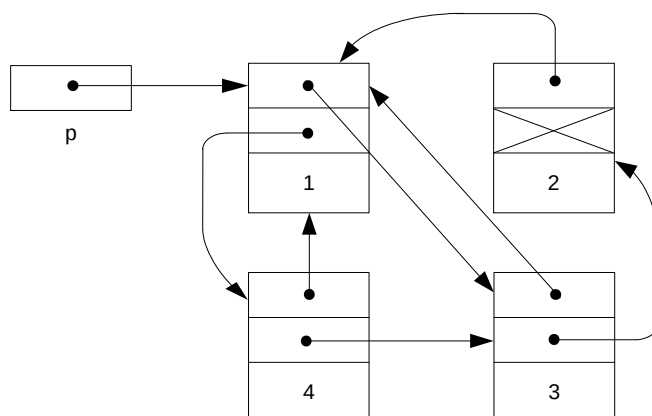
Варіант 26.



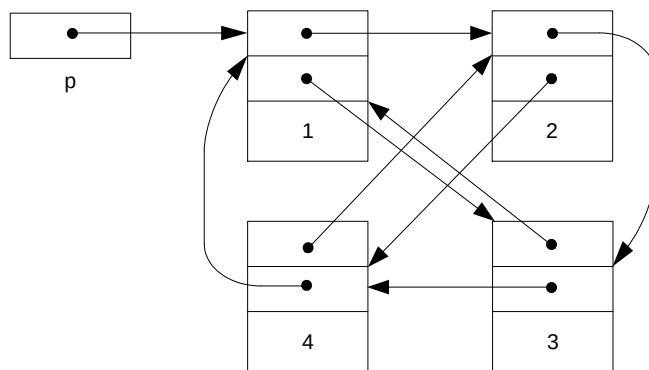
Варіант 27.



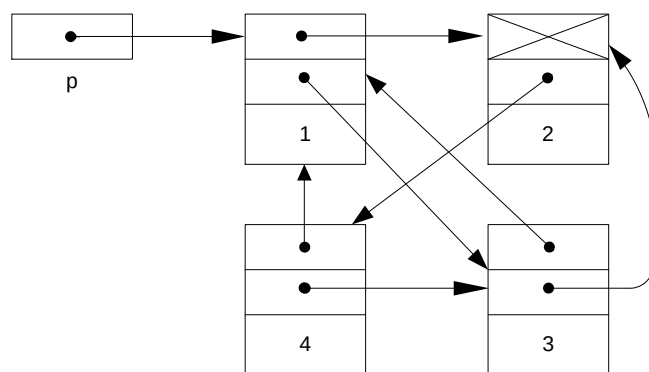
Варіант 28.



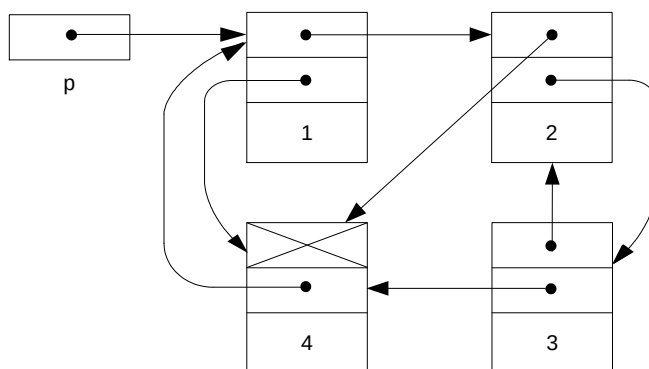
Варіант 29.



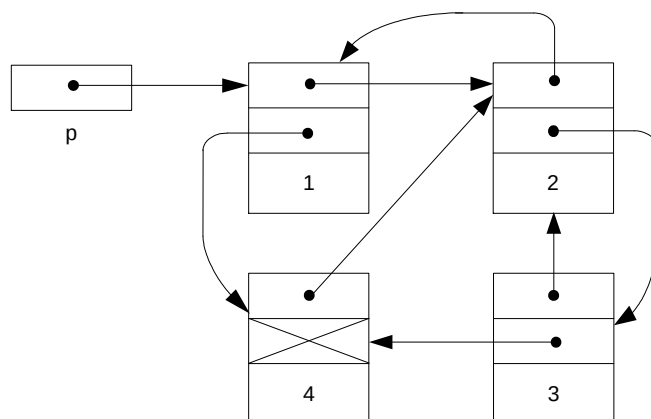
Варіант 30.



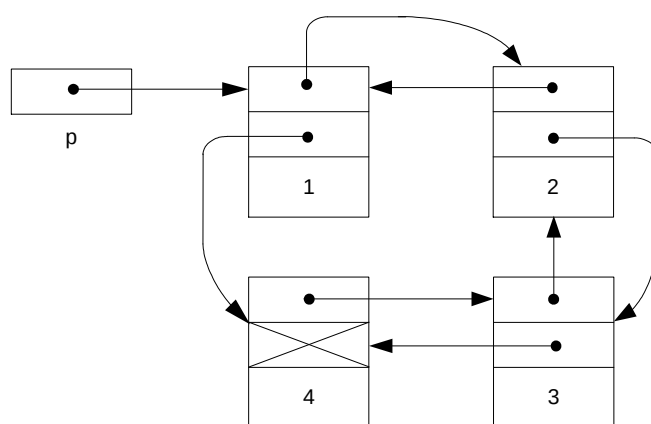
Варіант 31.



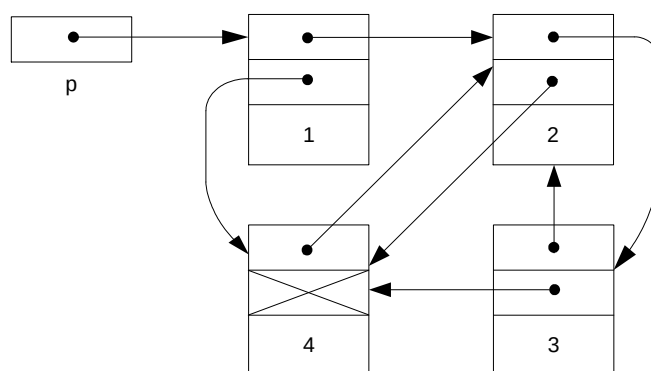
Варіант 32.



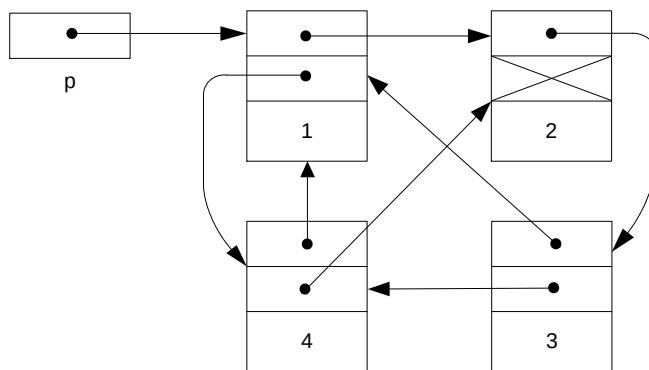
Варіант 33.



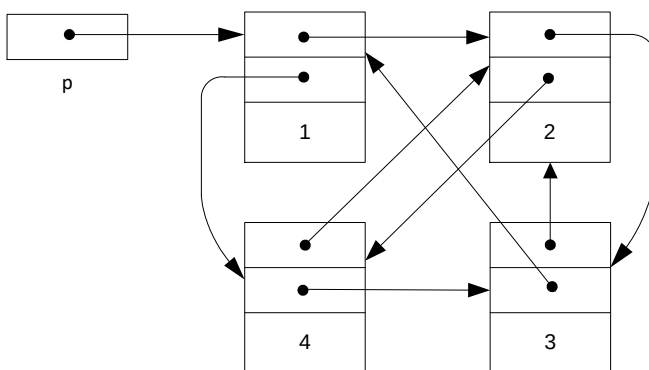
Варіант 34.



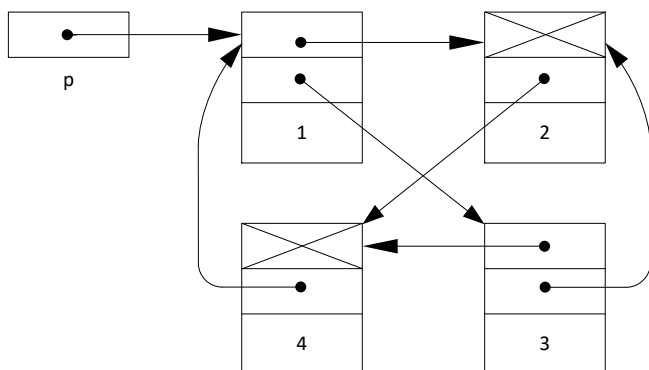
Варіант 35.



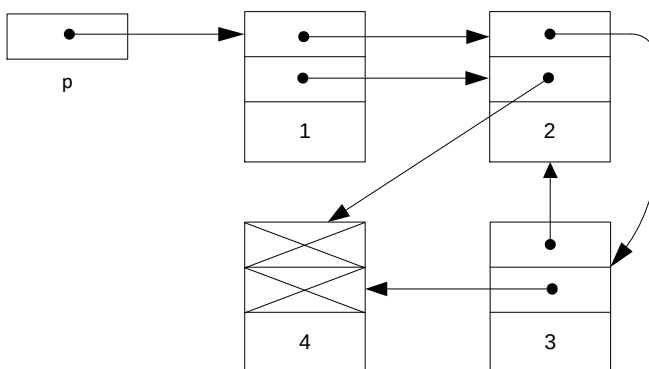
Варіант 36.



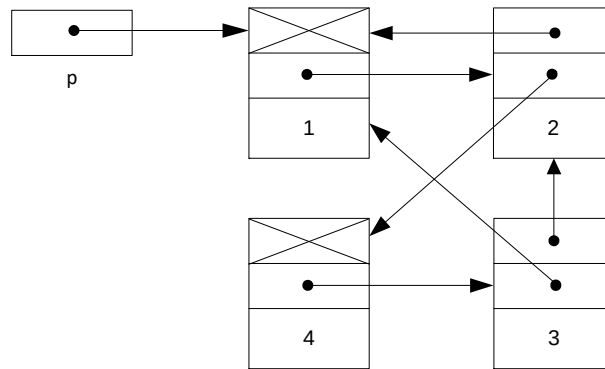
Варіант 37.



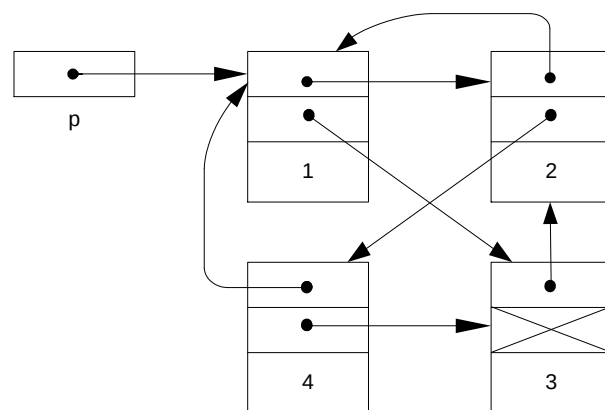
Варіант 38.



Варіант 39.



Варіант 40.



Лабораторна робота № 3.2. Опрацювання лінійного однонаправленого списку

Мета роботи

Навчитися опрацьовувати лінійні однонаправлені списки.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення елемента лінійного однонаправленого списку та вказівника на нього.
- 2) Загальні схеми створення лінійного однонаправленого списку (добавлення елемента), знищення лінійного однонаправленого списку (вилучення елемента).
- 3) Довільні лінійні однонаправлені списки та впорядковані лінійні однонаправлені списки (списки з пріоритетами).
- 4) Загальні схеми обходу лінійного однонаправленого списку.
- 5) Доступ до елементів лінійного однонаправленого списку.
- 6) Дії із лінійним однонаправленим списком.
- 7) Внутрішня реалізація лінійного однонаправленого списку.
- 8) Передавання та опрацювання лінійних однонаправлених списків у функціях.

Оформлення звіту

Вимоги та зразок оформлення звіту наведені у вступі до лабораторного практикуму.

Варіанти лабораторних завдань

Всі завдання слід реалізувати з використанням лінійних однонаправлених списків. Якщо при виконанні завдання слід утворити нові списки, – вони мають бути тої самої структури.

Якщо у завданні не вказано тип даних інформаційного поля, його слід вибрати самостійно.

Необхідно:

- сформулювати заданий список (списки);
- роздрукувати його (їх) – вивести значення елементів на екран;
- виконати вказані в завданні дії;
- вивести результат на екран.

Всі вказані дії слід виконувати в окремих підпрограмах, які всю інформацію приймають у вигляді параметрів. Використовувати нелокальні змінні не можна.

Кожна функція має виконувати лише одну роль, і ця роль має бути відображена у назві функції.

«Функція, яка повертає / обчислює / шукає ...» — має не виводити ці значення, а повернути їх у місце виклику як результат функції або як відповідний вихідний параметр.

До звіту необхідно подати діаграму змін стану пам'яті при виконанні всіх операцій із списком, вказаних у варіанті завдання.

Варіант 1.

Обчислити кількість елементів списку з непарними значеннями інформаційного поля.

Варіант 2.

Обчислити кількість елементів списку із заданим користувачем значенням інформаційного поля.

Варіант 3.

Обчислити суму елементів списку із парними значеннями інформаційного поля.

Варіант 4.

Обчислити суму елементів списку із додатними значеннями інформаційного поля.

Варіант 5.

Обчислити кількість елементів списку із від'ємними значеннями інформаційного поля.

Варіант 6.

Збільшити значення інформаційного поля кожного елемента списку на задану користувачем величину.

Варіант 7.

Продублювати кожний елемент списку із заданим користувачем значенням інформаційного поля.

Варіант 8.

Вилучити кожний елемент списку із заданим користувачем значенням інформаційного поля.

Варіант 9.

Вилучити кожний елемент списку, який передує елементу із заданим користувачем значенням інформаційного поля.

Варіант 10.

Вилучити кожний елемент списку, який слідує за елементом із заданим користувачем значенням інформаційного поля.

Варіант 11.

Перед кожним елементом списку із значенням інформаційного поля V1 вставити новий елемент із значенням інформаційного поля V2.

Варіант 12.

Після кожного елементу списку із значенням інформаційного поля V1 вставити новий елемент із значенням інформаційного поля V2.

Варіант 13.

Перевірити, чи елементи списку – впорядковані за не спаданням.

Варіант 14.

Визначити, чи список містить пару сусідніх елементів з однаковими значеннями інформаційного поля.

Варіант 15.*

Визначити, чи список містить пару елементів (не обов'язково сусідніх) з однаковими значеннями інформаційного поля.

Варіант 16.*

Визначити, чи всі елементи списку L1 входять у список L2 (тобто, чи список L2 містить всі елементи списку L1, розташовані у довільному порядку).

Варіант 17.*

Визначити, чи список L1 входить як частина у список L2 (тобто, чи список L2 містить всі елементи списку L1, розташовані у тому ж самому порядку).

Варіант 18.

На основі списку L створити два списки: $L1$ – із елементів з додатними значеннями інформаційного поля та $L2$ – із елементів з від’ємними значеннями інформаційного поля, зберігши порядок їх розташування.

Варіант 19.

Поміняти місцями елементи списку – змінити порядок розташування на зворотний.

Варіант 20.

Поміняти місцями елементи списку – перший з другим, третій з четвертим і т.д.

Варіант 21.

Обчислити кількість елементів списку з непарними значеннями інформаційного поля.

Варіант 22.

Обчислити кількість елементів списку із заданим користувачем значенням інформаційного поля.

Варіант 23.

Обчислити суму елементів списку із парними значеннями інформаційного поля.

Варіант 24.

Обчислити суму елементів списку із додатними значеннями інформаційного поля.

Варіант 25.

Обчислити кількість елементів списку із від’ємними значеннями інформаційного поля.

Варіант 26.

Збільшити значення інформаційного поля кожного елемента списку на задану користувачем величину.

Варіант 27.

Продублювати кожний елемент списку із заданим користувачем значенням інформаційного поля.

Варіант 28.

Вилучити кожний елемент списку із заданим користувачем значенням інформаційного поля.

Варіант 29.

Вилучити кожний елемент списку, який передує елементу із заданим користувачем значенням інформаційного поля.

Варіант 30.

Вилучити кожний елемент списку, який слідує за елементом із заданим користувачем значенням інформаційного поля.

Варіант 31.

Перед кожним елементом списку із значенням інформаційного поля V1 вставити новий елемент із значенням інформаційного поля V2.

Варіант 32.

Після кожного елементу списку із значенням інформаційного поля V1 вставити новий елемент із значенням інформаційного поля V2.

Варіант 33.

Перевірити, чи елементи списку – впорядковані за не спаданням.

Варіант 34.

Визначити, чи список містить пару сусідніх елементів з однаковими значеннями інформаційного поля.

Варіант 35.*

Визначити, чи список містить пару елементів (не обов'язково сусідніх) з однаковими значеннями інформаційного поля.

Варіант 36.*

Визначити, чи всі елементи списку L1 входять у список L2 (тобто, чи список L2 містить всі елементи списку L1, розташовані у довільному порядку).

Варіант 37.*

Визначити, чи список L1 входить як частина у список L2 (тобто, чи список L2 містить всі елементи списку L1, розташовані у тому ж самому порядку).

Варіант 38.

На основі списку L створити два списки: L1 – із елементів з додатними значеннями інформаційного поля та L2 – із елементів з від’ємними значеннями інформаційного поля, зберігши порядок їх розташування.

Варіант 39.

Поміняти місцями елементи списку – змінити порядок розташування на зворотний.

Варіант 40.

Поміняти місцями елементи списку – перший з другим, третій з четвертим і т.д.

Лабораторна робота № 3.3. Опрацювання лінійного двонаправленого списку

Мета роботи

Навчитися опрацьовувати лінійні двонаправлені списки.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення елемента лінійного двонаправленого списку та вказівника на нього.
- 2) Загальні схеми створення лінійного двонаправленого списку (добавлення елемента), знищення лінійного двонаправленого списку (вилучення елемента).
- 3) Довільні лінійні двонаправлені списки та впорядковані лінійні двонаправлені списки (списки з пріоритетами).
- 4) Загальні схеми обходу лінійного двонаправленого списку.
- 5) Доступ до елементів лінійного двонаправленого списку.
- 6) Дії із лінійним двонаправленим списком.
- 7) Внутрішня реалізація лінійного двонаправленого списку.
- 8) Передавання та опрацювання лінійних двонаправлених списків у функціях.

Оформлення звіту

Вимоги та зразок оформлення звіту наведені у вступі до лабораторного практикуму.

Варіанти лабораторних завдань

Всі завдання слід реалізувати з використанням лінійних двонаправлених списків. Якщо при виконанні завдання слід утворити нові списки, – вони мають бути тої самої структури.

Якщо у завданні не вказано тип даних інформаційного поля, його слід вибрати самостійно.

Необхідно:

- сформулювати заданий список (списки);
- роздрукувати його (їх) – вивести значення елементів на екран;
- виконати вказані в завданні дії;
- вивести результат на екран.

Всі вказані дії слід виконувати в окремих підпрограмах, які всю інформацію приймають у вигляді параметрів. Використовувати нелокальні змінні не можна.

Кожна функція має виконувати лише одну роль, і ця роль має бути відображена у назві функції.

«Функція, яка повертає / обчислює / шукає ...» — має не виводити ці значення, а повернути їх у місце виклику як результат функції або як відповідний вихідний параметр.

До звіту необхідно подати діаграму змін стану пам'яті при виконанні всіх операцій із списком, вказаних у варіанті завдання.

Варіант 1.

Продублювати кожний елемент списку із заданим користувачем значенням інформаційного поля.

Варіант 2.

Вилучити кожний елемент списку із заданим користувачем значенням інформаційного поля.

Варіант 3.

Вилучити кожний елемент списку, який передує елементу із заданим користувачем значенням інформаційного поля.

Варіант 4.

Вилучити кожний елемент списку, який слідує за елементом із заданим користувачем значенням інформаційного поля.

Варіант 5.

Перед кожним елементом списку із значенням інформаційного поля V1 вставити новий елемент із значенням інформаційного поля V2.

Варіант 6.

Після кожного елементу списку із значенням інформаційного поля V1 вставити новий елемент із значенням інформаційного поля V2.

Варіант 7.

Перевірити, чи елементи списку – впорядковані за не спаданням.

Варіант 8.

Визначити, чи список містить пару сусідніх елементів з однаковими значеннями інформаційного поля.

Варіант 9.*

Визначити, чи список містить пару елементів (не обов'язково сусідніх) з однаковими значеннями інформаційного поля.

Варіант 10.*

Визначити, чи всі елементи списку L1 входять у список L2 (тобто, чи список L2 містить всі елементи списку L1, розташовані у довільному порядку).

Варіант 11.*

Визначити, чи список L1 входить як частина у список L2 (тобто, чи список L2 містить всі елементи списку L1, розташовані у тому ж самому порядку).

Варіант 12.

На основі списку L створити два списки: L1 – із елементів з додатними значеннями інформаційного поля та L2 – із елементів з від'ємними значеннями інформаційного поля, зберігши порядок їх розташування.

Варіант 13.

Поміняти місцями елементи списку – змінити порядок розташування на зворотний.

Варіант 14.

Поміняти місцями елементи списку – перший з другим, третій з четвертим і т.д.

Варіант 15.

Обчислити кількість елементів списку з непарними значеннями інформаційного поля.

Варіант 16.

Обчислити кількість елементів списку із заданим користувачем значенням інформаційного поля.

Варіант 17.

Обчислити суму елементів списку із парними значеннями інформаційного поля.

Варіант 18.

Обчислити суму елементів списку із додатними значеннями інформаційного поля.

Варіант 19.

Обчислити кількість елементів списку із від'ємними значеннями інформаційного поля.

Варіант 20.

Збільшити значення інформаційного поля кожного елемента списку на задану користувачем величину.

Варіант 21.

Продублювати кожний елемент списку із заданим користувачем значенням інформаційного поля.

Варіант 22.

Вилучити кожний елемент списку із заданим користувачем значенням інформаційного поля.

Варіант 23.

Вилучити кожний елемент списку, який передує елементу із заданим користувачем значенням інформаційного поля.

Варіант 24.

Вилучити кожний елемент списку, який слідує за елементом із заданим користувачем значенням інформаційного поля.

Варіант 25.

Перед кожним елементом списку із значенням інформаційного поля V1 вставити новий елемент із значенням інформаційного поля V2.

Варіант 26.

Після кожного елементу списку із значенням інформаційного поля V1 вставити новий елемент із значенням інформаційного поля V2.

Варіант 27.

Перевірити, чи елементи списку – впорядковані за не спаданням.

Варіант 28.

Визначити, чи список містить пару сусідніх елементів з однаковими значеннями інформаційного поля.

Варіант 29.*

Визначити, чи список містить пару елементів (не обов'язково сусідніх) з однаковими значеннями інформаційного поля.

Варіант 30.*

Визначити, чи всі елементи списку L1 входять у список L2 (тобто, чи список L2 містить всі елементи списку L1, розташовані у довільному порядку).

Варіант 31.*

Визначити, чи список L1 входить як частина у список L2 (тобто, чи список L2 містить всі елементи списку L1, розташовані у тому ж самому порядку).

Варіант 32.

На основі списку L створити два списки: L1 – із елементів з додатними значеннями інформаційного поля та L2 – із елементів з від'ємними значеннями інформаційного поля, зберігши порядок їх розташування.

Варіант 33.

Поміняти місцями елементи списку – змінити порядок розташування на зворотний.

Варіант 34.

Поміняти місцями елементи списку – перший з другим, третій з четвертим і т.д.

Варіант 35.

Обчислити кількість елементів списку з непарними значеннями інформаційного поля.

Варіант 36.

Обчислити кількість елементів списку із заданим користувачем значенням інформаційного поля.

Варіант 37.

Обчислити суму елементів списку із парними значеннями інформаційного поля.

Варіант 38.

Обчислити суму елементів списку із додатними значеннями інформаційного поля.

Варіант 39.

Обчислити кількість елементів списку із від'ємними значеннями інформаційного поля.

Варіант 40.

Збільшити значення інформаційного поля кожного елемента списку на задану користувачем величину.

Лабораторна робота № 3.4. Опрацювання кільцевого однонаправленого списку

Мета роботи

Навчитися опрацьовувати кільцеві однонаправлені списки.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення елемента кільцевого однонаправленого списку та вказівника на нього.
- 2) Загальні схеми створення кільцевого однонаправленого списку (добавлення елемента), знищення кільцевого однонаправленого списку (вилучення елемента).
- 3) Довільні кільцеві однонаправлені списки та впорядковані кільцеві однонаправлені списки (списки з пріоритетами).
- 4) Загальні схеми обходу кільцевого однонаправленого списку.
- 5) Доступ до елементів кільцевого однонаправленого списку.
- 6) Дії із кільцевим однонаправленим списком.
- 7) Внутрішня реалізація кільцевого однонаправленого списку.
- 8) Передавання та опрацювання кільцевих однонаправлених списків у функціях.

Оформлення звіту

Вимоги та зразок оформлення звіту наведені у вступі до лабораторного практикуму.

Варіанти лабораторних завдань

Всі завдання слід реалізувати з використанням кільцевих однонаправлених списків. Якщо при виконанні завдання слід утворити нові списки, – вони мають бути тої самої структури.

Якщо у завданні не вказано тип даних інформаційного поля, його слід вибрати самостійно.

Необхідно:

- сформулювати заданий список (списки);
- роздрукувати його (їх) – вивести значення елементів на екран;
- виконати вказані в завданні дії;
- вивести результат на екран.

Всі вказані дії слід виконувати в окремих підпрограмах, які всю інформацію приймають у вигляді параметрів. Використовувати нелокальні змінні не можна.

Кожна функція має виконувати лише одну роль, і ця роль має бути відображена у назві функції.

«Функція, яка повертає / обчислює / шукає ...» – має не виводити ці значення, а повернути їх у місце виклику як результат функції або як відповідний вихідний параметр.

До звіту необхідно подати діаграму змін стану пам'яті при виконанні всіх операцій із списком, вказаних у варіанті завдання.

Варіант 1.

Перевірити, чи елементи списку – впорядковані за не спаданням.

Варіант 2.

Визначити, чи список містить пару сусідніх елементів з однаковими значеннями інформаційного поля.

Варіант 3.*

Визначити, чи список містить пару елементів (не обов'язково сусідніх) з однаковими значеннями інформаційного поля.

Варіант 4.*

Визначити, чи всі елементи списку L1 входять у список L2 (тобто, чи список L2 містить всі елементи списку L1, розташовані у довільному порядку).

Варіант 5.*

Визначити, чи список L1 входить як частина у список L2 (тобто, чи список L2 містить всі елементи списку L1, розташовані у тому ж самому порядку).

Варіант 6.

На основі списку L створити два списки: L1 – із елементів з додатними значеннями інформаційного поля та L2 – із елементів з від'ємними значеннями інформаційного поля, зберігши порядок їх розташування.

Варіант 7.

Поміняти місцями елементи списку – змінити порядок розташування на зворотний.

Варіант 8.

Поміняти місцями елементи списку – перший з другим, третій з четвертим і т.д.

Варіант 9.

Обчислити кількість елементів списку з непарними значеннями інформаційного поля.

Варіант 10.

Обчислити кількість елементів списку із заданим користувачем значенням інформаційного поля.

Варіант 11.

Обчислити суму елементів списку із парними значеннями інформаційного поля.

Варіант 12.

Обчислити суму елементів списку із додатними значеннями інформаційного поля.

Варіант 13.

Обчислити кількість елементів списку із від'ємними значеннями інформаційного поля.

Варіант 14.

Збільшити значення інформаційного поля кожного елемента списку на задану користувачем величину.

Варіант 15.

Продублювати кожний елемент списку із заданим користувачем значенням інформаційного поля.

Варіант 16.

Вилучити кожний елемент списку із заданим користувачем значенням інформаційного поля.

Варіант 17.

Вилучити кожний елемент списку, який передує елементу із заданим користувачем значенням інформаційного поля.

Варіант 18.

Вилучити кожний елемент списку, який слідує за елементом із заданим користувачем значенням інформаційного поля.

Варіант 19.

Перед кожним елементом списку із значенням інформаційного поля V1 вставити новий елемент із значенням інформаційного поля V2.

Варіант 20.

Після кожного елементу списку із значенням інформаційного поля V1 вставити новий елемент із значенням інформаційного поля V2.

Варіант 21.

Перевірити, чи елементи списку – впорядковані за не спаданням.

Варіант 22.

Визначити, чи список містить пару сусідніх елементів з однаковими значеннями інформаційного поля.

Варіант 23.*

Визначити, чи список містить пару елементів (не обов'язково сусідніх) з однаковими значеннями інформаційного поля.

Варіант 24.*

Визначити, чи всі елементи списку L1 входять у список L2 (тобто, чи список L2 містить всі елементи списку L1, розташовані у довільному порядку).

Варіант 25.*

Визначити, чи список L1 входить як частина у список L2 (тобто, чи список L2 містить всі елементи списку L1, розташовані у тому ж самому порядку).

Варіант 26.

На основі списку L створити два списки: L1 – із елементів з додатними значеннями інформаційного поля та L2 – із елементів з від'ємними значеннями інформаційного поля, зберігши порядок їх розташування.

Варіант 27.

Поміняти місцями елементи списку – змінити порядок розташування на зворотний.

Варіант 28.

Поміняти місцями елементи списку – перший з другим, третій з четвертим і т.д.

Варіант 29.

Обчислити кількість елементів списку з непарними значеннями інформаційного поля.

Варіант 30.

Обчислити кількість елементів списку із заданим користувачем значенням інформаційного поля.

Варіант 31.

Обчислити суму елементів списку із парними значеннями інформаційного поля.

Варіант 32.

Обчислити суму елементів списку із додатними значеннями інформаційного поля.

Варіант 33.

Обчислити кількість елементів списку із від'ємними значеннями інформаційного поля.

Варіант 34.

Збільшити значення інформаційного поля кожного елемента списку на задану користувачем величину.

Варіант 35.

Продублювати кожний елемент списку із заданим користувачем значенням інформаційного поля.

Варіант 36.

Вилучити кожний елемент списку із заданим користувачем значенням інформаційного поля.

Варіант 37.

Вилучити кожний елемент списку, який передує елементу із заданим користувачем значенням інформаційного поля.

Варіант 38.

Вилучити кожний елемент списку, який слідує за елементом із заданим користувачем значенням інформаційного поля.

Варіант 39.

Перед кожним елементом списку із значенням інформаційного поля V1 вставити новий елемент із значенням інформаційного поля V2.

Варіант 40.

Після кожного елементу списку із значенням інформаційного поля V1 вставити новий елемент із значенням інформаційного поля V2.

Лабораторна робота № 3.5. Стеки

Мета роботи

Навчитися опрацьовувати стеки.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення елемента стеку та вказівника на нього.
- 2) Загальні схеми створення стеку (добавлення елемента), знищення стеку (вилучення елемента).
- 3) Загальні схеми обходу стеку.
- 4) Доступ до елементів стеку.
- 5) Дії із стеком.
- 6) Внутрішня реалізація стеку.
- 7) Передавання та опрацювання стеків у функціях.

Оформлення звіту

Вимоги та зразок оформлення звіту наведені у вступі до лабораторного практикуму.

Варіанти лабораторних завдань

Всі завдання слід реалізувати з використанням списків-стеків. Якщо при виконанні завдання слід утворити нові списки, – вони мають бути тої самої структури.

Якщо у завданні не вказано тип даних інформаційного поля, його слід вибрати самостійно.

Необхідно:

- сформулювати заданий список (списки);
- роздрукувати його (їх) – вивести значення елементів на екран;
- виконати вказані в завданні дії;
- вивести результат на екран.

Всі вказані дії слід виконувати в окремих підпрограмах, які всю інформацію приймають у вигляді параметрів. Використовувати нелокальні змінні не можна.

Кожна функція має виконувати лише одну роль, і ця роль має бути відображена у назві функції.

«Функція, яка повертає / обчислює / шукає ...» – має не виводити ці значення, а повернути їх у місце виклику як результат функції або як відповідний вихідний параметр.

До звіту необхідно подати діаграму змін стану пам'яті при виконанні всіх операцій із списком, вказаних у варіанті завдання.

Варіант 1.

Використовуючи два стеки, упорядкувати послідовність символів, що подаються на вхід. Передбачити введення символів з клавіатури та зчитування з текстового файлу. Вивести неупорядкований та упорядкований стеки. Для створення упорядкованого за значеннями елементів стеку використовувати допоміжний стек.

Варіант 2.

Створити стек цілих чисел. Передбачити введення чисел з клавіатури та зчитування з текстового файлу. Визначити кількість додатних, від'ємних, нульових значень та довжину стеку. Вивести створений стек.

Варіант 3.

Створити стек цілих чисел. Передбачити введення чисел з клавіатури та зчитування з текстового файлу. Створити новий стек інверсією попереднього. Вивести перший та інвертований стеки.

Варіант 4.

Створити два стеки чисел. Передбачити введення чисел з клавіатури та зчитування з текстового файлу. Визначити довжини стеків. Здійснити конкатенацію стеків, створивши новий стек. Вивести на екран значення елементів усіх стеків.

Варіант 5.

Створити два стеки чисел. Передбачити введення чисел з клавіатури та зчитування з текстового файлу. Здійснити порівняння стеків. Якщо стеки однакові, здійснити інверсію одного з них, інакше утворити новий стек з від'ємних елементів двох попередніх стеків. Вивести на екран стеки до перетворення та після.

Варіант 6.

Створити стек чисел, що генеровані генератором випадкових чисел. Упорядкувати стек за зростанням значень елементів, використовуючи для цього допоміжний стек. Вивести на екран стеки до перетворення та після.

Варіант 7.

Задано текстовий файл. Створити стек слів, що є в текстовому файлі. Визначити, скільки разів зустрічається кожне слово в стеку. Вивести на екран елементи стеку та у вікно notepad вміст текстового файлу.

Варіант 8.

Задано текстовий файл, що містить декілька рядків. Створити стек, елементами якого є перші літери слів текстового файлу. Створити новий стек з символів попереднього стеку, упорядкувавши його елементи. Для створення упорядкованого за значеннями елементів стеку використовувати допоміжний стек. Вивести на екран вхідний текст; стек, створений із перших літер слів файлу та упорядкований стек.

Варіант 9.

В текстовому файлі задані арифметичні вирази. Визначити кількість операндів і операцій в арифметичних виразах, використовуючи для цього стек. Вивести на екран значення елементів стеку.

Варіант 10.

В текстовому файлі записаний арифметичний вираз в префіксній формі $-a * +b - c - / da - bcd$. Знайти його значення при заданих a, b, c, d . Використати стекову структуру. Надрукувати елементи стеку під час розрахунку значення виразу.

Варіант 11.

В текстовому файлі записаний арифметичний вираз в постфіксній формі $abcd a + cd - a // * - +$. Знайти його значення при заданих a, b, c, d . Використати стекову структуру. Надрукувати елементи стеку під час розрахунку значення виразу.

Варіант 12.

Текстовий файл містить текст із однаковою кількістю дужок, що відкриваються та закриваються. Побудувати стек, елементами якого є частини тексту, що знаходяться між парою дужок. Надрукувати значення елементів стеку та номери позицій в тексті кожної пари дужок, що відкриваються та закриваються, наприклад: 8 10; 12 16; і т.д.

Варіант 13.

Текстовий файл містить арифметичний вираз, записаний у звичайній інфікській формі. Перекласти вираз із звичайної форми запису в постфіксну і записати її в текстовий файл. Використати стек. Наприклад, інфіксна форма виразу: $a-b$, $a*b$; постфіксна форма: $ab-$, ab^* .

Варіант 14.

Побудувати стек, елементами якого є слова тексту, що зчитуються з текстового файлу. Слова відокремлюються одне від одного комами, пробілами та іншими розділовими символами. Створити новий стек, в якому містяться слова заданої довжини, що вибрані з першого стеку. Вивести вміст першого та другого стеків.

Варіант 15.

З текстового файлу зчитати натуральне число n та дійсні числа $x_1, \dots, x_n, n \geq 2$. Побудувати стек, що містить значення $x_i - x_n, i = \overline{1, n}$. Серед значень стеку визначити найбільше та найменше та їх позиції в стеку. Вивести на екран значення елементів стеку.

Варіант 16.

Побудувати стек натуральних чисел, що зчитані з текстового файлу. Вивести на екран створений стек. Надрукувати в зворотному порядку числа стеку, пропускаючи числа, які кратні введеному з клавіатури числу.

Варіант 17.

Побудувати стек, елементами якого є слова, зчитані з текстового файлу. Вивести на екран вміст стеку. Надрукувати всі слова стеку, що починаються та закінчуються заданими користувачем літерами.

Варіант 18.

Побудувати стек слів, що зчитані з текстового файлу. Вивести на екран вміст стеку. Визначити довжину стеку, кількість повторень однакових слів, найдовше та найкоротше слова та їх порядкові номери.

Варіант 19.

Створити стек слів, що зчитані з текстового файлу. Розбити стек на два стеки рівної довжини, якщо кількість слів в стеку парна, інакше створити дублікат стеку. Вивести на екран створені стеки.

Варіант 20.

Із текстового файлу вибрати слова, які утворюють таку послідовність, що перша літера одного слова збігається з останньою літерою іншого слова. Вибрані слова записати до стеку, надрукувати його вміст та найдовше слово.

Варіант 21.

Створити стек з натуральних чисел, зчитавши їх значення з текстового файлу. Ввести з клавіатури два натуральних числа n, m . Групу з n елементів стеку скопіювати в новий стек. Групу з m верхніх елементів стеку вилучити з нього. Надрукувати вхідний та вихідні стеки.

Варіант 22.

Створити стек з натуральних чисел, зчитавши їх значення з текстового файлу. Визначити значення елемента, що міститься у заданій з клавіатури позиції стеку. Здійснити пошук заданого з клавіатури значення елемента в стеку. Якщо елемент не знайдено, вивести відповідне повідомлення.

Варіант 23.

Створити стек з чисел, зчитавши їх значення з текстового файлу. Визначити, чи можна з кожної трійки чисел утворити прямокутний трикутник, вважаючи числа стеку довжинами сторін трикутника. Якщо можна, обчислити та записати в стек радіуси вписаних кіл.

Варіант 24.

Створити стек з чисел, зчитавши їх значення з текстового файлу. Розрахувати їх середнє арифметичне та середнє геометричне. Надрукувати вміст стеку, визначити кількість його елементів.

Варіант 25.

У текстовому файлі записані координати $x_i, y_i, i = \overline{1, n}, n > 3$ матеріальних точок. Створити стек, що містить зчитані з текстового файлу координати точок. Розрахувати центр мас матеріальних точок, заданих своїми координатами. Вивести на екран вміст стеку.

Варіант 26.

Створити програму демонстрації операцій обробки стеку: перевірка стеку на пустоту, друк (перегляд) стека; додавання елемента у вершину стека; витягання елемента з вершини стека; пошук завданого значення в стеку, очищення стеку.

Варіант 27.

В кожному рядку текстового файлу записано по два числа. Перше число визначає номер стека, друге число – значення елемента стека. Створити стеки відповідно до значень, заданих у текстовому файлі, визначити кількість елементів в кожному стеку та надрукувати їх.

Варіант 28.

Використовуючи два стеки, упорядкувати послідовність символів, що подаються на вхід. Передбачити введення символів з клавіатури та зчитування з текстового файлу. Вивести неупорядкований та упорядкований стеки. Для створення упорядкованого за значеннями елементів стеку використовувати допоміжний стек.

Варіант 29.

Створити стек цілих чисел. Передбачити введення чисел з клавіатури та зчитування з текстового файлу. Визначити кількість додатних, від'ємних, нульових значень та довжину стеку. Вивести створений стек.

Варіант 30.

Створити стек цілих чисел. Передбачити введення чисел з клавіатури та зчитування з текстового файлу. Створити новий стек інверсією попереднього. Вивести перший та інвертований стеки.

Варіант 31.

Створити два стеки чисел. Передбачити введення чисел з клавіатури та зчитування з текстового файлу. Визначити довжини стеків. Здійснити конкатенацію стеків, створивши новий стек. Вивести на екран значення елементів усіх стеків.

Варіант 32.

Створити два стеки чисел. Передбачити введення чисел з клавіатури та зчитування з текстового файлу. Здійснити порівняння стеків. Якщо стеки однакові, здійснити інверсію

одного з них, інакше утворити новий стек з від'ємних елементів двох попередніх стеків. Вивести на екран стеки до перетворення та після.

Варіант 33.

Створити стек чисел, що генеровані генератором випадкових чисел. Упорядкувати стек за зростанням значень елементів, використовуючи для цього допоміжний стек. Вивести на екран стеки до перетворення та після.

Варіант 34.

Задано текстовий файл. Створити стек слів, що є в текстовому файлі. Визначити, скільки разів зустрічається кожне слово в стеку. Вивести на екран елементи стеку та у вікно `notepad` вміст текстового файлу.

Варіант 35.

Задано текстовий файл, що містить декілька рядків. Створити стек, елементами якого є перші літери слів текстового файлу. Створити новий стек з символів попереднього стеку, упорядкувавши його елементи. Для створення упорядкованого за значеннями елементів стеку використовувати допоміжний стек. Вивести на екран вхідний текст; стек, створений із перших літер слів файлу та упорядкований стек.

Варіант 36.

В текстовому файлі задані арифметичні вирази. Визначити кількість операндів і операцій в арифметичних виразах, використовуючи для цього стек. Вивести на екран значення елементів стеку.

Варіант 37.

В текстовому файлі записаний арифметичний вираз в префіксній формі $-a * +b - c - / da - bcd$. Знайти його значення при заданих a, b, c, d . Використати стекову структуру. Надрукувати елементи стеку під час розрахунку значення виразу.

Варіант 38.

В текстовому файлі записаний арифметичний вираз в постфіксній формі $abcd a + cd - a // * - +$. Знайти його значення при заданих a, b, c, d . Використати стекову структуру. Надрукувати елементи стеку під час розрахунку значення виразу.

Варіант 39.

Текстовий файл містить текст із однаковою кількістю дужок, що відкриваються та закриваються. Побудувати стек, елементами якого є частини тексту, що знаходяться між парою дужок. Надрукувати значення елементів стеку та номери позицій в тексті кожної пари дужок, що відкриваються та закриваються, наприклад: 8 10;12 16; і т.д.

Варіант 40.

Текстовий файл містить арифметичний вираз, записаний у звичайній інфікській формі. Перекласти вираз із звичайної форми запису в постфіксну і записати її в текстовий файл. Використати стек. Наприклад, інфіксна форма виразу: $a-b$, $a*b$; постфіксна форма: $ab-$, ab^* .

Лабораторна робота № 3.6. Черги

Мета роботи

Навчитися опрацьовувати черги.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення елемента черги та вказівника на нього.
- 2) Загальні схеми створення черги (добавлення елемента), знищення черги (вилучення елемента).
- 3) Загальні схеми обходу черги.
- 4) Доступ до елементів черги.
- 5) Дії із чергою.
- 6) Внутрішня реалізація черги.
- 7) Передавання та опрацювання черг у функціях.

Оформлення звіту

Вимоги та зразок оформлення звіту наведені у вступі до лабораторного практикуму.

Варіанти лабораторних завдань

Всі завдання слід реалізувати з використанням списків-черг. Якщо при виконанні завдання слід утворити нові списки, – вони мають бути тої самої структури.

Якщо у завданні не вказано тип даних інформаційного поля, його слід вибрати самостійно.

Необхідно:

- сформулювати заданий список (списки);
- роздрукувати його (їх) – вивести значення елементів на екран;
- виконати вказані в завданні дії;
- вивести результат на екран.

Всі вказані дії слід виконувати в окремих підпрограмах, які всю інформацію приймають у вигляді параметрів. Використовувати нелокальні змінні не можна.

Кожна функція має виконувати лише одну роль, і ця роль має бути відображена у назві функції.

«Функція, яка повертає / обчислює / шукає ...» – має не виводити ці значення, а повернути їх у місце виклику як результат функції або як відповідний вихідний параметр.

До звіту необхідно подати діаграму змін стану пам'яті при виконанні всіх операцій із списком, вказаних у варіанті завдання.

Варіант 1.

Дано набір з 10 чисел. Створити дві черги: перша має містити числа з початкового набору з непарними номерами (1, 3, ..., 9), а друга – з парними (2, 4, ..., 10); порядок чисел в кожній черзі має збігатися з порядком чисел у початковому наборі. Вивести вказівники на початок і кінець кожної з отриманих черг.

Варіант 2.

Побудувати чергу, елементами якої є числа, що зчитані з текстового файлу. За допомогою генератора випадкових чисел генеруються числа 0 або 1. Якщо генерується число 0, з черги видаляється елемент та виводиться на екран. Якщо генерується число 1, до черги додається елемент, значення якого вводиться з клавіатури. Надрукувати вхідну чергу та результати її перетворення.

Варіант 3.

Надрукувати в порядку зростання перші n натуральних чисел, в розкладання яких на прості множники входять тільки числа 2, 3, 5. Використати черги для збереження множників.

Варіант 4.

Створити чергу з символів рядка, зчитаного з текстового файлу. Вибрати з черги англійські символи та записати їх у нову чергу. Усі інші символи, зберігши їх порядок, записати у третю чергу. Вивести на екран вміст усіх черг.

Варіант 5.*

Використовуючи динамічну структуру черги, промодельовати процес обслуговування черги покупців в магазині. У магазині стоїть черга з n покупців. Час обслуговування покупця з черги – випадкове ціле число в діапазоні від 1 до t_1 . Час додавання нового покупця до черги - випадкове ціле число в діапазоні від 1 до t_2 . Промодельовати стан черги (тобто показати час виникнення подій – обслуговування та додавання покупця) за період часу T ($T > t_1, T > t_2$). Вивести на екран залишок черги.

Варіант 6.

Створити чергу із слів, зчитаних з текстового файлу. Здійснити інверсію черги та виведення заданої кількості елементів черги. Вивести на екран результати перетворення вхідної черги.

Варіант 7.

В текстовому файлі задані два рядки. Створити дві черги із слів, визначених двома рядками текстового файлу. Вивести на екран вміст черг. Здійснити конкатенацію двох черг, утворивши третю чергу. Виконати порівняння двох перших черг з виведення відповідних повідомлень.

Варіант 8.

Створити телефонний довідник у вигляді черги, елементами якої є дані про абонента телефонної компанії (прізвище, номер телефону). Здійснити пошук абонента за його прізвищем та за номером. У разі відсутності даних вивести відповідне повідомлення. Надрукувати телефонний довідник у вигляді таблиці.

Варіант 9.

Створити чергу з елементів, значення яких зчитані з текстового файлу. Здійснити перевірку черги на пустоту, копіювання черги, визначення кількості її елементів. Вивести вхідну та вихідну черги та відповідні повідомлення.

Варіант 10.

Створити чергу з чисел, значення яких зчитані з текстового файлу. Надрукувати елементи черги. Визначити мінімальне, максимальне значення та їх місцезнаходження в черзі. Знайти середнє арифметичне мінімального та максимального значень черги.

Варіант 11.

Створити чергу з чисел, значення яких зчитані з текстового файлу. Надрукувати елементи черги. Здійснити сортування елементів черги, використавши для цього допоміжну чергу або стек. Надрукувати відсортовану чергу.

Варіант 12.

В текстовому файлі задані назви задач та їх пріоритети. Створити чергу елементами якої є назви задач та їх пріоритети. Першими елементами черги є задачі з вищими

пріоритетами. Задачі з меншими пріоритетами слід записати в кінець черги. Перевірити чергу на пустоту та надрукувати вміст файлу та черги.

Варіант 13.

Створити чергу з чисел, значення яких зчитані з текстового файлу. Надрукувати елементи черги. Здійснити пошук елемента, значення якого вводиться з клавіатури. У разі успішного пошуку поділити чергу на дві. Першим елементом другої черги має стати знайдений елемент. Вивести на екран отримані черги.

Варіант 14.*

Промодельовати процес посадки в літак, що розрахований на 100 місць. В черзі на посадку стоять 100 пасажирів. Першою входить старий чоловік. Зайшовши в салон, він сідає на будь-яке випадково вибране місце. Кожний з наступних пасажирів, зайшовши в салон, сідає на своє (позначене в білеті) місце, якщо воно вільне, і на будь-яке з вільних – в протилежному випадку. Яка ймовірність, що останній в черзі пасажир сяде на своє місце?

Варіант 15.*

Промодельовати процес обслуговування пацієнтів лікарем. Лікар приймає кожного пацієнта 15 хвилин. До нього прийшли 4 пацієнти з інтервалом в 5 хвилин. Який максимальний час сидіння пацієнтів в черзі? Яка максимальна кількість людина виявиться в черги протягом робочого дня? Вивести на екран повідомлення про процес, що моделюється, та розраховані значення. Елементи черги зобразити значеннями про пацієнтів (прізвище, діагноз, час перебування у черзі).

Варіант 16.

Створити програму демонстрації операцій обробки черги: перевірка черги на пустоту, друк (перегляд) черги, додавання елемента в кінець черги; вибір елемента з початку черги, пошук заданого значення в черзі, очищення черги.

Варіант 17.

Створити чергу із слів, які зчитані з текстового файлу. Надрукувати елементи черги. Знайти найдовше та найкоротше слово в черзі та їх порядкові номери. Скопіювати частину черги між знайденими елементами. Вивести на екран отриману чергу.

Варіант 18.

Створити чергу із слів, які зчитані з текстового файлу. Надрукувати елементи черги. Визначити кількість її елементів Вивести на екран слова, довжина яких відповідає заданому з клавіатури значенню. Знищити чергу на вимогу користувача.

Варіант 19.

Текстовий файл містить декілька рядків. З елементів (слів) парних рядків створити одну чергу. З елементів (слів) непарних рядків – іншу чергу. Порівняти черги. Якщо вони однакові, то знищити одну чергу, інакше об'єднати початкові черги в третю чергу та надрукувати її елементи.

Варіант 20.

Сформувати дві черги по n елементів, зчитавши дані з текстового файлу. Об'єднати черги в одну, в якій елементи початкових черг чергуються, починаючи з першого елементу першої черги. Вивести на екран початкові та вихідну черги.

Варіант 21.

Створити чергу повідомлень, що зчитуються з текстового файлу. Кожному повідомленню присвоїти пріоритет, що генерується генератором псевдовипадкових чисел. Перебудувати чергу повідомлень відповідно до отриманих пріоритетів. Надрукувати початкову та нову черги. Здійснити вибір елементів із заданим пріоритетом.

Варіант 22.

Реалізувати чергу за допомогою масиву, списку та двох стеків. Порівняти швидкодію виконання операцій додавання, вилучення та пошуку елементів з черги. Надрукувати відповідні черги.

Варіант 23.*

Написати програму для моделювання роботи черги з випадковим числом елементів, що додаються і видаляються. Елементами черги є латинські літери. Нехай за одиницю часу в чергу або додається, або вилучається випадкове число елементів в кількості від 1 до 3-х. Одночасно додавання і видалення не відбувається. Тип операції з чергою (додавання або видалення) задається випадково. Це призводить до того, що черга випадково росте і зменшується. Програма має наочно показувати стан черги в кожен момент часу.

Варіант 24.

Створити циклічну чергу, в якій елемент, що її покидає, переводиться у кінець черги. Значення елементів зчитуються з текстового файлу. Тип елементів задає користувач.

Варіант 25.*

Сто чоловік стоять у черзі так, що кожен бачить тільки тих, хто стоять перед ним, але чує усе, що говорять інші. На кожну людину одягають ковпак одного з трьох кольорів (скажімо, червоного, жовтого або зеленого), колір, зрозуміло, йому невідомий. Починаючи з останнього в черзі (того, хто бачить усіх), у кожної людини запитують, якого кольору у неї ковпак. Якщо вгадує – його залишають в черзі, помиляється – його вилучають. Необхідно придумати, як домовитися цим 100 людям, щоб мінімізувати жертви.

Варіант 26.

Дано набір з 10 чисел. Створити дві черги: перша має містити числа з початкового набору з непарними номерами (1, 3, ..., 9), а друга – з парними (2, 4, ..., 10); порядок чисел в кожній черзі має збігатися з порядком чисел у початковому наборі. Вивести вказівники на початок і кінець кожної з отриманих черг.

Варіант 27.

Побудувати чергу, елементами якої є числа, що зчитані з текстового файлу. За допомогою генератора випадкових чисел генеруються числа 0 або 1. Якщо генерується число 0, з черги видаляється елемент та виводиться на екран. Якщо генерується число 1, до черги додається елемент, значення якого вводиться з клавіатури. Надрукувати вхідну чергу та результати її перетворення.

Варіант 28.

Надрукувати в порядку зростання перші n натуральних чисел, в розкладання яких на прості множники входять тільки числа 2, 3, 5. Використати черги для збереження множників.

Варіант 29.

Створити чергу з символів рядка, зчитаного з текстового файлу. Вибрати з черги англійські символи та записати їх у нову чергу. Усі інші символи, зберігши їх порядок, записати у третю чергу. Вивести на екран вміст усіх черг.

Варіант 30.*

Використовуючи динамічну структуру черги, промодельовати процес обслуговування черги покупців в магазині. У магазині стоїть черга з n покупців. Час обслуговування покупця з черги – випадкове ціле число в діапазоні від 1 до t_1 . Час додавання нового покупця до черги - випадкове ціле число в діапазоні від 1 до t_2 . Промодельовати стан черги (тобто показати час виникнення подій – обслуговування та додавання покупця) за період часу T ($T > t_1, T > t_2$). Вивести на екран залишок черги.

Варіант 31.

Створити чергу із слів, зчитаних з текстового файлу. Здійснити інверсію черги та виведення заданої кількості елементів черги. Вивести на екран результати перетворення вхідної черги.

Варіант 32.

В текстовому файлі задані два рядки. Створити дві черги із слів, визначених двома рядками текстового файлу. Вивести на екран вміст черг. Здійснити конкатенацію двох черг, утворивши третю чергу. Виконати порівняння двох перших черг з виведення відповідних повідомлень.

Варіант 33.

Створити телефонний довідник у вигляді черги, елементами якої є дані про абонента телефонної компанії (прізвище, номер телефону). Здійснити пошук абонента за його прізвищем та за номером. У разі відсутності даних вивести відповідне повідомлення. Надрукувати телефонний довідник у вигляді таблиці.

Варіант 34.

Створити чергу з елементів, значення яких зчитані з текстового файлу. Здійснити перевірку черги на пустоту, копіювання черги, визначення кількості її елементів. Вивести вхідну та вихідну черги та відповідні повідомлення.

Варіант 35.

Створити чергу з чисел, значення яких зчитані з текстового файлу. Надрукувати елементи черги. Визначити мінімальне, максимальне значення та їх місцезнаходження в черзі. Знайти середнє арифметичне мінімального та максимального значень черги.

Варіант 36.

Створити чергу з чисел, значення яких зчитані з текстового файлу. Надрукувати елементи черги. Здійснити сортування елементів черги, використавши для цього допоміжну чергу або стек. Надрукувати відсортовану чергу.

Варіант 37.

В текстовому файлі задані назви задач та їх пріоритети. Створити чергу елементами якої є назви задач та їх пріоритети. Першими елементами черги є задачі з вищими пріоритетами. Задачі з меншими пріоритетами слід записати в кінець черги. Перевірити чергу на пустоту та надрукувати вміст файлу та черги.

Варіант 38.

Створити чергу з чисел, значення яких зчитані з текстового файлу. Надрукувати елементи черги. Здійснити пошук елемента, значення якого вводиться з клавіатури. У разі успішного пошуку поділити чергу на дві. Першим елементом другої черги має стати знайдений елемент. Вивести на екран отримані черги.

Варіант 39.*

Промодельовати процес посадки в літак, що розрахований на 100 місць. В черзі на посадку стоять 100 пасажирів. Першою входить старий чоловік. Зайшовши в салон, він сідає на будь-яке випадково вибране місце. Кожний з наступних пасажирів, зайшовши в салон, сідає на своє (позначене в білеті) місце, якщо воно вільне, і на будь-яке з вільних – в протилежному випадку. Яка ймовірність, що останній в черзі пасажир сяде на своє місце?

Варіант 40.*

Промодельовати процес обслуговування пацієнтів лікарем. Лікар приймає кожного пацієнта 15 хвилин. До нього прийшли 4 пацієнти з інтервалом в 5 хвилин. Який максимальний час сидіння пацієнтів в черзі? Яка максимальна кількість людина виявиться в черги протягом робочого дня? Вивести на екран повідомлення про процес, що моделюється, та розраховані значення. Елементи черги зобразити значеннями про пацієнтів (прізвище, діагноз, час перебування у черзі).

Лабораторна робота № 3.7. Однозв'язні списки

Мета роботи

Навчитися опрацьовувати однонаправлені списки.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення елемента однонаправленого списку та вказівника на нього.
- 2) Загальні схеми створення однонаправленого списку (добавлення елемента), знищення однонаправленого списку (вилучення елемента).
- 3) Довільні однонаправлені списки та впорядковані однонаправлені списки (списки з пріоритетами).
- 4) Загальні схеми обходу однонаправленого списку.
- 5) Доступ до елементів однонаправленого списку.
- 6) Дії із однонаправленим списком.
- 7) Внутрішня реалізація однонаправленого списку.
- 8) Передавання та опрацювання однонаправлених списків у функціях.

Варіанти лабораторних завдань

Всі завдання слід реалізувати з використанням однонаправлених списків. Якщо при виконанні завдання слід утворити нові списки, – вони мають бути тої самої структури.

Якщо у завданні не вказано тип даних інформаційного поля, його слід вибрати самостійно. Необхідно:

- сформувати заданий список (списки);
- роздрукувати його (їх) – вивести значення елементів на екран;
- виконати вказані в завданні дії;
- вивести результат на екран.

Всі вказані дії слід виконувати в окремих підпрограмах, які всю інформацію приймають у вигляді параметрів. Використовувати нелокальні змінні не можна.

Кожна функція має виконувати лише одну роль, і ця роль має бути відображена у назві функції.

«Функція, яка повертає / обчислює / шукає ...» – має не виводити ці значення, а повернути їх у місце виклику як результат функції або як відповідний вихідний параметр.

До звіту необхідно подати діаграму змін стану пам'яті при виконанні всіх операцій із списком, вказаних у варіанті завдання.

Варіант 1.

З введеного з клавіатури або зчитаного з файлу рядка утворити послідовність слів. Побудувати однозв'язний список, у якому кожне наступне слово починається на літеру, що завершує попереднє слово. Якщо такий зв'язок слів зробити неможливо, побудувати список, у якому слова розташовані за алфавітом. Надрукувати створений список.

Варіант 2.

З введеного з клавіатури або зчитаного з файлу рядка утворити послідовність чисел. Побудувати однозв'язний список, у якому числа впорядковані за зростанням. Надрукувати впорядкований список. Вилучити із списку всі від'ємні елементи та надрукувати їх.

Варіант 3.

Задано дві послідовності цілих і дійсних чисел. Побудувати однозв'язний список, у якому числа послідовностей чергуються. Якщо останнім елементом списку є ціле число, перенести його на початок списку. Надрукувати списки до та після зміни.

Варіант 4.

Побудувати однозв'язний список з елементами, значення яких зчитані з текстового файлу і являють собою номери вершин графа та довжину ребер між ними. Визначити кількість елементів списку. Знайти елемент із заданим значенням та надрукувати його порядковий номер в списку.

Варіант 5.

Побудувати однозв'язний список, значення елементів якого зчитані з текстового файлу або введені з клавіатури. Здійснити видалення елемента за його значенням та за його порядковим номером у списку. У разі відсутності елемента із заданим значенням видати відповідне повідомлення. Надрукувати вхідний та вихідний списки.

Варіант 6.

Побудувати однозв'язний список, елементи якого є цілими числами, що зчитані з текстового файлу або введені з клавіатури. Визначити елементи списку, що є квадратами та кубами чисел. Надрукувати вхідний та вихідний списки.

Варіант 7.

Побудувати однозв'язний список, елементи якого є цілими числами, що зчитані з текстового файлу або введені з клавіатури. Визначити довжину списку. Здійснити інверсію списку та пошук елементів, значення яких є числами Фібоначчі. Надрукувати вхідний та вихідний список та знайдені значення.

Варіант 8.

Побудувати однозв'язний список, елементи якого є цілими числами, що зчитані з текстового файлу або введені з клавіатури. Замінити усі входження елемента m значеннями n , заданими з клавіатури. Надрукувати початковий та отриманий списки.

Варіант 9.

Створити кільцевий список, елементами якого є числа. Послідовно вилучати кожне третє число. Підрахувати кількість вилучених чисел. Вивести початковий список та елементи, що вилучаються.

Варіант 10.

Побудувати два однозв'язних списків, значення елементів яких зчитані з текстового файлу або введені з клавіатури. Порівняти списки. Якщо списки не однакові, виконати їх конкатенацію, інакше утворити новий список, інформаційні поля якого містять значення елементів одного списку та кількість повторів однакових елементів. Надрукувати вхідні та вихідні списки.

Варіант 11.

Побудувати два однозв'язних списків, значення елементів яких зчитані з текстового файлу або введені з клавіатури. Об'єднати два списки в результуючий список, в якому чергуються елементи початкових списків. Визначити довжину списку, серединний елемент та його порядковий номер.

Варіант 12.

Побудувати однозв'язний список, значення елементів якого зчитані з текстового файлу або введені з клавіатури. Переписати у новий список підсписок, починаючи з елемента з номером N і закінчуючи елементом $N + K$, де номери елементів задаються з клавіатури. Надрукувати список та підсписок

Варіант 13.

Побудувати два однозв'язних списків, значення елементів яких зчитані з текстового файлу або введені з клавіатури. Визначити теоретико-множинне об'єднання, різницю та перетин двох списків. Надрукувати вхідний та вихідні списки.

Варіант 14.

Побудувати однозв'язний список, елементи якого є цілими числами, що зчитані з текстового файлу або введені з клавіатури. Видалити зі списку усі парні значення, значення з непарними номерами подвоїти, вставити в список у задану позицію підсписок із заданою кількістю елементів. Надрукувати вхідний та вихідний списки.

Варіант 15.

Побудувати два однозв'язних списків. Значеннями елементів першого списку є рядки, що зчитані з текстового файлу. Значеннями елементів другого списку є натуральні числа, що введені з клавіатури. Видалити з першого списку елементи, порядкові номери яких задані в другому списку. Надрукувати вхідні та вихідні списки.

Варіант 16.

Побудувати однозв'язний список, значеннями елементів якого є елементи матриці, що зчитані з текстового файлу або введені з клавіатури. Побудувати новий список, елементами якого є елементи транспонованої матриці. Виділити діагональні елементи матриці та записати їх у новий список. Надрукувати вхідний та вихідні списки.

Варіант 17.

Створити однозв'язний список абонентів телефонної мережі. Елементи списку містять дані про прізвище, номер телефону та заборгованість з платежу. Визначити абонентів, що мають заборгованість вище заданого користувачем значення, та перевисати їх у новий список. Надрукувати створені списки.

Варіант 18.

Створити однозв'язний список студентів групи. Елементи списку містять дані про прізвище, середній бал та стипендію. Визначити студентів із заданим середнім балом. Вилучити студентів, що не мають стипендії. Надрукувати вхідний та вихідний списки.

Варіант 19.

Створити однозв'язний список потягів, що прибувають на вокзал. Елементи списку

містять дані про номер потягу та час його прибуття. Визначити потяги, що прибувають у заданий користувачем діапазон часу, та переписати дані про них у новий список, додавши до кожного запису номер колії. Надрукувати створені списки.

Варіант 20.

Створити однозв'язний список справ, які людина має виконати. Елементи списку містять дані про назву роботи, термін та результат її виконання. Визначити роботи, які не були виконані, переписати їх у новий список, призначити їм нові терміни виконання та штрафи за порушення планових термінів. Надрукувати вхідний та вихідний список.

Варіант 21.

Побудувати однозв'язний список абітурієнтів, які подали документи до приймальної комісії ВНЗ. Інформаційне поле елементів списку містить дані про прізвище абітурієнта, бал сертифікату, тип документу про середню освіту (дублікат чи оригінал). Передбачити зчитування даних з файлу або введення з клавіатури. Утворити новий список з даними про абітурієнтів, що рекомендовані до зарахування за прохідним балом, значення якого користувач вводить з клавіатури. Надрукувати вхідний та вихідний списки.

Варіант 22.

Побудувати однозв'язний список підручників, що подані на здобуття Державної премії України. Інформаційне поле елементів списку містить дані про прізвище автора, назву підручника, кількість схвальних відгуків на його підтримку, які визначаються на етапі голосування в Комітеті з Державних премій. Передбачити зчитування даних з файлу або введення з клавіатури. Утворити новий список з даними про підручники, які рекомендовані до розгляду на заключному засіданні Комітету з Державних премій за критерієм кількості схвальних відгуків, заданої користувачем з клавіатури. Надрукувати вхідний та вихідний списки.

Варіант 23.

Побудувати однозв'язний список команд, які беруть участь у змаганнях. Інформаційне поле елементів списку містить дані про назву команди, кількість балів, що отримала команда. Передбачити зчитування даних з файлу або введення з клавіатури. Змагання проходить у декілька етапів. На кожному етапі вилучається половина команд, як беруть участь у змаганні в поточному етапі. Продемонструвати хід змагань, виводивши на екран дані про команди, що вилучаються, та ті, що залишаються. Обчислити кількість етапів до визначення переможця змагань.

Варіант 24.

Побудувати однозв'язний список, який моделює процес посадки в літак, що розрахований на 100 місць. В черзі на посадку стоять 100 пасажирів. Першою входить старий чоловік. Зайшовши в салон, він сідає на будь-яке випадково вибране місце. Кожний з наступних пасажирів, зайшовши в салон, сідає на своє (позначене в білеті) місце, якщо воно вільне, і на будь-яке з вільних – в протилежному випадку. Яка ймовірність, що останній в черзі пасажир сяде на своє місце?

Варіант 25.

Побудувати однозв'язний список, який моделює процес обслуговування пацієнтів лікарем. Лікар приймає кожного пацієнта 15 хвилин. До нього прийшли 4 пацієнти з інтервалом в 5 хвилин. Який максимальний час сидіння пацієнтів в черзі? Яка максимальна кількість людина виявиться в черги протягом робочого дня? Вивести на екран повідомлення про процес, що моделюється, та розраховані значення. Елементи черги зобразити значеннями про пацієнтів (прізвище, діагноз, час перебування у черзі).

Варіант 26.

З введеного з клавіатури або зчитаного з файлу рядка утворити послідовність слів. Побудувати однозв'язний список, у якому кожне наступне слово починається на літеру, що завершує попереднє слово. Якщо такий зв'язок слів зробити неможливо, побудувати список, у якому слова розташовані за алфавітом. Надрукувати створений список.

Варіант 27.

З введеного з клавіатури або зчитаного з файлу рядка утворити послідовність чисел. Побудувати однозв'язний список, у якому числа впорядковані за зростанням. Надрукувати впорядкований список. Вилучити із списку всі від'ємні елементи та надрукувати їх.

Варіант 28.

Задано дві послідовності цілих і дійсних чисел. Побудувати однозв'язний список, у якому числа послідовностей чергуються. Якщо останнім елементом списку є ціле число, перенести його на початок списку. Надрукувати списки до та після зміни.

Варіант 29.

Побудувати однозв'язний список з елементами, значення яких зчитані з текстового файлу і являють собою номери вершин графа та довжину ребер між ними. Визначити

кількість елементів списку. Знайти елемент із заданим значенням та надрукувати його порядковий номер в списку.

Варіант 30.

Побудувати однозв'язний список, значення елементів якого зчитані з текстового файлу або введені з клавіатури. Здійснити видалення елемента за його значенням та за його порядковим номером у списку. У разі відсутності елемента із заданим значенням видати відповідне повідомлення. Надрукувати вхідний та вихідний списки.

Варіант 31.

Побудувати однозв'язний список, елементи якого є цілими числами, що зчитані з текстового файлу або введені з клавіатури. Визначити елементи списку, що є квадратами та кубами чисел. Надрукувати вхідний та вихідний списки.

Варіант 32.

Побудувати однозв'язний список, елементи якого є цілими числами, що зчитані з текстового файлу або введені з клавіатури. Визначити довжину списку. Здійснити інверсію списку та пошук елементів, значення яких є числами Фібоначчі. Надрукувати вхідний та вихідний список та знайдені значення.

Варіант 33.

Побудувати однозв'язний список, елементи якого є цілими числами, що зчитані з текстового файлу або введені з клавіатури. Замінити усі входження елемента m значеннями n , заданими з клавіатури. Надрукувати початковий та отриманий списки.

Варіант 34.

Створити кільцевий список, елементами якого є числа. Послідовно вилучати кожне третє число. Підрахувати кількість вилучених чисел. Вивести початковий список та елементи, що вилучаються.

Варіант 35.

Побудувати два однозв'язних списків, значення елементів яких зчитані з текстового файлу або введені з клавіатури. Порівняти списки. Якщо списки не однакові, виконати їх конкатенацію, інакше утворити новий список, інформаційні поля якого містять значення елементів одного списку та кількість повторів однакових елементів. Надрукувати вхідні та вихідні списки.

Варіант 36.

Побудувати два однозв'язних списків, значення елементів яких зчитані з текстового файлу або введені з клавіатури. Об'єднати два списки в результуючий список, в якому чергуються елементи початкових списків. Визначити довжину списку, серединний елемент та його порядковий номер.

Варіант 37.

Побудувати однозв'язний список, значення елементів якого зчитані з текстового файлу або введені з клавіатури. Переписати у новий список підсписок, починаючи з елемента з номером N і закінчуючи елементом $N + K$, де номери елементів задаються з клавіатури. Надрукувати список та підсписок

Варіант 38.

Побудувати два однозв'язних списків, значення елементів яких зчитані з текстового файлу або введені з клавіатури. Визначити теоретико-множинне об'єднання, різницю та перетин двох списків. Надрукувати вхідний та вихідні списки.

Варіант 39.

Побудувати однозв'язний список, елементи якого є цілими числами, що зчитані з текстового файлу або введені з клавіатури. Видалити зі списку усі парні значення, значення з непарними номерами подвоїти, вставити в список у задану позицію підсписок із заданою кількістю елементів. Надрукувати вхідний та вихідний списки.

Варіант 40.

Побудувати два однозв'язних списків. Значеннями елементів першого списку є рядки, що зчитані з текстового файлу. Значеннями елементів другого списку є натуральні числа, що введені з клавіатури. Видалити з першого списку елементи, порядкові номери яких задані в другому списку. Надрукувати вхідні та вихідні списки.

Лабораторна робота № 3.8. Двозв'язні списки

Мета роботи

Навчитися опрацьовувати двонаправлені списки.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення елемента двонаправленого списку та вказівників на нього.
- 2) Загальні схеми створення двонаправленого списку (добавлення елемента), знищення двонаправленого списку (вилучення елемента).
- 3) Довільні двонаправлені списки та впорядковані двонаправлені списки (списки з пріоритетами).
- 4) Загальні схеми обходу двонаправленого списку.
- 5) Доступ до елементів двонаправленого списку.
- 6) Дії із двонаправленим списком.
- 7) Внутрішня реалізація двонаправленого списку.
- 8) Передавання та опрацювання двонаправлених списків у функціях.

Варіанти лабораторних завдань

Всі завдання слід реалізувати з використанням двонаправлених списків. Якщо при виконанні завдання слід утворити нові списки, – вони мають бути тої самої структури.

Якщо у завданні не вказано тип даних інформаційного поля, його слід вибрати самостійно. Необхідно:

- сформувати заданий список (списки);
- роздрукувати його (їх) – вивести значення елементів на екран;
- виконати вказані в завданні дії;
- вивести результат на екран.

Всі вказані дії слід виконувати в окремих підпрограмах, які всю інформацію приймають у вигляді параметрів. Використовувати нелокальні змінні не можна.

Кожна функція має виконувати лише одну роль, і ця роль має бути відображена у назві функції.

«Функція, яка повертає / обчислює / шукає ...» – має не виводити ці значення, а повернути їх у місце виклику як результат функції або як відповідний вихідний параметр.

До звіту необхідно подати діаграму змін стану пам'яті при виконанні всіх операцій із списком, вказаних у варіанті завдання.

Варіант 1.

Побудувати двозв'язний список із слів, зчитаних з рядків текстового файлу. Надрукувати створений список. Ввести з клавіатури слово та ціле число, що визначатиме порядковий номер елементу в списку. Поділити список на дві частини, зробивши введений елемент останнім в одному списку і першим в другому. Надрукувати результати поділу списку.

Варіант 2.

Створити двозв'язний список, елементами якого є слова текстового файлу. Вивести слова, що стоять на парних місцях при проході по списку в одному напрямку, та слова на непарних позиціях при проході в зворотному напрямку. Визначити кількість таких слів.

Варіант 3.

Побудувати кільцевий (циклічний) двонаправлений список. Значення елементів зчитати з текстового файлу. Визначити, чи є в списку хоч би один елемент, який дорівнює елементу, що йде за ним по колу. Надрукувати такі елементи. Видалити із списку, що містить не менше двох елементів, усі елементи, у яких однакові «сусіди» (перший і останній елементи вважати сусідами).

Варіант 4.

Побудувати кільцевий (циклічний) двонаправлений список. Значення елементів зчитати з текстового файлу. Задати заголовний елемент списку. Надрукувати елементи списку, здійснивши його обхід за та проти годинникової стрілки. Вставити новий елемент в список на задану користувачем позицію з урахуванням заголовного елементу. Видалити заголовний елемент. Надрукувати елементи списку

Варіант 5.

Створити програму, що демонструє типові операції над елементами двонаправленого списку: додавання елемента на початок, в кінець та в середину списку, видалення елементів з кінця, з початку та з середини списку, обхід списку в двох напрямках (з початку в кінець, та з кінця на початок).

Варіант 6.

Побудувати двонаправлений список цілих чисел, зчитаних з текстового файлу. У створеному списку переставити його елементи так, щоб спочатку знаходились елементи зі значенням менше середнього арифметичного, а потім більше цього значення. Надрукувати вхідний та вихідний списки.

Варіант 7.

Побудувати двонаправлений список. Елемент списку містить дані про ім'я, розмір, дату створення файлу і права доступу до нього (читати, писати, доступ заборонений). Переписати в однонаправлений список дані про файли, права доступу до яких заборонені. Визначити кількість таких файлів. Надрукувати вхідний та вихідний списки.

Варіант 8.

Побудувати двонаправлений список з даних, що зчитані з текстового файлу. Елементами списку є дані про книги, що містяться в бібліотеці. Реалізувати дії, що обробляють каталог книг (включення, вилучення книг, пошук потрібної книги).

Варіант 9.

Представити автотрасу у вигляді двонаправленого списку, елементи якого містять інформацію про назву населених пунктів і відстань між ними. По заданій відстані між двома населеними пунктами вибрати назви двох населених пунктів, відстань між якими мінімально відрізняється від заданої. По заданих назвах двох населених пунктів визначити відстань між ними.

Варіант 10.

Побудувати кільцевий двонаправлений список із заголовним елементом. Підрахувати кількість таких елементів, у яких рівні сусіди, визначити та надрукувати їх. Знайти елемент з найменшим значенням в списку та видалити його. Надрукувати вхідний та вихідний списки.

Варіант 11.

Створити кільцевий двонаправлений список із заголовним елементом. В списку переставити в зворотному порядку усі елементи між першим і останнім входженнями заданого елементу, якщо він входить в список не менше двох разів.

Варіант 12.

Створити двонаправлений список, в якому заданий запис многочлена $f(x)$ довільного степеня з цілими коефіцієнтами, причому його одночлени можуть бути і не впорядковані за степенями x , одночлени одного степеня можуть повторюватися. Привести подібні члени в цьому многочлені та роздрукувати його за спаданням степеня x . Наприклад, $-2x^4 + 5x^2 + 7x^3 - 8x^4 + 10 - 3x^3$ подати у вигляді $-10x^4 + 4x^3 + 5x^2 + 10$. Надрукувати двонаправлені списки до та після приведення подібних членів в многочлені.

Варіант 13.

Представити у вигляді двонаправленого списку схему автобусного маршруту: назви зупинок та час прибуття автобуса на зупинку. Задати початковий і кінцевий пункти призначення пасажирів та визначити напрям руху для якнайшвидшого досягнення цілі.

Варіант 14.

Побудувати двозв'язний список команд, які беруть участь у змаганнях. Інформаційне поле елементів списку містить дані про назву команди, кількість балів, що отримала команда. Передбачити зчитування даних з файлу або введення з клавіатури. Змагання проходить у декілька етапів. На кожному етапі вилучається половина команд, які беруть участь у змаганні в поточному етапі. Продемонструвати хід змагань, виводивши на екран дані про команди, що вилучаються, та ті, що залишаються. Обчислити кількість етапів до визначення переможця змагань.

Варіант 15.

Побудувати двозв'язний список підручників, що подані на здобуття Державної премії України. Інформаційне поле елементів списку містить дані про прізвище автора, назву підручника, кількість схвальних відгуків на його підтримку, які визначаються на етапі голосування в Комітеті з Державних премій. Передбачити зчитування даних з файлу або введення з клавіатури. Утворити новий список з даними про підручники, які рекомендовані до розгляду на заключному засіданні Комітету з Державних премій за критерієм кількості схвальних відгуків, заданої користувачем з клавіатури. Надрукувати вхідний та вихідний списки.

Варіант 16.

Побудувати двозв'язний список абітурієнтів, які подали документи до приймальної комісії ВНЗ. Інформаційне поле елементів списку містить дані про прізвище абітурієнта, бал

сертифікату, тип документу про середню освіту (дублікат чи оригінал). Передбачити зчитування даних з файлу або введення з клавіатури. Утворити новий список з даними про абітурієнтів, що рекомендовані до зарахування за прохідним балом, значення якого користувач вводить з клавіатури. Надрукувати вхідний та вихідний списки.

Варіант 17.

Створити двозв'язний список справ, які людина має виконати. Елементи списку містять дані про назву роботи, термін та результат її виконання. Визначити роботи, які не були виконані, переписати їх у новий список, призначити їм нові терміни виконання та штрафи за порушення планових термінів. Надрукувати вхідний та вихідний список.

Варіант 18.

Створити двозв'язний список потягів, що прибувають на вокзал. Елементи списку містять дані про номер потягу та час його прибуття. Визначити потяги, що прибувають у заданий користувачем діапазон часу, та переписати дані про них у новий список, додавши до кожного запису номер колії. Надрукувати створені списки.

Варіант 19.

Створити двозв'язний список студентів групи. Елементи списку містять дані про прізвище, середній бал та стипендію. Визначити студентів із заданим середнім балом. Вилучити студентів, що не мають стипендії. Надрукувати вхідний та вихідний списки.

Варіант 20.

Створити двозв'язний список абонентів телефонної мережі. Елементи списку містять дані про прізвище, номер телефону та заборгованість з платежу. Визначити абонентів, що мають заборгованість вище заданого користувачем значення, та переписати їх у новий список. Надрукувати створені списки.

Варіант 21.

Побудувати двозв'язний список, значеннями елементів якого є елементи матриці, що зчитані з текстового файлу або введені з клавіатури. Побудувати новий список, елементами якого є елементи транспонованої матриці. Виділити діагональні елементи матриці та записати їх у новий список. Надрукувати вхідний та вихідні списки.

Варіант 22.

Побудувати два однозв'язних списків. Значеннями елементів першого списку є рядки, що зчитані з текстового файлу. Значеннями елементів другого списку є натуральні числа, що введені з клавіатури. Видалити з першого списку елементи, порядкові номери яких задані в другому списку. Надрукувати вхідні та вихідні списки.

Варіант 23.

Побудувати однозв'язний список, елементи якого є цілими числами, що зчитані з текстового файлу або введені з клавіатури. Видалити зі списку усі парні значення, значення з непарними номерами подвоїти, вставити в список у задану позицію підсписок із заданою кількістю елементів. Надрукувати вхідний та вихідний списки.

Варіант 24.

Побудувати два однозв'язних списків, значення елементів яких зчитані з текстового файлу або введені з клавіатури. Визначити теоретико-множинне об'єднання, різницю та перетин двох списків. Надрукувати вхідний та вихідні списки.

Варіант 25.

Особливим типом списку є дек (від англ. deque – double-ended queue, тобто черга з двома кінцями). Дек – це такий послідовний список, в якому як включення, так і виключення елементів може здійснюватися з будь-якого з двох його кінців. Створити дек з чисел $x_i, i = \overline{1, n}$, значення яких зчитані з текстового файлу. Визначити кількість та надрукувати значення елементів дека. Якщо дек має парну кількість елементів, визначити значення виразу $(x_1 - x_n) + (x_2 - x_{n-1}) + \dots + (x_{n/2} - x_{n/2+1})$, інакше обчислити суму його елементів.

Варіант 26.

Побудувати двозв'язний список із слів, зчитаних з рядків текстового файлу. Надрукувати створений список. Ввести з клавіатури слово та ціле число, що визначатиме порядковий номер елементу в списку. Поділити список на дві частини, зробивши введений елемент останнім в одному списку і першим в другому. Надрукувати результати поділу списку.

Варіант 27.

Створити двозв'язний список, елементами якого є слова текстового файлу. Вивести слова, що стоять на парних місцях при проході по списку в одному напрямку, та слова на непарних позиціях при проході в зворотному напрямку. Визначити кількість таких слів.

Варіант 28.

Побудувати кільцевий (циклічний) двонаправлений список. Значення елементів зчитати з текстового файлу. Визначити, чи є в списку хоч би один елемент, який дорівнює елементу, що йде за ним по колу. Надрукувати такі елементи. Видалити із списку, що містить не менше двох елементів, усі елементи, у яких однакові «сусіди» (перший і останній елементи вважати сусідами).

Варіант 29.

Побудувати кільцевий (циклічний) двонаправлений список. Значення елементів зчитати з текстового файлу. Задати заголовний елемент списку. Надрукувати елементи списку, здійснивши його обхід за та проти годинникової стрілки. Вставити новий елемент в список на задану користувачем позицію з урахуванням заголовного елементу. Видалити заголовний елемент. Надрукувати елементи списку

Варіант 30.

Створити програму, що демонструє типові операції над елементами двонаправленого списку: додавання елемента на початок, в кінець та в середину списку, видалення елементів з кінця, з початку та з середини списку, обхід списку в двох напрямках (з початку в кінець, та з кінця на початок).

Варіант 31.

Побудувати двонаправлений список цілих чисел, зчитаних з текстового файлу. У створеному списку переставити його елементи так, щоб спочатку знаходились елементи зі значенням менше середнього арифметичного, а потім більше цього значення. Надрукувати вхідний та вихідний списки.

Варіант 32.

Побудувати двонаправлений список. Елемент списку містить дані про ім'я, розмір, дату створення файлу і права доступу до нього (читати, писати, доступ заборонений). Переписати в однонаправлений список дані про файли, права доступу до яких заборонені. Визначити кількість таких файлів. Надрукувати вхідний та вихідний списки.

Варіант 33.

Побудувати двонаправлений список з даних, що зчитані з текстового файлу. Елементами списку є дані про книги, що містяться в бібліотеці. Реалізувати дії, що обробляють каталог книг (включення, вилучення книг, пошук потрібної книги).

Варіант 34.

Представити автотрасу у вигляді двонаправленого списку, елементи якого містять інформацію про назву населених пунктів і відстань між ними. По заданій відстані між двома населеними пунктами вибрати назви двох населених пунктів, відстань між якими мінімально відрізняється від заданої. По заданих назвах двох населених пунктів визначити відстань між ними.

Варіант 35.

Побудувати кільцевий двонаправлений список із заголовним елементом. Підрахувати кількість таких елементів, у яких рівні сусіди, визначити та надрукувати їх. Знайти елемент з найменшим значенням в списку та видалити його. Надрукувати вхідний та вихідний списки.

Варіант 36.

Створити кільцевий двонаправлений список із заголовним елементом. В списку переставити в зворотному порядку усі елементи між першим і останнім входженнями заданого елемента, якщо він входить в список не менше двох разів.

Варіант 37.

Створити двонаправлений список, в якому заданий запис многочлена $f(x)$ довільного степеня з цілими коефіцієнтами, причому його одночлени можуть бути і не впорядковані за степенями x , одночлени одного степеня можуть повторюватися. Привести подібні члени в цьому многочлені та роздрукувати його за спаданням степеня x . Наприклад, $-2x^4 + 5x^2 + 7x^3 - 8x^4 + 10 - 3x^3$ подати у вигляді $-10x^4 + 4x^3 + 5x^2 + 10$. Надрукувати двонаправлені списки до та після приведення подібних членів в многочлені.

Варіант 38.

Представити у вигляді двонаправленого списку схему автобусного маршруту: назви зупинок та час прибуття автобуса на зупинку. Задати початковий і кінцевий пункти призначення пасажирів та визначити напрям руху для якнайшвидшого досягнення цілі.

Варіант 39.

Побудувати двозв'язний список команд, які беруть участь у змаганнях. Інформаційне поле елементів списку містить дані про назву команди, кількість балів, що отримала команда. Передбачити зчитування даних з файлу або введення з клавіатури. Змагання проходить у декілька етапів. На кожному етапі вилучається половина команд, як беруть участь у змаганні в поточному етапі. Продемонструвати хід змагань, виводивши на екран дані про команди, що вилучаються, та ті, що залишаються. Обчислити кількість етапів до визначення переможця змагань.

Варіант 40.

Побудувати двозв'язний список підручників, що подані на здобуття Державної премії України. Інформаційне поле елементів списку містить дані про прізвище автора, назву підручника, кількість схвальних відгуків на його підтримку, які визначаються на етапі голосування в Комітеті з Державних премій. Передбачити зчитування даних з файлу або введення з клавіатури. Утворити новий список з даними про підручники, які рекомендовані до розгляду на заключному засіданні Комітету з Державних премій за критерієм кількості схвальних відгуків, заданої користувачем з клавіатури. Надрукувати вхідний та вихідний списки.

Лабораторна робота № 3.9. Опрацювання бінарних дерев

Мета роботи

Навчитися створювати та опрацьовувати бінарне дерево.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення елемента бінарного дерева та вказівника на нього.
- 2) Загальні схеми створення бінарного дерева (добавлення елемента), знищення бінарного дерева (вилучення елемента).
- 3) Довільні бінарні дерева та впорядковані бінарні дерева (дерева пошуку).
- 4) Загальні схеми обходу бінарного дерева.
- 5) Доступ до елементів бінарного дерева.
- 6) Дії із бінарним деревом.
- 7) Внутрішня реалізація бінарного дерева.
- 8) Передавання та опрацювання бінарних дерев у функціях.

Оформлення звіту

Вимоги та зразок оформлення звіту наведені у вступі до лабораторного практикуму.

Варіанти лабораторних завдань

Всі дії слід реалізувати за допомогою окремих підпрограм, які всю необхідну інформацію приймають лише за допомогою параметрів – використання глобальних змінних не допускається. Програма має містити меню, за допомогою якого користувач може вибрати необхідну йому дію.

Завдання, спільні для всіх варіантів:

1. Описати та реалізувати бінарне дерево / бінарне дерево пошуку із елементами, які можуть містити дані цілого типу.
2. Реалізувати такі дії:
 - 1) створення бінарного дерева із наперед вказаною кількістю елементів;
 - 2) вивід бінарного дерева / бінарного дерева пошуку на екран (з виділенням поточного елемента);
 - 3) створення бінарного дерева пошуку із наперед вказаною кількістю елементів;

- 4) додавання нового елемента із збереженням порядку (для побудови бінарного дерева пошуку);
- 5) збалансування бінарного дерева пошуку;
- 6) видалення елемента бінарного дерева / бінарного дерева пошуку, який не має дочірніх вузлів;
- 7) видалення елемента бінарного дерева / бінарного дерева пошуку, який має один дочірній вузол;
- 8) видалення елемента бінарного дерева / бінарного дерева пошуку, який має два дочірніх вузли;
- 9) видалення елемента бінарного дерева / бінарного дерева пошуку з усіма його дочірніми вузлами (тобто, видалення цілої гілки);
- 10) опрацювання бінарного дерева / бінарного дерева пошуку – виконати завдання свого варіанту.

Кожна функція має виконувати лише одну роль, і ця роль має бути відображена у назві функції.

«Функція, яка повертає / обчислює / шукає ...» – має не виводити ці значення, а повернути їх у місце виклику як результат функції або як відповідний вихідний параметр.

До звіту необхідно подати діаграму змін стану пам'яті при виконанні всіх операцій із деревом, вказаних у варіанті завдання.

Варіант 1.

Визначити, чи входить у бінарне дерево / бінарне дерево пошуку елемент із значенням інформаційного поля **k**.

Варіант 2.

Визначити кількість входжень в бінарне дерево / бінарне дерево пошуку елементів із значенням інформаційного поля **k**.

Варіант 3.

Визначити кількість елементів бінарного дерева / бінарного дерева пошуку.

Варіант 4.

Визначити суму всіх елементів бінарного дерева / бінарного дерева пошуку.

Варіант 5.

Визначити суму всіх елементів бінарного дерева / бінарного дерева пошуку із значеннями, кратними **k**.

Варіант 6.

Знайти перший елемент бінарного дерева із максимальний значенням – при «префіксному» обході – «зверху вниз» (для того щоб розпізнати «перший елемент дерева із максимальний значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 7.

Знайти останній елемент бінарного дерева із максимальний значенням – при «інфіксному» обході – «зліва направо» (для того щоб розпізнати «останній елемент дерева із максимальний значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 8.

Знайти перший елемент бінарного дерева із мінімальним значенням – при «постфіксному» обході – «знизу вверху» (для того щоб розпізнати «перший елемент дерева із мінімальним значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 9.

Знайти останній елемент бінарного дерева із мінімальним значенням – при «префіксному» обході – «зверху вниз» (для того щоб розпізнати «останній елемент дерева із мінімальним значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 10.

Визначити максимальну глибину бінарного дерева / бінарного дерева пошуку.

Варіант 11.

Визначити кількість елементів на n -ому рівні ієрархії бінарного дерева / бінарного дерева пошуку.

Варіант 12.

Визначити кількість елементів бінарного дерева / бінарного дерева пошуку, які не мають дочірніх.

Варіант 13.

Визначити кількість елементів бінарного дерева / бінарного дерева пошуку, які мають обидва дочірні елементи.

Варіант 14.

Визначити кількість елементів бінарного дерева / бінарного дерева пошуку, батьківські до яких мають значення k .

Варіант 15.

Знайти перший елемент бінарного дерева із максимальний значенням – при «інфіксному» обході – «зліва направо» (для того щоб розпізнати «перший елемент дерева із максимальний значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 16.

Знайти останній елемент бінарного дерева із максимальний значенням – при «постфіксному» обході – «знизу вгору» (для того щоб розпізнати «останній елемент дерева із максимальний значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 17.

Знайти перший елемент бінарного дерева із мінімальним значенням – при «префіксному» обході – «зверху вниз» (для того щоб розпізнати «перший елемент дерева із мінімальним значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 18.

Знайти останній елемент бінарного дерева із мінімальним значенням – при «інфіксному» обході – «зліва направо» (для того щоб розпізнати «останній елемент дерева із мінімальним значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 19.

Знайти перший елемент бінарного дерева із максимальний значенням – при «постфіксному» обході – «знизу вгору» (для того щоб розпізнати «перший елемент дерева із максимальний значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 20.

Знайти останній елемент бінарного дерева із максимальний значенням – при «префіксному» обході – «зверху вниз» (для того щоб розпізнати «останній елемент дерева із максимальний значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 21.

Знайти перший елемент бінарного дерева із мінімальним значенням – при «інфіксному» обході – «зліва направо» (для того щоб розпізнати «перший елемент дерева із мінімальним значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 22.

Знайти останній елемент бінарного дерева із мінімальним значенням – при «постфіксному» обході – «знизу вгору» (для того щоб розпізнати «останній елемент дерева із мінімальним значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 23.

Визначити, чи входить у бінарне дерево / бінарне дерево пошуку елемент із значенням інформаційного поля **k**.

Варіант 24.

Визначити кількість входжень в бінарне дерево / бінарне дерево пошуку елементів із значенням інформаційного поля **k**.

Варіант 25.

Визначити кількість елементів бінарного дерева / бінарного дерева пошуку.

Варіант 26.

Визначити суму всіх елементів бінарного дерева / бінарного дерева пошуку.

Варіант 27.

Визначити суму всіх елементів бінарного дерева / бінарного дерева пошуку із значеннями, кратними **k**.

Варіант 28.

Знайти перший елемент бінарного дерева із максимальний значенням – при «префіксному» обході – «зверху вниз» (для того щоб розпізнати «перший елемент дерева із максимальний значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 29.

Знайти останній елемент бінарного дерева із максимальний значенням – при «інфіксному» обході – «зліва направо» (для того щоб розпізнати «останній елемент дерева із максимальний значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 30.

Знайти перший елемент бінарного дерева із мінімальним значенням – при «постфіксному» обході – «знизу вверх» (для того щоб розпізнати «перший елемент дерева із мінімальним значенням», елементи дерева або мають мати по два інформаційних поля,

наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 31.

Знайти останній елемент бінарного дерева із мінімальним значенням – при «префіксному» обході – «зверху вниз» (для того щоб розпізнати «останній елемент дерева із мінімальним значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 32.

Визначити максимальну глибину бінарного дерева / бінарного дерева пошуку.

Варіант 33.

Визначити кількість елементів на n -ому рівні ієрархії бінарного дерева / бінарного дерева пошуку.

Варіант 34.

Визначити кількість елементів бінарного дерева / бінарного дерева пошуку, які не мають дочірніх.

Варіант 35.

Визначити кількість елементів бінарного дерева / бінарного дерева пошуку, які мають обидва дочірні елементи.

Варіант 36.

Визначити кількість елементів бінарного дерева / бінарного дерева пошуку, батьківські до яких мають значення k .

Варіант 37.

Знайти перший елемент бінарного дерева із максимальний значенням – при «інфіксному» обході – «зліва направо» (для того щоб розпізнати «перший елемент дерева із максимальний значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 38.

Знайти останній елемент бінарного дерева із максимальним значенням – при «постфіксному» обході – «знизу вгору» (для того щоб розпізнати «останній елемент дерева із максимальним значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 39.

Знайти перший елемент бінарного дерева із мінімальним значенням – при «префіксному» обході – «зверху вниз» (для того щоб розпізнати «перший елемент дерева із мінімальним значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Варіант 40.

Знайти останній елемент бінарного дерева із мінімальним значенням – при «інфіксному» обході – «зліва направо» (для того щоб розпізнати «останній елемент дерева із мінімальним значенням», елементи дерева або мають мати по два інформаційних поля, наприклад: основне – цілі числа, допоміжне – символи; або слід виводити бінарне дерево на екран з виділенням знайденого елемента).

Лабораторна робота № 3.10. Дерева

Мета роботи

Навчитися створювати та опрацьовувати бінарне дерево.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення елемента дерева та вказівника на нього.
- 2) Загальні схеми створення дерева (добавлення елемента), знищення бінарного дерева (вилучення елемента).
- 3) Довільні дерева та впорядковані дерева (дерева пошуку).
- 4) Загальні схеми обходу дерева.
- 5) Доступ до елементів дерева.
- 6) Дії із деревом.
- 7) Внутрішня реалізація дерева.
- 8) Передавання та опрацювання дерев у функціях.

Оформлення звіту

Вимоги та зразок оформлення звіту наведені у вступі до лабораторного практикуму.

Варіанти лабораторних завдань

Всі дії слід реалізувати за допомогою окремих підпрограм, які всю необхідну інформацію приймають лише за допомогою параметрів – використання глобальних змінних не допускається. Програма має містити меню, за допомогою якого користувач може вибрати необхідну йому дію.

Реалізувати операції додавання, видалення та пошуку вузлів дерева.

Кожна функція має виконувати лише одну роль, і ця роль має бути відображена у назві функції.

«Функція, яка повертає / обчислює / шукає ...» – має не виводити ці значення, а повернути їх у місце виклику як результат функції або як відповідний вихідний параметр.

До звіту необхідно подати діаграму змін стану пам'яті при виконанні всіх операцій із деревом, вказаних у варіанті завдання.

Варіант 1.

Текстовий файл містить програму мовою C++. Надрукувати в алфавітному порядку всі ідентифікатори цієї програми, вказавши для кожного з них кількість входжень у код. Для

збереження ідентифікаторів використати структуру типу дерева, елементами якого є ідентифікатор і кількість його входжень у код.

Варіант 2.

Текстовий файл містить програму мовою C++. Надрукувати в порядку зростання номери всіх рядків, що містять ідентифікатори програми. Для збереження ідентифікаторів використати структуру типу дерева, елементами якого є ідентифікатор і номер рядка, де він зустрічається.

Варіант 3.

З текстового файлу зчитати рядок, що містить арифметичний вираз. Побудувати дерево розбору виразу, де коренем дерева та його вузлами є операції *, /, +, -, а листками є операнди. Надрукувати дерево переліком його вузлів, що розміщені на рівнях, які відповідають послідовності виконання операцій.

Варіант 4.

Створити дерево, кількість нащадків кожного вузла якого генерується випадково. Обчислити кількість вузлів побудованого дерева та надрукувати значення його вузлів.

Варіант 5.

Побудувати дерево бінарного пошуку, значення елементів якого є слова, зчитані з текстового файлу. Визначити кількість рівнів та вузлів дерева та здійснити пошук значення, що вводиться з клавіатури

Варіант 6.

Побудувати бінарне дерево, значеннями елементів якого є числа. Знайти значення найбільшого елемента цього дерева, надрукувати його та номер рівня, на якому він розташований.

Варіант 7.

Побудувати бінарне дерево, вивести його на екран, підрахувати кількість вузлів на n -му рівні цього дерева (корінь дерева розташований на нульовому рівні) та надрукувати значення елементів на n -му рівні.

Варіант 8.

Побудувати бінарне дерево. Дані вводяться з клавіатури або зчитуються з файлу. Вивести дерево на екран. Визначити максимальну глибину дерева – число вузлів на найбільшому з маршрутів від кореня дерева до його листків. Вивести цей маршрут.

Варіант 9.

Побудувати два бінарних дерева, зчитавши значення їх вузлів з двох файлів. Порівняти побудовані дерева. Якщо дерева однакові, визначити їх кількість рівнів та вузлів, інакше зробити копію одного з дерев за вибором користувача.

Варіант 10.

Побудувати довільне незбалансоване бінарне дерево, зчитавши значення його вузлів з файлу. Збалансувати його шляхом перенесення деяких вузлів з одних гілок до інших. Надрукувати незбалансоване та збалансоване бінарні дерева.

Варіант 11.

Побудувати довільне бінарне дерево, зчитавши числові значення його вузлів з файлу. Зменшити парне значення кожного вузла дерева в два рази та збільшити непарне значення кожного вузла дерева в три рази. Надрукувати дерева до перетворення та після нього.

Варіант 12.

Побудувати довільне бінарне дерево, зчитавши числові значення його вузлів з файлу. Для кожного вузла дерева, що має два дочірніх вузла, поміняти місцями значення їх інформаційних полів. Для кожного вузла, що має один дочірній вузол, утворити другий із значенням, яке генерує генератор псевдовипадкових чисел.

Варіант 13.

Побудувати довільне бінарне дерево, зчитавши числові значення його вузлів з файлу. Видалити з дерева усі вузли, значення яких є парними числами. Надрукувати дерево, що утворено до та після видалення його вузлів.

Варіант 14.

Побудувати довільне бінарне дерево, зчитавши числові значення його вузлів з файлу. Видалити піддерева, значення кореня яких є числом Фібоначчі. Порахувати кількість видалених вузлів. Надрукувати дерево до та після видалення його вузлів.

Варіант 15.

Побудувати довільне бінарне дерево, зчитавши значення його вузлів з файлу. Здійснити обхід дерева з кореня до його листків та надрукувати значення його вузлів за префіксною, постфіксною та інфіксною формами. Визначити кількість вузлів між заданими користувачем значеннями.

Варіант 16.

З текстового файлу зчитати рядок, що містить арифметичний вираз. Створити дерево розбору виразу, в якому кожний вузол містить символ арифметичної операції (+, -, /, *), ліве та праве дочірні піддерева будь-якого вузла відповідають арифметичним виразам, що записані ліворуч і праворуч від знаку операції, листками дерева є числа. Вивести на екран побудоване дерево та вказівник на його корінь.

Варіант 17.

Зчитавши числові значення з текстового файлу, побудувати довільне дерево, кожний вузол якого матиме довільну кількість дочірніх вузлів. Визначити кількість рівнів дерева, Обчислити суму значень вузлів на кожному рівні та в крайньому зліва та крайньому справа піддеревах.

Варіант 18.

Побудувати довільне бінарне дерево, зчитавши числові значення його вузлів з файлу. Видалити з дерева усі вузли, значення яких знаходяться в діапазоні між мінімальним та середнім значеннями. Надрукувати дерево, що утворено до та після видалення його вузлів.

Варіант 19.

У створеному дереві, значення вузлів якого введені з клавіатури або зчитані з файлу, видалити мінімальну кількість вузлів (разом з інцидентними ребрами) так, щоб в графі, що утворився, не виявилось зв'язних компонент із заданою користувачем кількістю вузлів. Надрукувати вхідне та вихідне дерево.

Варіант 20.

У створеному дереві, значення вузлів якого введені з клавіатури або зчитані з файлу, знайти найближчого предка, спільного для кожної пари вузлів, значення яких вказані користувачем. Вивести знайдені значення предків та номери рівнів, на яких вони знаходяться.

Варіант 21.

Побудувати дерево, вузлами якого є прізвища керівників депутатських фракцій, лисками є прізвища членів фракцій та кількість їх відвідувань засідань Верховної Ради. Визначити членів фракцій, які мають найменшу кількість відвідувань засідань, та керівника фракції, до якої належить «прогульник». Якщо знайдуться такі керівники фракцій, члени якої жодного разу не були присутні на засіданні, вилучити таке піддерево на чолі з керівником. Надрукувати вхідне та вихідне піддерева.

Варіант 22.

В текстовому файлі задано текст. Підрахувати кількість повторень кожного слова тексту. Побудувати дерево, вузлами якого є слова тексту. Коренем дерева вважати слово, що зустрічається найбільшу кількість разів. Кожний наступний рівень міститиме вузли, в яких зберігатимуться слова з меншою кількістю повторень. Надрукувати отримане дерево та кількість повторень кожного слова.

Варіант 23.

Написати програму, що будує дерево, значення вузлів якого зчитані з файлу. Поміняти місцями найбільший та найменший елементи дерева. Надрукувати дерево до та після заміни елементів, найбільше та найменше значення елементів та номери рівнів їх місцезнаходження.

Варіант 24.

У турнірі з вибуванням беруть участь 2^n гравців, де n – кількість кіл турніру. Кожен програвший вибуває з турніру. Переможці грають попарно, нові переможці проходять в наступне коло і так доти, поки не залишиться тільки один переможець. Переможець кожної пари вказується у внутрішніх вузлах. Побудувати збалансоване бінарне дерево, що зображує турнір, та надрукувати його.

Варіант 25.

Побудувати бінарне дерево, вузлами якого є слова тексту, що зчитані з рядків текстового файлу. Коренем дерева є однолітерне слово. Якщо таких слів декілька, то вибирається будь-яке випадковим чином. На n -му рівні зберігаються слова довжини n . В межах одного рівня слова однакової довжини упорядковані за алфавітом. Якщо слів однакової довжини більше, ніж можна розмістити на одному рівні, то в дерево включаються

ті, що вибрані користувачем. Надрукувати отримане дерево, довжини слів, що увійшли до дерева та слова, що залишилися за межами дерева.

Варіант 26.

Текстовий файл містить програму мовою C++. Надрукувати в алфавітному порядку всі ідентифікатори цієї програми, вказавши для кожного з них кількість входжень у код. Для збереження ідентифікаторів використати структуру типу дерева, елементами якого є ідентифікатор і кількість його входжень у код.

Варіант 27.

Текстовий файл містить програму мовою C++. Надрукувати в порядку зростання номери всіх рядків, що містять ідентифікатори програми. Для збереження ідентифікаторів використати структуру типу дерева, елементами якого є ідентифікатор і номер рядка, де він зустрічається.

Варіант 28.

З текстового файлу зчитати рядок, що містить арифметичний вираз. Побудувати дерево розбору виразу, де коренем дерева та його вузлами є операції *, /, +, -, а листками є операнди. Надрукувати дерево переліком його вузлів, що розміщені на рівнях, які відповідають послідовності виконання операцій.

Варіант 29.

Створити дерево, кількість нащадків кожного вузла якого генерується випадково. Обчислити кількість вузлів побудованого дерева та надрукувати значення його вузлів.

Варіант 30.

Побудувати дерево бінарного пошуку, значення елементів якого є слова, зчитані з текстового файлу. Визначити кількість рівнів та вузлів дерева та здійснити пошук значення, що вводиться з клавіатури

Варіант 31.

Побудувати бінарне дерево, значеннями елементів якого є числа. Знайти значення найбільшого елемента цього дерева, надрукувати його та номер рівня, на якому він розташований.

Варіант 32.

Побудувати бінарне дерево, вивести його на екран, підрахувати кількість вузлів на n -му рівні цього дерева (корінь дерева розташований на нульовому рівні) та надрукувати значення елементів на n -му рівні.

Варіант 33.

Побудувати бінарне дерево. Дані вводяться з клавіатури або зчитуються з файлу. Вивести дерево на екран. Визначити максимальну глибину дерева – число гілок на найбільшому з маршрутів від кореня дерева до його листків. Вивести цей маршрут.

Варіант 34.

Побудувати два бінарних дерева, зчитавши значення їх вузлів з двох файлів. Порівняти побудовані дерева. Якщо дерева однакові, визначити їх кількість рівнів та вузлів, інакше зробити копію одного з дерев за вибором користувача.

Варіант 35.

Побудувати довільне незбалансоване бінарне дерево, зчитавши значення його вузлів з файлу. Збалансувати його шляхом перенесення деяких вузлів з одних гілок до інших. Надрукувати незбалансоване та збалансоване бінарні дерева.

Варіант 36.

Побудувати довільне бінарне дерево, зчитавши числові значення його вузлів з файлу. Зменшити парне значення кожного вузла дерева в два рази та збільшити непарне значення кожного вузла дерева в три рази. Надрукувати дерева до перетворення та після нього.

Варіант 37.

Побудувати довільне бінарне дерево, зчитавши числові значення його вузлів з файлу. Для кожного вузла дерева, що має два дочірніх вузла, поміняти місцями значення їх інформаційних полів. Для кожного вузла, що має один дочірній вузол, утворити другий із значенням, яке генерує генератор псевдовипадкових чисел.

Варіант 38.

Побудувати довільне бінарне дерево, зчитавши числові значення його вузлів з файлу. Видалити з дерева усі вузли, значення яких є парними числами. Надрукувати дерево, що утворено до та після видалення його вузлів.

Варіант 39.

Побудувати довільне бінарне дерево, зчитавши числові значення його вузлів з файлу. Видалити піддерева, значення кореня яких є числом Фібоначчі. Порахувати кількість видалених вузлів. Надрукувати дерево до та після видалення його вузлів.

Варіант 40.

Побудувати довільне бінарне дерево, зчитавши значення його вузлів з файлу. Здійснити обхід дерева з кореня до його листків та надрукувати значення його вузлів за префіксною, постфіксною та інфіксною формами. Визначити кількість вузлів між заданими користувачем значеннями.

Лабораторна робота № 3.11. Опрацювання динамічних структур даних – 1

Мета роботи

Навчитися опрацьовувати динамічні структури даних.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення вказівникових типів та змінних-вказівників.
- 2) Загальний синтаксис оголошення динамічних змінних.
- 3) Доступ до змінних за допомогою вказівника.
- 4) Дії з вказівниками та динамічними змінними.
- 5) Внутрішня реалізація вказівників, динамічних та статичних змінних.
- 6) Передавання та опрацювання вказівників та динамічних змінних у підпрограмах.

Оформлення звіту

Вимоги та зразок оформлення звіту наведені у вступі до лабораторного практикуму.

Варіанти лабораторних завдань

Програма має містити меню. Необхідно передбачити контроль помилок користувача при введенні даних. Передбачити можливість збереження всієї структури даних у файл та завантаження з файлу. Ім'я файлу має вводитися з клавіатури.

При розробці програми застосувати технологію покрокової деталізації. Логічно закінчені фрагменти оформити у вигляді підпрограм, всі необхідні дані яким передаються через список параметрів. Використання глобальних змінних слід уникати.

Завдання складається з двох частин:

Частина 1. Реалізувати структуру даних у вигляді списку;

Частина 2. Реалізувати структуру даних у вигляді бінарного дерева.

Кожна функція має виконувати лише одну роль, і ця роль має бути відображена у назві функції.

«Функція, яка повертає / обчислює / шукає ...» — має не виводити ці значення, а повернути їх у місце виклику як результат функції або як відповідний вихідний параметр.

До звіту необхідно подати діаграму змін стану пам'яті при виконанні всіх операцій із списком та деревом, вказаних у варіанті завдання.

Реалізувати стандартні дії із списком / бінарним деревом: додавання елемента, вилучення елемента, перегляд, сортування списку, балансування дерева.

Варіант 1.

Скласти програму, яка містить динамічну інформацію про наявність автобусів в автобусному парку.

Відомості про кожен автобус містять:

- номер автобуса;
- прізвище і ініціали водія;
- номер маршруту.

Програма має забезпечувати такі функціональні можливості:

- початкове формування даних про всі автобуси в парку у вигляді списку / бінарного дерева;
- при виїзді кожного автобуса з парку вводиться номер автобуса, і програма видаляє дані про цей автобус із списку / бінарного дерева автобусів, що знаходяться в парку, і записує ці дані в список / бінарне дерево автобусів, що знаходяться на маршруті;
- при в'їзді кожного автобуса до парку вводиться номер автобуса, і програма видаляє дані про цей автобус із списку / бінарного дерева автобусів, що знаходяться на маршруті, і записує ці дані в список / бінарне дерево автобусів, що знаходяться в парку;
- за запитом видаються відомості про автобуси, що знаходяться в парку, або про автобуси, що знаходяться на маршруті.

Варіант 2.

Скласти програму, яка містить поточну інформацію про книги в бібліотеці.

Відомості про книги містять:

- номер УДК;
- прізвище і ініціали автора;
- назва;
- рік видання;
- кількість екземплярів даної книги в бібліотеці.

Програма має забезпечувати такі функціональні можливості:

- початкове формування даних про всі книги в бібліотеці у вигляді списку / бінарного дерева;
- додавання даних про книги, що знову поступають в бібліотеку;
- видалення даних про книги, що списуються;

- за запитом видаються зведення про наявність книг в бібліотеці, впорядковані по роках видання.

Варіант 3.

Скласти програму, яка містить поточну інформацію про заявки на авіаквитки.

Кожна заявка містить:

- пункт призначення;
- номер рейсу;
- прізвище і ініціали пасажирів;
- бажану дату вильоту.

Програма має забезпечувати:

- зберігання всіх заявок у вигляді списку / бінарного дерева;
- додавання заявок в список / бінарне дерево;
- видалення заявок;
- виведення заявок по заданому номеру рейсу і даті вильоту;
- виведення всіх заявок.

Варіант 4.

Скласти програму, яка містить поточну інформацію про заявки на авіаквитки.

Кожна заявка містить:

- пункт призначення;
- номер рейсу;
- прізвище і ініціали пасажирів;
- бажану дату вильоту.

Програма має забезпечувати:

- зберігання всіх заявок у вигляді списку / бінарного дерева;
- додавання і видалення заявок;
- по заданому номеру рейсу і даті вильоту виведення заявок з їх подальшим видаленням;
- виведення всіх заявок.

Варіант 5.

Скласти програму, яка містить поточну інформацію про книги в бібліотеці.

Відомості про книги містять:

- номер УДК;

- прізвище і ініціали автора;
- назва;
- рік видання;
- кількість екземплярів даної книги в бібліотеці.

Програма має забезпечувати такі функціональні можливості:

- початкове формування даних про всі книги в бібліотеці у вигляді списку / бінарного дерева;
- при узятті кожної книги вводиться номер УДК, і програма зменшує значення кількості книг на одиницю або видає повідомлення про те, що необхідної книги в бібліотеці немає або вона видана і є в когось із читачів на руках;
- при поверненні кожної книги вводиться номер УДК, і програма збільшує значення кількості книг на одиницю;
- за запитом видаються зведення про наявність книг в бібліотеці.

Варіант 6.

Скласти програму, яка містить динамічну інформацію про наявність автобусів в автобусному парку.

Відомості про кожен автобус містять:

- номер автобуса;
- прізвище і ініціали водія;
- номер маршруту;
- ознаку того, де є автобус – на маршруті або в парку.

Програма має забезпечувати такі функціональні можливості:

- початкове формування даних про всі автобуси у вигляді списку / бінарного дерева;
- при виїзді кожного автобуса з парку вводиться номер автобуса, і програма встановлює значення ознаки «автобус на маршруті»;
- при в'їзді кожного автобуса до парку вводиться номер автобуса, і програма встановлює значення ознаки «автобус в парку»;
- за запитом видаються відомості про автобуси, що знаходяться в парку, або про автобуси, що знаходяться на маршруті.

Варіант 7.

Скласти програму, що відшукує прохід по лабіринту.

Лабіринт представляється у вигляді матриці, що складається з квадратів. Кожен квадрат або відкритий, або закритий. Вхід в закритий квадрат заборонений. Якщо квадрат відкритий, то

вхід в нього можливий з боку, але не з кута. Кожен квадрат визначається його координатами в матриці.

Програма знаходить прохід через лабіринт, рухаючись від заданого входу. Після відшукування проходу програма виводить знайдений шлях у вигляді координат квадратів.

1. Для зберігання шляху використовувати стек (список).
2. Для зберігання даних про лабіринт використовувати бінарне дерево

Варіант 8.

Гаражна стоянка має одну смугу стоянки, причому єдиний в'їзд і єдиний виїзд знаходяться в одному кінці смуги. Якщо власник автомашини приходить забрати свій автомобіль, який не є найближчим до виходу, то всі автомашини, що загороджують проїзд, віддаляються і машина даного власника виводиться із стоянки, після чого інші машини повертаються на стоянку в початковому порядку.

Написати програму, яка моделює процес прибуття і від'їзду машин. Прибуття або від'їзд автомашини задається користувачем – даними, що містять ознаку прибуття або від'їзду і номер машини. Програма має виводити повідомлення при прибутті або виїзді будь-якої машини. При виїзді автомашини із стоянки повідомлення має містити число разів, яке машина віддалялася із стоянки для забезпечення виїзду інших автомобілів.

Варіант 9.

Скласти програму, що моделює заповнення гнучкого магнітного диска (пристрій зберігання даних для комп'ютерів 1980-1990-х років).

Загальний об'єм пам'яті на диску 360 Кбайт. Файли мають довільну довжину від 18 байт до 32 Кбайт. В процесі роботи файли або записуються на диск, або видаляються з нього.

На початку роботи файли записуються підряд один за одним. Після видалення файлу на диску утворюється вільна ділянка пам'яті, і новий файл, що записується або розміщується на вільній ділянці, або, якщо файл не вміщається у вільну ділянку, розміщується після останнього записаного файлу.

У випадку, коли файл перевершує довжину найбільшої вільної ділянки, видається аварійне повідомлення. Вимога на запис або видалення файлу задається користувачем – даними, що містять ім'я файлу, його довжину в байтах, ознака запису або видалення. Програма має виводити дані про зайняті і вільні ділянки пам'яті на диску.

ПОРАДА

Слід створити список / бінарне дерево зайнятих ділянок і список / бінарне дерево вільних ділянок пам'яті на диску.

Варіант 10.

У файловій системі каталог файлів організований як лінійний список / бінарне дерево. Для кожного файлу в каталозі містяться такі відомості:

- ім'я файлу;
- дата створення;
- кількість звернень до файлу.

Скласти програму, яка забезпечує:

- початкове формування каталогу файлів;
- виведення каталогу файлів;
- видалення файлів, дата створення яких менша заданої;
- вибірку файлу з найбільшою кількістю звернень.

Варіант 11.

Предметний покажчик організований як лінійний список / бінарне дерево.

Кожна компонента покажчику містить слово і номери сторінок, на яких це слово зустрічається. Кількість номерів сторінок, що відносяться до одного слова, може лежати в інтервалі від одного до десяти.

Скласти програму, яка забезпечує:

- початкове формування предметного покажчика;
- виведення предметного покажчика;
- виведення номерів сторінок для заданого слова.

Варіант 12.

Текст допомоги для деякої програми організований як лінійний список / бінарне дерево.

Кожна компонента тексту допомоги містить термін (слово) і текст пояснення до цього терміну. Допустима кількість рядків тексту, що відносяться до одного терміну, – від однієї до п'яти.

Скласти програму, яка забезпечує:

- початкове формування тексту допомоги;
- виведення тексту допомоги;
- виведення пояснюючого тексту для заданого терміну.

Варіант 13.

Картотека в бюро обміну квартир організована як лінійний список / бінарне дерево. Відомості про кожну квартиру містять:

- кількість кімнат;
- поверх;
- площа;
- адреса.

Скласти програму, яка забезпечує:

- початкове формування картотеки;
- введення заявки на обмін;
- пошук в картотеці відповідного варіанту: при рівності кількості кімнат і поверху і відмінності площ в межах 10 % виводиться відповідна карточка, яка потім вилучається із списку / бінарного дерева; в іншому випадку заявка, що поступила, включається в список / бінарного дерева;
- виведення всього списку / бінарного дерева.

Варіант 14.

Англо-український словник побудований як список / бінарне дерево.

Кожна компонента містить англійське слово, відповідне йому українське слово і лічильник кількості звернень до цієї компоненти.

Первинний список / дерево формується згідно англійському алфавіту. В процесі експлуатації словника при кожному зверненні до компоненти в лічильник звернень додається одиниця.

Скласти програму, яка:

- забезпечує початкове введення словника з конкретними значеннями лічильника звернень;
- формує нове представлення словника у вигляді списку / бінарного дерева за таким алгоритмом:
 - 1) у старому словнику шукається компонента з найбільшим значенням лічильника звернень;
 - 2) знайдена компонента заноситься в новий словник і видаляється із старого;
 - 3) перехід до п. 1 до вичерпання початкового словника;
- проводить виведення початкового і нового словників.

Варіант 15.

Анкета для опитування населення містить дві групи питань. Перша група містить відомості про респондента:

- вік;

- статъ;
- освіта (початкова, середня, вища).

Друга група містить власне питання анкети, відповіддю на яке може бути або «Так», або «Ні».

Скласти програму, яка:

- забезпечує початкове введення анкет і формує з них лінійний список / бінарне дерево;
- на основі аналізу анкет видає відповіді на такі питання:
 - 1) скільки чоловіків старше 40 років, що мають вищу освіту, відповіли «Та» на питання анкети;
 - 2) скільки жінок молодше 30 років, що мають середню освіту, відповіли «Ні» на питання анкети;
 - 3) скільки чоловіків молодше 25 років, що мають початкову освіту, відповіли «Та» на питання анкети;
- забезпечує виведення всіх анкет і відповідей на питання.

Варіант 16.

Скласти програму, яка містить поточну інформацію про книги в бібліотеці.

Відомості про книги містять:

- номер УДК;
- прізвище і ініціали автора;
- назву;
- рік видання;
- кількість екземплярів цієї книги в бібліотеці.

Програма має забезпечувати такі функціональні можливості:

- початкове формування даних про всі книги в бібліотеці у вигляді списку / бінарного дерева;
- додавання даних про книги, що знову поступають в бібліотеку;
- видалення даних про списані книги;
- за запитом видається інформація про наявність книг в бібліотеці, впорядкована по роках видання.

Варіант 17.

На міжміській телефонній станції картотека абонентів, що містить відомості про телефони і їх власників, організована як лінійний список / бінарне дерево.

Скласти програму, яка:

- забезпечує початкове формування картотеки у вигляді лінійного списку / бінарного

дерева;

- проводить виведення всієї картотеки;
- отримує номер телефону і час розмови;
- виводить повідомлення на оплату телефонної розмови.

Варіант 18.

На міжміській телефонній станції картотека абонентів, що містить відомості про телефони і їх власників, організована як список / двійкове дерево.

Скласти програму, яка:

- забезпечує початкове формування картотеки у вигляді списку / бінарного дерева;
- забезпечує виведення всієї картотеки;
- отримує номер телефону і час розмови;
- виводить повідомлення на оплату телефонної розмови.

Варіант 19.

Автоматизована інформаційна система на залізничному вокзалі містить інформацію про відправлення поїздів дальнього слідування. Для кожного поїзда вказується:

- номер поїзда;
- станція призначення;
- час відправлення.

Дані в інформаційній системі організовані у вигляді лінійного списку / бінарного дерева.

Скласти програму, яка:

- забезпечує первинне введення даних в інформаційну систему і формування лінійного списку / бінарного дерева;
- проводить виведення всього списку / бінарного дерева;
- отримує номер поїзда і виводить всі дані про цей поїзд;
- отримує назву станції призначення і виводить дані про всі поїзди, що слідують до цієї станції.

Варіант 20.

Автоматизована інформаційна система на залізничному вокзалі містить інформацію про відправлення поїздів дальнього слідування.

Для кожного поїзда вказується:

- номер поїзда;
- станція призначення;

- час відправлення.

Дані в інформаційній системі організовані у вигляді списку / бінарного дерева. Скласти програму, яка:

- забезпечує первинне введення даних в інформаційну систему і формування списку / бінарного дерева а;
- забезпечує виведення всього списку / бінарного дерева;
- отримує номер поїзда і виводить всі дані про цей поїзд;
- отримує назву станції призначення і виводить дані про всі поїзди, що слідують до цієї станції.

Варіант 21.

Скласти програму, яка містить динамічну інформацію про наявність автобусів в автобусному парку.

Відомості про кожен автобус містять:

- номер автобуса;
- прізвище і ініціали водія;
- номер маршруту.

Програма має забезпечувати такі функціональні можливості:

- початкове формування даних про всі автобуси в парку у вигляді списку / бінарного дерева;
- при виїзді кожного автобуса з парку вводиться номер автобуса, і програма видаляє дані про цей автобус із списку / бінарного дерева автобусів, що знаходяться в парку, і записує ці дані в список / бінарне дерево автобусів, що знаходяться на маршруті;
- при в'їзді кожного автобуса до парку вводиться номер автобуса, і програма видаляє дані про цей автобус із списку / бінарного дерева автобусів, що знаходяться на маршруті, і записує ці дані в список / бінарне дерево автобусів, що знаходяться в парку;
- за запитом видаються відомості про автобуси, що знаходяться в парку, або про автобуси, що знаходяться на маршруті.

Варіант 22.

Скласти програму, яка містить поточну інформацію про книги в бібліотеці.

Відомості про книги містять:

- номер УДК;
- прізвище і ініціали автора;
- назва;

- рік видання;
- кількість екземплярів цієї книги в бібліотеці.

Програма має забезпечувати такі функціональні можливості:

- початкове формування даних про всі книги в бібліотеці у вигляді списку / бінарного дерева;
- додавання даних про книги, що знову поступають в бібліотеку;
- видалення даних про книги, що списуються;
- за запитом видаються зведення про наявність книг в бібліотеці, впорядковані по роках видання.

Варіант 23.

Скласти програму, яка містить поточну інформацію про заявки на авіаквитки.

Кожна заявка містить:

- пункт призначення;
- номер рейсу;
- прізвище і ініціали пасажирів;
- бажану дату вильоту.

Програма має забезпечувати:

- зберігання всіх заявок у вигляді списку / бінарного дерева;
- додавання заявок в список / бінарне дерево;
- видалення заявок;
- виведення заявок по заданому номеру рейсу і даті вильоту;
- виведення всіх заявок.

Варіант 24.

Скласти програму, яка містить поточну інформацію про заявки на авіаквитки.

Кожна заявка містить:

- пункт призначення;
- номер рейсу;
- прізвище і ініціали пасажирів;
- бажану дату вильоту.

Програма має забезпечувати:

- зберігання всіх заявок у вигляді списку / бінарного дерева;
- додавання і видалення заявок;

- по заданому номеру рейсу і даті вильоту виведення заявок з їх подальшим видаленням;
- виведення всіх заявок.

Варіант 25.

Скласти програму, яка містить поточну інформацію про книги в бібліотеці.

Відомості про книги містять:

- номер УДК;
- прізвище і ініціали автора;
- назва;
- рік видання;
- кількість екземплярів цієї книги в бібліотеці.

Програма має забезпечувати такі функціональні можливості:

- початкове формування даних про всі книги в бібліотеці у вигляді списку / бінарного дерева;
- при узятті кожної книги вводиться номер УДК, і програма зменшує значення кількості книг на одиницю або видає повідомлення про те, що необхідної книги в бібліотеці немає або вона видана і є в когось із читачів на руках;
- при поверненні кожної книги вводиться номер УДК, і програма збільшує значення кількості книг на одиницю;
- за запитом видаються зведення про наявність книг в бібліотеці.

Варіант 26.

Скласти програму, яка містить динамічну інформацію про наявність автобусів в автобусному парку.

Відомості про кожен автобус містять:

- номер автобуса;
- прізвище і ініціали водія;
- номер маршруту;
- ознаку того, де є автобус – на маршруті або в парку.

Програма має забезпечувати такі функціональні можливості:

- початкове формування даних про всі автобуси у вигляді списку / бінарного дерева;
- при виїзді кожного автобуса з парку вводиться номер автобуса, і програма встановлює значення ознаки «автобус на маршруті»;
- при в'їзді кожного автобуса до парку вводиться номер автобуса, і програма встановлює

значення ознаки «автобус в парку»;

- за запитом видаються відомості про автобуси, що знаходяться в парку, або про автобуси, що знаходяться на маршруті.

Варіант 27.

Скласти програму, що відшукує прохід по лабіринту.

Лабіринт представляється у вигляді матриці, що складається з квадратів. Кожен квадрат або відкритий, або закритий. Вхід в закритий квадрат заборонений. Якщо квадрат відкритий, то вхід в нього можливий з боку, але не з кута. Кожен квадрат визначається його координатами в матриці.

Програма знаходить прохід через лабіринт, рухаючись від заданого входу. Після відшукання проходу програма виводить знайдений шлях у вигляді координат квадратів.

1. Для зберігання шляху використовувати стек (список).
2. Для зберігання даних про лабіринт використовувати бінарне дерево

Варіант 28.

Гаражна стоянка має одну смугу стоянки, причому єдиний в'їзд і єдиний виїзд знаходяться в одному кінці смуги. Якщо власник автомашини приходить забрати свій автомобіль, який не є найближчим до виходу, то всі автомашини, що загороджують проїзд, віддаляються і машина цього власника виводиться із стоянки, після чого інші машини повертаються на стоянку в початковому порядку.

Написати програму, яка моделює процес прибуття і від'їзду машин. Прибуття або від'їзд автомашини задається користувачем – даними, що містять ознаку прибуття або від'їзду і номер машини. Програма має виводити повідомлення при прибутті або виїзді будь-якої машини. При виїзді автомашини із стоянки повідомлення має містити число разів, яке машина віддалялася із стоянки для забезпечення виїзду інших автомобілів.

Варіант 29.

Скласти програму, що моделює заповнення гнучкого магнітного диска (пристрій зберігання даних для комп'ютерів 1980-1990-х років).

Загальний об'єм пам'яті на диску 360 Кбайт. Файли мають довільну довжину від 18 байт до 32 Кбайт. В процесі роботи файли або записуються на диск, або видаляються з нього.

На початку роботи файли записуються підряд один за одним. Після видалення файлу на диску утворюється вільна ділянка пам'яті, і новий файл, що записується або розміщується на вільній ділянці, або, якщо файл не вміщається у вільну ділянку, розміщується після останнього записаного файлу.

У випадку, коли файл перевершує довжину найбільшої вільної ділянки, видається аварійне повідомлення. Вимога на запис або видалення файлу задається користувачем – даними, що містять ім'я файлу, його довжину в байтах, ознака запису або видалення. Програма має виводити дані про зайняті і вільні ділянки пам'яті на диску.

ПОРАДА

Слід створити список / бінарне дерево зайнятих ділянок і список / бінарне дерево вільних ділянок пам'яті на диску.

Варіант 30.

У файловій системі каталог файлів організований як лінійний список / бінарне дерево. Для кожного файлу в каталозі містяться такі відомості:

- ім'я файлу;
- дата створення;
- кількість звернень до файлу.

Скласти програму, яка забезпечує:

- початкове формування каталогу файлів;
- виведення каталогу файлів;
- видалення файлів, дата створення яких менша заданої;
- вибірку файлу з найбільшою кількістю звернень.

Варіант 31.

Предметний покажчик організований як лінійний список / бінарне дерево.

Кожна компонента покажчика містить слово і номери сторінок, на яких це слово зустрічається. Кількість номерів сторінок, що відносяться до одного слова, може лежати в інтервалі від одного до десяти.

Скласти програму, яка забезпечує:

- початкове формування предметного покажчика;
- виведення предметного покажчика;
- виведення номерів сторінок для заданого слова.

Варіант 32.

Текст допомоги для деякої програми організований як лінійний список / бінарне дерево.

Кожна компонента тексту допомоги містить термін (слово) і текст пояснення до цього терміну. Допустима кількість рядків тексту, що відносяться до одного терміну, – від однієї до п'яти.

Скласти програму, яка забезпечує:

- початкове формування тексту допомоги;
- виведення тексту допомоги;
- виведення пояснюючого тексту для заданого терміну.

Варіант 33.

Картотека в бюро обміну квартир організована як лінійний список / бінарне дерево.

Відомості про кожну квартиру містять:

- кількість кімнат;
- поверх;
- площа;
- адреса.

Скласти програму, яка забезпечує:

- початкове формування картотеки;
- введення заявки на обмін;
- пошук в картотеці відповідного варіанту: при рівності кількості кімнат і поверху і відмінності площ в межах 10 % виводиться відповідна карточка, яка потім вилучається із списку / бінарного дерева; в іншому випадку заявка, що поступила, включається в список / бінарного дерева;
- виведення всього списку / бінарного дерева.

Варіант 34.

Англо-український словник побудований як список / бінарне дерево.

Кожна компонента містить англійське слово, відповідне йому українське слово і лічильник кількості звернень до цієї компоненти.

Первинний список / дерево формується згідно англійському алфавіту. В процесі експлуатації словника при кожному зверненні до компоненти в лічильник звернень додається одиниця.

Скласти програму, яка:

- забезпечує початкове введення словника з конкретними значеннями лічильника звернень;
- формує нове представлення словника у вигляді списку / бінарного дерева за таким алгоритмом:
 - 1) у старому словнику шукається компонента з найбільшим значенням лічильника звернень;

- 2) знайдена компонента заноситься в новий словник і видаляється із старого;
- 3) перехід до п. 1 до вичерпання початкового словника;
- проводить виведення початкового і нового словників.

Варіант 35.

Анкета для опитування населення містить дві групи питань. Перша група містить відомості про респондента:

- вік;
- стать;
- освіта (початкова, середня, вища).

Друга група містить власне питання анкети, відповіддю на яке може бути або «Так», або «Ні».

Скласти програму, яка:

- забезпечує початкове введення анкет і формує з них лінійний список / бінарне дерево;
- на основі аналізу анкет видає відповіді на такі питання:
 - 1) скільки чоловіків старше 40 років, що мають вищу освіту, відповіли «Та» на питання анкети;
 - 2) скільки жінок молодше 30 років, що мають середню освіту, відповіли «Ні» на питання анкети;
 - 3) скільки чоловіків молодше 25 років, що мають початкову освіту, відповіли «Та» на питання анкети;
- забезпечує виведення всіх анкет і відповідей на питання.

Варіант 36.

Скласти програму, яка містить поточну інформацію про книги в бібліотеці.

Відомості про книги містять:

- номер УДК;
- прізвище і ініціали автора;
- назву;
- рік видання;
- кількість екземплярів цієї книги в бібліотеці.

Програма має забезпечувати такі функціональні можливості:

- початкове формування даних про всі книги в бібліотеці у вигляді списку / бінарного дерева;
- додавання даних про книги, що знову поступають в бібліотеку;
- видалення даних про списані книги;

- за запитом видається інформація про наявність книг в бібліотеці, впорядкована по роках видання.

Варіант 37.

На міжміській телефонній станції картотека абонентів, що містить відомості про телефони і їх власників, організована як лінійний список / бінарне дерево.

Скласти програму, яка:

- забезпечує початкове формування картотеки у вигляді лінійного списку / бінарного дерева;
- проводить виведення всієї картотеки;
- отримує номер телефону і час розмови;
- виводить повідомлення на оплату телефонної розмови.

Варіант 38.

На міжміській телефонній станції картотека абонентів, що містить відомості про телефони і їх власників, організована як список / двійкове дерево.

Скласти програму, яка:

- забезпечує початкове формування картотеки у вигляді списку / бінарного дерева;
- забезпечує виведення всієї картотеки;
- отримує номер телефону і час розмови;
- виводить повідомлення на оплату телефонної розмови.

Варіант 39.

Автоматизована інформаційна система на залізничному вокзалі містить інформацію про відправлення поїздів дальнього слідування. Для кожного поїзда вказується:

- номер поїзда;
- станція призначення;
- час відправлення.

Дані в інформаційній системі організовані у вигляді лінійного списку / бінарного дерева.

Скласти програму, яка:

- забезпечує первинне введення даних в інформаційну систему і формування лінійного списку / бінарного дерева;
- проводить виведення всього списку / бінарного дерева;
- отримує номер поїзда і виводить всі дані про цей поїзд;
- отримує назву станції призначення і виводить дані про всі поїзди, що слідують до цієї

станції.

Варіант 40.

Автоматизована інформаційна система на залізничному вокзалі містить інформацію про відправлення поїздів дальнього слідування.

Для кожного поїзда вказується:

- номер поїзда;
- станція призначення;
- час відправлення.

Дані в інформаційній системі організовані у вигляді списку / бінарного дерева. Скласти програму, яка:

- забезпечує первинне введення даних в інформаційну систему і формування списку / бінарного дерева а;
- забезпечує виведення всього списку / бінарного дерева;
- отримує номер поїзда і виводить всі дані про цей поїзд;
- отримує назву станції призначення і виводить дані про всі поїзди, що слідують до цієї станції.

Лабораторна робота № 3.12. Опрацювання динамічних структур даних – 2

Мета роботи

Навчитися опрацьовувати динамічні структури даних.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення вказівникових типів та змінних-вказівників.
- 2) Загальний синтаксис оголошення динамічних змінних.
- 3) Доступ до змінних за допомогою вказівника.
- 4) Дії з вказівниками та динамічними змінними.
- 5) Внутрішня реалізація вказівників, динамічних та статичних змінних.
- 6) Передавання та опрацювання вказівників та динамічних змінних у підпрограмах.

Оформлення звіту

Вимоги та зразок оформлення звіту наведені у вступі до лабораторного практикуму.

Варіанти лабораторних завдань

Програма має містити меню. Необхідно передбачити контроль помилок користувача при введенні даних.

Необхідні програмі дані слід реалізувати за допомогою списку / бінарного дерева. Передбачити можливість збереження всієї структури даних у файл та завантаження з файлу. Ім'я файлу має вводитися з клавіатури.

При розробці програми застосувати технологію низхідного проектування. Логічно закінчені фрагменти оформити у вигляді підпрограм, всі необхідні дані яким передаються через список параметрів. Використовувати глобальні змінні – не можна.

Завдання складається з двох частин:

Частина 1. Реалізувати структуру даних у вигляді списку;

Частина 2. Реалізувати структуру даних у вигляді бінарного дерева.

Кожна функція має виконувати лише одну роль, і ця роль має бути відображена у назві функції.

«Функція, яка повертає / обчислює / шукає ...» – має не виводити ці значення, а повернути їх у місце виклику як результат функції або як відповідний вихідний параметр.

До звіту необхідно подати діаграму змін стану пам'яті при виконанні всіх операцій із списком та деревом, вказаних у варіанті завдання.

Реалізувати стандартні дії із списком / бінарним деревом: додавання елементу, вилучення елементу, перегляд, сортування списку, балансування дерева.

Варіант 1.

Звітна відомість результатів екзаменаційної сесії студентської групи для кожного студента містить прізвище, ініціали і оцінки з п'яти предметів.

Скласти програму, за допомогою якої можна коректувати дані (добавляти, вилучати, редагувати інформацію) і отримувати:

- Список / бінарне дерево всіх студентів;
- Список / бінарне дерево студентів, що склали іспити тільки на «5»;
- Список / бінарне дерево студентів, що мають трійки;
- Список / бінарне дерево студентів, що мають двійки. При цьому студент, що має більш ніж одну двійку, виключається із основного списку / бінарного дерева.

Варіант 2.

Підприємство має місцеву телефонну станцію. Телефонний довідник даного підприємства для кожного номера телефону містить номер приміщення і список службовців, що сидять у цьому приміщенні.

Скласти програму, яка:

- коректує базу (добавляє, вилучає, редагує інформацію);
- за номером телефону видає номер приміщення і список людей, що сидять в ньому;
- за номером приміщення видає номер телефону;
- за прізвищем службовця видає номер телефону і номер приміщення.

Номер телефону – двозначний. У одному приміщенні може знаходитися від одного до чотирьох службовців.

Варіант 3.

У готелі є 15 номерів, з них 5 одномісних і 10 двомісних. Скласти програму, яка заповнює та коректує дані про мешканців (добавляє, вилучає, редагує інформацію) і за прізвищем визначає номер, де проживає мешканець.

Програма запрошує прізвище мешканця.

- Якщо мешканця з таким прізвищем немає, про це видається повідомлення.
- Якщо мешканець з таким прізвищем в готелі єдиний, програма видає прізвище мешканця і номер помешкання.
- Якщо в готелі проживає два або більше мешканців з таким прізвищем, програма додатково запрошує ініціали.

Варіант 4.

Є список / бінарне дерево службовців. Для кожного службовця вказано прізвище і ініціали, назву посади, рік прийому на роботу і оклад (величину заробітної плати).

Написати програму, що виконує такі дії:

- корегування списку / бінарного дерева з клавіатури (добавлення, вилучення, редагування інформації);
- сортування за прізвищем, окладом або роком прийому на роботу;
- вивід на екран інформації про службовця, прізвище якого введене з клавіатури.

Варіант 5.

Розклад електричок зберігається у вигляді сукупності записів. Кожен запис містить назву пункту призначення, позначки типу «звичайний», «підвищеного комфорту», «швидкісний експрес» та час відправлення.

Написати програму, що виконує такі дії:

- коректування розкладу з клавіатури (добавлення, вилучення, редагування інформації);
- сортування за станцією призначення або за часом відправлення;
- вивід на екран інформації про поїзди, що відходять після введеного часу.

Варіант 6.

Є список / бінарне дерево товарів. Для кожного товару вказані його назва, вартість одиниці товару в гривнях, кількість і одиниця вимірювання (наприклад, кількість: 100 шт., одиниця вимірювання: упаковка по 20 кг.).

Написати програму, що виконує такі дії:

- коректування списку / бінарного дерева з клавіатури (добавлення, вилучення, редагування інформації);
- сортування по назві товару або за загальною вартістю;
- вивід на екран інформації про товар, назва якого введена з клавіатури;
- вивід на екран інформації про товари із заданого з клавіатури діапазону вартості.

Варіант 7.

Є список / бінарне дерево товарів. Для кожного товару вказані його назва, назва магазину, в якому продається товар, вартість одиниці товару в гривнях і його кількість з вказівкою одиниці вимірювання (наприклад, кількість: 100 шт., одиниця вимірювання: упаковка по 20 кг.).

Написати програму, що виконує такі дії:

- коректування списку / бінарного дерева з клавіатури (добавлення, вилучення, редагування інформації);
- сортування за назвою товару або за назвою магазину;
- вивід на екран інформації про товар, назва якого введена з клавіатури;
- вивід на екран інформації про товари, що продаються в магазині, назва якого введена з клавіатури.

Варіант 8.

Є список / бінарне дерево студентської групи. Кожен рядок списку / бінарного дерева містить прізвище студента і три екзаменаційні оцінки, причому список ніяк не впорядкований.

Скласти програму, яка коректує список / бінарне дерево (добавляє, вилучає, редагує інформацію) і сортує його або за середнім балом, або за прізвищами – в алфавітному порядку, або за оцінками із заданого предмету.

Варіант 9.

Є список / бінарне дерево товарів. Для кожного товару вказані його назва, назва магазину, в якому продається товар, вартість одиниці товару в гривнях і його кількість з вказівкою одиниці вимірювання (наприклад, кількість: 100 шт., одиниця вимірювання: упаковка по 20 кг.).

Написати програму, що виконує такі дії:

- коректування списку / бінарного дерева з клавіатури (добавлення, вилучення, редагування інформації);
- сортування за назвою магазину або за загальною вартістю;
- вивід на екран інформації про товари, що продаються в магазині, назва якого введена з клавіатури;
- вивід на екран інформації про товари із заданого з клавіатури діапазону вартості.

Варіант 10.

Описати структуру з ім'ям **Route**, що містить такі поля:

- назву початкового пункту маршруту;
- назву кінцевого пункту маршруту;

- номер маршруту.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу `Route`;
- впорядкування записів за номерами маршрутів;
- вивід на екран інформації про маршрут, номер якого введений з клавіатури; якщо такого маршруту немає, вивести на екран відповідне повідомлення.

Варіант 11.

Описати структуру з ім'ям `Route`, що містить такі поля:

- назву початкового пункту маршруту;
- назву кінцевого пункту маршруту;
- номер маршруту.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу `Route`;
- впорядкування записів за номерами маршрутів;
- вивід на екран інформації про маршрути, які починаються або закінчуються в пункті, назва якого введена з клавіатури; якщо таких маршрутів немає, вивести на екран відповідне повідомлення.

Варіант 12.

Описати структуру з ім'ям `Note`, що містить такі поля:

- прізвище, ім'я;
- номер телефону;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу `Note`;
- впорядкування записів за датами днів народження;
- вивід на екран інформації про людину, номер телефону якої введений з клавіатури; якщо такої людини немає, вивести на екран відповідне повідомлення.

Варіант 13.

Описати структуру з ім'ям `Note`, що містить такі поля:

- прізвище, ім'я;
- номер телефону;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу `Note`;
- впорядкування записів за алфавітом;
- вивід на екран інформації про людей, що народилися в тому місяці, значення якого введене з клавіатури; якщо таких немає, вивести на екран відповідне повідомлення.

Варіант 14.

Описати структуру з ім'ям `Note`, що містить такі поля:

- прізвище, ім'я;
- номер телефону;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу `Note`;
- впорядкування записів за телефонними номерами;
- вивід на екран інформації про людину, прізвище якої введене з клавіатури; якщо такої немає, вивести на екран відповідне повідомлення.

Варіант 15.

Описати структуру з ім'ям `Zodiac`, що містить такі поля:

- прізвище, ім'я;
- знак Зодіаку;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу `Zodiac`;
- впорядкування записів за датами днів народження.

- вивід на екран інформації про людину, чиє прізвище введене з клавіатури; якщо такої людини немає, вивести на екран відповідне повідомлення.

Варіант 16.

Описати структуру з ім'ям `Zodiac`, що містить такі поля:

- прізвище, ім'я;
- знак Зодіаку;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу `Zodiac`;
- впорядкування записів за прізвищами.
- вивід на екран інформації про людей, що народилися під знаком, найменування якого введене з клавіатури; якщо таких немає, вивести на екран відповідне повідомлення.

Варіант 17.

Описати структуру з ім'ям `Zodiac`, що містить такі поля:

- прізвище, ім'я;
- знак Зодіаку;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу `Zodiac`;
- впорядкування записів за знаками Зодіаку;
- вивід на екран інформації про людей, що народилися в тому місяці, значення якого введене з клавіатури; якщо таких немає, вивести на екран відповідне повідомлення.

Варіант 18.

Описати структуру з ім'ям `Price`, що містить такі поля:

- назва товару;
- назва магазину, в якому продається товар;
- вартість товару в гривнях.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу Price;
- впорядкування записів за алфавітним порядком назв магазинів;
- вивід на екран інформації про товар, назва якого введена з клавіатури; якщо таких товарів немає, вивести на екран відповідне повідомлення.

Варіант 19.

Описати структуру з ім'ям Price, що містить такі поля:

- назва товару;
- назва магазину, в якому продається товар;
- вартість товару в гривнях.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу Price;
- впорядкування записів за алфавітним порядком назв товарів;
- вивід на екран інформації про товари, що продаються в магазині, назва якого введена з клавіатури; якщо такого магазину немає, вивести на екран відповідне повідомлення.

Варіант 20.

Описати структуру з ім'ям Bill, що містить такі поля:

- розрахунковий рахунок платника;
- розрахунковий рахунок одержувача;
- перерахована сума в гривнях.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу Bill;
- впорядкування записів за алфавітним порядком розрахункових рахунків платників;
- вивід на екран інформації про суму, зняту з розрахункового рахунку платника, введеного з клавіатури; якщо такого розрахункового рахунку немає, вивести на екран відповідне повідомлення.

Варіант 21.

Звітна відомість результатів екзаменаційної сесії студентської групи для кожного студента містить прізвище, ініціали і оцінки з п'яти предметів.

Скласти програму, за допомогою якої можна коректувати дані (добавляти, вилучати, редагувати інформацію) і отримувати:

- Список / бінарне дерево всіх студентів;
- Список / бінарне дерево студентів, що склали іспити тільки на «5»;
- Список / бінарне дерево студентів, що мають трійки;
- Список / бінарне дерево студентів, що мають двійки. При цьому студент, що має більш ніж одну двійку, виключається із основного списку / бінарного дерева.

Варіант 22.

Підприємство має місцеву телефонну станцію. Телефонний довідник даного підприємства для кожного номера телефону містить номер приміщення і список службовців, що сидять у цьому приміщенні.

Скласти програму, яка:

- коректує базу (добавляє, вилучає, редагує інформацію);
- за номером телефону видає номер приміщення і список людей, що сидять в ньому;
- за номером приміщення видає номер телефону;
- за прізвищем службовця видає номер телефону і номер приміщення.

Номер телефону – двозначний. У одному приміщенні може знаходитися від одного до чотирьох службовців.

Варіант 23.

У готелі є 15 номерів, з них 5 одномісних і 10 двомісних. Скласти програму, яка заповнює та коректує дані про мешканців (добавляє, вилучає, редагує інформацію) і за прізвищем визначає номер, де проживає мешканець.

Програма запрошує прізвище мешканця.

- Якщо мешканця з таким прізвищем немає, про це видається повідомлення.
- Якщо мешканець з таким прізвищем в готелі єдиний, програма видає прізвище мешканця і номер помешкання.
- Якщо в готелі проживає два або більше мешканців з таким прізвищем, програма додатково запрошує ініціали.

Варіант 24.

Є список / бінарне дерево службовців. Для кожного службовця вказано прізвище і ініціали, назву посади, рік прийому на роботу і оклад (величину заробітної плати).

Написати програму, що виконує такі дії:

- корегування списку / бінарного дерева з клавіатури (добавлення, вилучення, редагування інформації);
- сортування за прізвищем, окладом або роком прийому на роботу;
- вивід на екран інформації про службовця, прізвище якого введене з клавіатури.

Варіант 25.

Розклад електричок зберігається у вигляді сукупності записів. Кожен запис містить назву пункту призначення, позначки типу «звичайний», «підвищеного комфорту», «швидкісний експрес» та час відправлення.

Написати програму, що виконує такі дії:

- коректування розкладу з клавіатури (добавлення, вилучення, редагування інформації);
- сортування за станцією призначення або за часом відправлення;
- вивід на екран інформації про поїзди, що відходять після введеного часу.

Варіант 26.

Є список / бінарне дерево товарів. Для кожного товару вказані його назва, вартість одиниці товару в гривнях, кількість і одиниця вимірювання (наприклад, кількість: 100 шт., одиниця вимірювання: упаковка по 20 кг.).

Написати програму, що виконує такі дії:

- коректування списку / бінарного дерева з клавіатури (добавлення, вилучення, редагування інформації);
- сортування по назві товару або за загальною вартістю;
- вивід на екран інформації про товар, назва якого введена з клавіатури;
- вивід на екран інформації про товари із заданого з клавіатури діапазону вартості.

Варіант 27.

Є список / бінарне дерево товарів. Для кожного товару вказані його назва, назва магазину, в якому продається товар, вартість одиниці товару в гривнях і його кількість з вказівкою одиниці вимірювання (наприклад, кількість: 100 шт., одиниця вимірювання: упаковка по 20 кг.).

Написати програму, що виконує такі дії:

- коректування списку / бінарного дерева з клавіатури (добавлення, вилучення, редагування інформації);
- сортування за назвою товару або за назвою магазину;
- вивід на екран інформації про товар, назва якого введена з клавіатури;
- вивід на екран інформації про товари, що продаються в магазині, назва якого введена з клавіатури.

Варіант 28.

Є список / бінарне дерево студентської групи. Кожен рядок списку / бінарного дерева містить прізвище студента і три екзаменаційні оцінки, причому список ніяк не впорядкований.

Скласти програму, яка коректує список / бінарне дерево (добавляє, вилучає, редагує інформацію) і сортує його або за середнім балом, або за прізвищами – в алфавітному порядку, або за оцінками із заданого предмету.

Варіант 29.

Є список / бінарне дерево товарів. Для кожного товару вказані його назва, назва магазину, в якому продається товар, вартість одиниці товару в гривнях і його кількість з вказівкою одиниці вимірювання (наприклад, кількість: 100 шт., одиниця вимірювання: упаковка по 20 кг.).

Написати програму, що виконує такі дії:

- коректування списку / бінарного дерева з клавіатури (добавлення, вилучення, редагування інформації);
- сортування за назвою магазину або за загальною вартістю;
- вивід на екран інформації про товари, що продаються в магазині, назва якого введена з клавіатури;
- вивід на екран інформації про товари із заданого з клавіатури діапазону вартості.

Варіант 30.

Описати структуру з ім'ям **Route**, що містить такі поля:

- назву початкового пункту маршруту;
- назву кінцевого пункту маршруту;
- номер маршруту.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу **Route**;
- впорядкування записів за номерами маршрутів;

- вивід на екран інформації про маршрут, номер якого введений з клавіатури; якщо такого маршруту немає, вивести на екран відповідне повідомлення.

Варіант 31.

Описати структуру з ім'ям **Route**, що містить такі поля:

- назву початкового пункту маршруту;
- назву кінцевого пункту маршруту;
- номер маршруту.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу **Route**;
- впорядкування записів за номерами маршрутів;
- вивід на екран інформації про маршрути, які починаються або закінчуються в пункті, назва якого введена з клавіатури; якщо таких маршрутів немає, вивести на екран відповідне повідомлення.

Варіант 32.

Описати структуру з ім'ям **Note**, що містить такі поля:

- прізвище, ім'я;
- номер телефону;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу **Note**;
- впорядкування записів за датами днів народження;
- вивід на екран інформації про людину, номер телефону якої введений з клавіатури; якщо такої людини немає, вивести на екран відповідне повідомлення.

Варіант 33.

Описати структуру з ім'ям **Note**, що містить такі поля:

- прізвище, ім'я;
- номер телефону;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу Note;
- впорядкування записів за алфавітом;
- вивід на екран інформації про людей, що народилися в тому місяці, значення якого введене з клавіатури; якщо таких немає, вивести на екран відповідне повідомлення.

Варіант 34.

Описати структуру з ім'ям Note, що містить такі поля:

- прізвище, ім'я;
- номер телефону;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу Note;
- впорядкування записів за телефонними номерами;
- вивід на екран інформації про людину, прізвище якої введене з клавіатури; якщо такої немає, вивести на екран відповідне повідомлення.

Варіант 35.

Описати структуру з ім'ям Zodiac, що містить такі поля:

- прізвище, ім'я;
- знак Зодіаку;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу Zodiac;
- впорядкування записів за датами днів народження.
- вивід на екран інформації про людину, чиє прізвище введене з клавіатури; якщо такої людини немає, вивести на екран відповідне повідомлення.

Варіант 36.

Описати структуру з ім'ям Zodiac, що містить такі поля:

- прізвище, ім'я;
- знак Зодіаку;

- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу **Zodiac**;
- впорядкування записів за прізвищами.
- вивід на екран інформації про людей, що народилися під знаком, найменування якого введене з клавіатури; якщо таких немає, вивести на екран відповідне повідомлення.

Варіант 37.

Описати структуру з ім'ям **Zodiac**, що містить такі поля:

- прізвище, ім'я;
- знак Зодіаку;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу **Zodiac**;
- впорядкування записів за знаками Зодіаку;
- вивід на екран інформації про людей, що народилися в тому місяці, значення якого введене з клавіатури; якщо таких немає, вивести на екран відповідне повідомлення.

Варіант 38.

Описати структуру з ім'ям **Price**, що містить такі поля:

- назва товару;
- назва магазину, в якому продається товар;
- вартість товару в гривнях.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу **Price**;
- впорядкування записів за алфавітним порядком назв магазинів;
- вивід на екран інформації про товар, назва якого введена з клавіатури; якщо таких товарів немає, вивести на екран відповідне повідомлення.

Варіант 39.

Описати структуру з ім'ям `Price`, що містить такі поля:

- назва товару;
- назва магазину, в якому продається товар;
- вартість товару в гривнях.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу `Price`;
- впорядкування записів за алфавітним порядком назв товарів;
- вивід на екран інформації про товари, що продаються в магазині, назва якого введена з клавіатури; якщо такого магазину немає, вивести на екран відповідне повідомлення.

Варіант 40.

Описати структуру з ім'ям `Bill`, що містить такі поля:

- розрахунковий рахунок платника;
- розрахунковий рахунок одержувача;
- перерахована сума в гривнях.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у список / бінарне дерево, що складається з елементів типу `Bill`;
- впорядкування записів за алфавітним порядок розрахункових рахунків платників;
- вивід на екран інформації про суму, зняту з розрахункового рахунку платника, введеного з клавіатури; якщо такого розрахункового рахунку немає, вивести на екран відповідне повідомлення.

Лабораторна робота № 3.13. Опрацювання префіксних дерев та хеш-таблиць

Мета роботи

Навчитися опрацьовувати динамічні структури даних: префіксні дерева та хеш-таблиці.

Питання, які необхідно вивчити та пояснити на захисті

- 1) Загальний синтаксис оголошення вказівникових типів та змінних-вказівників.
- 2) Загальний синтаксис оголошення динамічних змінних.
- 3) Доступ до змінних за допомогою вказівника.
- 4) Дії з вказівниками та динамічними змінними.
- 5) Внутрішня реалізація вказівників, динамічних та статичних змінних.
- 6) Передавання та опрацювання вказівників та динамічних змінних у підпрограмах.

Оформлення звіту

Вимоги та зразок оформлення звіту наведені у вступі до лабораторного практикуму.

Варіанти лабораторних завдань

Програма має містити меню. Необхідно передбачити контроль помилок користувача при введенні даних.

Необхідні програмі дані слід реалізувати за допомогою префіксного дерева або відкритої чи закритої хеш-таблиці.

Передбачити можливість збереження всієї структури даних у файл та завантаження з файлу. Ім'я файлу має вводитися з клавіатури.

При розробці програми застосувати технологію низхідного проектування. Логічно закінчені фрагменти оформити у вигляді підпрограм, всі необхідні дані яким передаються через список параметрів. Використовувати глобальні змінні – не можна.

Завдання складається з трьох частин:

1. Реалізувати структуру даних у вигляді префіксного дерева;
2. Реалізувати структуру даних у вигляді закритої хеш-таблиці;
3. Реалізувати структуру даних у вигляді відкритої хеш-таблиці.

Кожна функція має виконувати лише одну роль, і ця роль має бути відображена у назві функції.

«Функція, яка повертає / обчислює / шукає ...» – має не виводити ці значення, а повернути їх у місце виклику як результат функції або як відповідний вихідний параметр.

До звіту необхідно подати діаграму змін стану пам'яті при виконанні всіх операцій структурою даних, вказаних у варіанті завдання.

Реалізувати стандартні дії: додавання елементу, вилучення елементу, перегляд.

Завдання:

Виконати завдання свого варіанту лабораторної роботи 12.12 за допомогою:

1. префіксного дерева;
2. закритої хеш-таблиці;
3. відкритої хеш-таблиці.

Варіант 1.

Звітна відомість результатів екзаменаційної сесії студентської групи для кожного студента містить прізвище, ініціали і оцінки з п'яти предметів.

Скласти програму, за допомогою якої можна коректувати дані (добавляти, вилучати, редагувати інформацію) і отримувати:

- Префіксне дерево / закриту / відкриту хеш-таблицю всіх студентів;
- Префіксне дерево / закриту / відкриту хеш-таблицю студентів, що склали іспити тільки на «5»;
- Префіксне дерево / закриту / відкриту хеш-таблицю студентів, що мають трійки;
- Префіксне дерево / закриту / відкриту хеш-таблицю студентів, що мають двійки. При цьому студент, що має більш ніж одну двійку, виключається із основного списку / бінарного дерева.

Варіант 2.

Підприємство має місцеву телефонну станцію. Телефонний довідник цього підприємства для кожного номера телефону містить номер приміщення і список службовців, що сидять у цьому приміщенні.

Скласти програму, яка:

- коректує базу (добавляє, вилучає, редагує інформацію);
- за номером телефону видає номер приміщення і список людей, що сидять в ньому;

- за номером приміщення видає номер телефону;
- за прізвищем службовця видає номер телефону і номер приміщення.

Номер телефону – двозначний. У одному приміщенні може знаходитися від одного до чотирьох службовців.

Варіант 3.

У готелі є 15 номерів, з них 5 одномісних і 10 двомісних. Скласти програму, яка заповнює та коректує дані про мешканців (добавляє, вилучає, редагує інформацію) і за прізвищем визначає номер, де проживає мешканець.

Програма запрошує прізвище мешканця.

- Якщо мешканця з таким прізвищем немає, про це видається повідомлення.
- Якщо мешканець з таким прізвищем в готелі єдиний, програма видає прізвище мешканця і номер помешкання.
- Якщо в готелі проживає два або більше мешканців з таким прізвищем, програма додатково запрошує ініціали.

Варіант 4.

Є префіксне дерево / закрыта / відкрита хеш-таблиця дерево службовців. Для кожного службовця вказано прізвище і ініціали, назву посади, рік прийому на роботу і оклад (величину заробітної плати).

Написати програму, що виконує такі дії:

- корегування префіксного дерева / закрытої / відкритої хеш-таблиці з клавіатури (добавлення, вилучення, редагування інформації);
- вивід на екран інформації про службовця, прізвище якого введено з клавіатури.

Варіант 5.

Розклад електричок зберігається у вигляді сукупності записів. Кожен запис містить назву пункту призначення, позначки типу «звичайний», «підвищеного комфорту», «швидкісний експрес» та час відправлення.

Написати програму, що виконує такі дії:

- коректування розкладу з клавіатури (добавлення, вилучення, редагування інформації);
- вивід на екран інформації про поїзди, що відходять після введенного часу.

Варіант 6.

Є префіксне дерево / закрыта / відкрита хеш-таблиця товарів. Для кожного товару вказані його назва, вартість одиниці товару в гривнях, кількість і одиниця вимірювання (наприклад, кількість: 100 шт., одиниця вимірювання: упаковка по 20 кг.).

Написати програму, що виконує такі дії:

- коректування префіксного дерева / закрытої / відкритої хеш-таблиці з клавіатури (добавлення, вилучення, редагування інформації);
- вивід на екран інформації про товар, назва якого введена з клавіатури;
- вивід на екран інформації про товари із заданого з клавіатури діапазону вартості.

Варіант 7.

Є префіксне дерево / закрыта / відкрита хеш-таблиця товарів. Для кожного товару вказані його назва, назва магазину, в якому продається товар, вартість одиниці товару в гривнях і його кількість з вказівкою одиниці вимірювання (наприклад, кількість: 100 шт., одиниця вимірювання: упаковка по 20 кг.).

Написати програму, що виконує такі дії:

- коректування префіксного дерева / закрытої / відкритої хеш-таблиці з клавіатури (добавлення, вилучення, редагування інформації);
- вивід на екран інформації про товар, назва якого введена з клавіатури;
- вивід на екран інформації про товари, що продаються в магазині, назва якого введена з клавіатури.

Варіант 8.

Є префіксне дерево / закрыта / відкрита хеш-таблиця студентської групи. Кожний елемент цієї структури містить прізвище студента і три екзаменаційні оцінки, причому дані ніяк не впорядковані.

Скласти програму, яка корегує префіксне дерево / закрыту / відкриту хеш-таблицю (добавляє, вилучає, редагує інформацію).

Варіант 9.

Є префіксне дерево / закрыта / відкрита хеш-таблиця товарів. Для кожного товару вказані його назва, назва магазину, в якому продається товар, вартість одиниці товару в гривнях і його кількість з вказівкою одиниці вимірювання (наприклад, кількість: 100 шт., одиниця вимірювання: упаковка по 20 кг.).

Написати програму, що виконує такі дії:

- корегування префіксного дерева / закритої / відкритої хеш-таблиці з клавіатури (добавлення, вилучення, редагування інформації);
- вивід на екран інформації про товари, що продаються в магазині, назва якого введена з клавіатури;
- вивід на екран інформації про товари із заданого з клавіатури діапазону вартості.

Варіант 10.

Описати структуру з ім'ям **Route**, що містить такі поля:

- назву початкового пункту маршруту;
- назву кінцевого пункту маршруту;
- номер маршруту.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу **Route**;
- вивід на екран інформації про маршрут, номер якого введений з клавіатури; якщо такого маршруту немає, вивести на екран відповідне повідомлення.

Варіант 11.

Описати структуру з ім'ям **Route**, що містить такі поля:

- назву початкового пункту маршруту;
- назву кінцевого пункту маршруту;
- номер маршруту.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу **Route**;
- вивід на екран інформації про маршрути, які починаються або закінчуються в пункті, назва якого введена з клавіатури; якщо таких маршрутів немає, вивести на екран відповідне повідомлення.

Варіант 12.

Описати структуру з ім'ям **Note**, що містить такі поля:

- прізвище, ім'я;
- номер телефону;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу Note;
- вивід на екран інформації про людину, номер телефону якої введений з клавіатури; якщо такої людини немає, вивести на екран відповідне повідомлення.

Варіант 13.

Описати структуру з ім'ям Note, що містить такі поля:

- прізвище, ім'я;
- номер телефону;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу Note;
- вивід на екран інформації про людей, що народилися в тому місяці, значення якого введене з клавіатури; якщо таких немає, вивести на екран відповідне повідомлення.

Варіант 14.

Описати структуру з ім'ям Note, що містить такі поля:

- прізвище, ім'я;
- номер телефону;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу Note;
- вивід на екран інформації про людину, прізвище якої введене з клавіатури; якщо такої немає, вивести на екран відповідне повідомлення.

Варіант 15.

Описати структуру з ім'ям Zodiac, що містить такі поля:

- прізвище, ім'я;
- знак Зодіаку;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблиця, що складається з елементів типу **Zodiac**;
- вивід на екран інформації про людину, чиє прізвище введене з клавіатури; якщо такої людини немає, вивести на екран відповідне повідомлення.

Варіант 16.

Описати структуру з ім'ям **Zodiac**, що містить такі поля:

- прізвище, ім'я;
- знак Зодіаку;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу **Zodiac**;
- вивід на екран інформації про людей, що народилися під знаком, найменування якого введене з клавіатури; якщо таких немає, вивести на екран відповідне повідомлення.

Варіант 17.

Описати структуру з ім'ям **Zodiac**, що містить такі поля:

- прізвище, ім'я;
- знак Зодіаку;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу **Zodiac**;
- вивід на екран інформації про людей, що народилися в тому місяці, значення якого введене з клавіатури; якщо таких немає, вивести на екран відповідне повідомлення.

Варіант 18.

Описати структуру з ім'ям **Price**, що містить такі поля:

- назва товару;
- назва магазину, в якому продається товар;
- вартість товару в гривнях.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу Price;
- вивід на екран інформації про товар, назва якого введена з клавіатури; якщо таких товарів немає, вивести на екран відповідне повідомлення.

Варіант 19.

Описати структуру з ім'ям Price, що містить такі поля:

- назва товару;
- назва магазину, в якому продається товар;
- вартість товару в гривнях.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу Price;
- вивід на екран інформації про товари, що продаються в магазині, назва якого введена з клавіатури; якщо такого магазину немає, вивести на екран відповідне повідомлення.

Варіант 20.

Описати структуру з ім'ям Bill, що містить такі поля:

- розрахунковий рахунок платника;
- розрахунковий рахунок одержувача;
- перерахована сума в гривнях.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу Bill;
- вивід на екран інформації про суму, зняту з розрахункового рахунку платника, введеного з клавіатури; якщо такого розрахункового рахунку немає, вивести на екран відповідне повідомлення.

Варіант 21.

Звітна відомість результатів екзаменаційної сесії студентської групи для кожного студента містить прізвище, ініціали і оцінки з п'яти предметів.

Скласти програму, за допомогою якої можна коректувати дані (добавляти, вилучати, редагувати інформацію) і отримувати:

- Префіксне дерево / закриту / відкриту хеш-таблицю всіх студентів;

- Префіксне дерево / закрито / відкрито хеш-таблицю студентів, що склали іспити тільки на «5»;
- Префіксне дерево / закрито / відкрито хеш-таблицю студентів, що мають трійки;
- Префіксне дерево / закрито / відкрито хеш-таблицю студентів, що мають двійки. При цьому студент, що має більш ніж одну двійку, виключається із основного списку / бінарного дерева.

Варіант 22.

Підприємство має місцеву телефонну станцію. Телефонний довідник цього підприємства для кожного номера телефону містить номер приміщення і список службовців, що сидять у цьому приміщенні.

Скласти програму, яка:

- коректує базу (добавляє, вилучає, редагує інформацію);
- за номером телефону видає номер приміщення і список людей, що сидять в ньому;
- за номером приміщення видає номер телефону;
- за прізвищем службовця видає номер телефону і номер приміщення.

Номер телефону – двозначний. У одному приміщенні може знаходитися від одного до чотирьох службовців.

Варіант 23.

У готелі є 15 номерів, з них 5 одномісних і 10 двомісних. Скласти програму, яка заповнює та коректує дані про мешканців (добавляє, вилучає, редагує інформацію) і за прізвищем визначає номер, де проживає мешканець.

Програма запрошує прізвище мешканця.

- Якщо мешканця з таким прізвищем немає, про це видається повідомлення.
- Якщо мешканець з таким прізвищем в готелі єдиний, програма видає прізвище мешканця і номер помешкання.
- Якщо в готелі проживає два або більше мешканців з таким прізвищем, програма додатково запрошує ініціали.

Варіант 24.

Є префіксне дерево / закрита / відкрита хеш-таблиця дерево службовців. Для кожного службовця вказано прізвище і ініціали, назву посади, рік прийому на роботу і оклад (величину заробітної плати).

Написати програму, що виконує такі дії:

- корегування префіксного дерева / закритої / відкритої хеш-таблиці з клавіатури (добавлення, вилучення, редагування інформації);
- вивід на екран інформації про службовця, прізвище якого введене з клавіатури.

Варіант 25.

Розклад електричок зберігається у вигляді сукупності записів. Кожен запис містить назву пункту призначення, позначки типу «звичайний», «підвищеного комфорту», «швидкісний експрес» та час відправлення.

Написати програму, що виконує такі дії:

- коректування розкладу з клавіатури (добавлення, вилучення, редагування інформації);
- вивід на екран інформації про поїзди, що відходять після введеного часу.

Варіант 26.

Є префіксне дерево / закрита / відкрита хеш-таблиця товарів. Для кожного товару вказані його назва, вартість одиниці товару в гривнях, кількість і одиниця вимірювання (наприклад, кількість: 100 шт., одиниця вимірювання: упаковка по 20 кг.).

Написати програму, що виконує такі дії:

- коректування префіксного дерева / закритої / відкритої хеш-таблиці з клавіатури (добавлення, вилучення, редагування інформації);
- вивід на екран інформації про товар, назва якого введена з клавіатури;
- вивід на екран інформації про товари із заданого з клавіатури діапазону вартості.

Варіант 27.

Є префіксне дерево / закрита / відкрита хеш-таблиця товарів. Для кожного товару вказані його назва, назва магазину, в якому продається товар, вартість одиниці товару в гривнях і його кількість з вказівкою одиниці вимірювання (наприклад, кількість: 100 шт., одиниця вимірювання: упаковка по 20 кг.).

Написати програму, що виконує такі дії:

- коректування префіксного дерева / закритої / відкритої хеш-таблиці з

клавіатури (добавлення, вилучення, редагування інформації);

- вивід на екран інформації про товар, назва якого введена з клавіатури;
- вивід на екран інформації про товари, що продаються в магазині, назва якого введена з клавіатури.

Варіант 28.

Є префіксне дерево / закрита / відкрита хеш-таблиця студентської групи. Кожний елемент цієї структури містить прізвище студента і три екзаменаційні оцінки, причому дані ніяк не впорядковані.

Скласти програму, яка корегує префіксне дерево / закриту / відкриту хеш-таблицю (добавляє, вилучає, редагує інформацію).

Варіант 29.

Є префіксне дерево / закрита / відкрита хеш-таблиця товарів. Для кожного товару вказані його назва, назва магазину, в якому продається товар, вартість одиниці товару в гривнях і його кількість з вказівкою одиниці вимірювання (наприклад, кількість: 100 шт., одиниця вимірювання: упаковка по 20 кг.).

Написати програму, що виконує такі дії:

- корегування префіксного дерева / закритої / відкритої хеш-таблиці з клавіатури (добавлення, вилучення, редагування інформації);
- вивід на екран інформації про товари, що продаються в магазині, назва якого введена з клавіатури;
- вивід на екран інформації про товари із заданого з клавіатури діапазону вартості.

Варіант 30.

Описати структуру з ім'ям `Route`, що містить такі поля:

- назву початкового пункту маршруту;
- назву кінцевого пункту маршруту;
- номер маршруту.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу `Route`;
- вивід на екран інформації про маршрут, номер якого введений з клавіатури; якщо такого маршруту немає, вивести на екран відповідне повідомлення.

Варіант 31.

Описати структуру з ім'ям **Route**, що містить такі поля:

- назву початкового пункту маршруту;
- назву кінцевого пункту маршруту;
- номер маршруту.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу **Route**;
- вивід на екран інформації про маршрути, які починаються або закінчуються в пункті, назва якого введена з клавіатури; якщо таких маршрутів немає, вивести на екран відповідне повідомлення.

Варіант 32.

Описати структуру з ім'ям **Note**, що містить такі поля:

- прізвище, ім'я;
- номер телефону;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу **Note**;
- вивід на екран інформації про людину, номер телефону якої введений з клавіатури; якщо такої людини немає, вивести на екран відповідне повідомлення.

Варіант 33.

Описати структуру з ім'ям **Note**, що містить такі поля:

- прізвище, ім'я;
- номер телефону;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу **Note**;
- вивід на екран інформації про людей, що народилися в тому місяці, значення якого введене з клавіатури; якщо таких немає, вивести на екран відповідне повідомлення.

Варіант 34.

Описати структуру з ім'ям `Note`, що містить такі поля:

- прізвище, ім'я;
- номер телефону;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу `Note`;
- вивід на екран інформації про людину, прізвище якої введене з клавіатури; якщо такої немає, вивести на екран відповідне повідомлення.

Варіант 35.

Описати структуру з ім'ям `Zodiac`, що містить такі поля:

- прізвище, ім'я;
- знак Зодіаку;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу `Zodiac`;
- вивід на екран інформації про людину, чиє прізвище введене з клавіатури; якщо такої людини немає, вивести на екран відповідне повідомлення.

Варіант 36.

Описати структуру з ім'ям `Zodiac`, що містить такі поля:

- прізвище, ім'я;
- знак Зодіаку;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу `Zodiac`;
- вивід на екран інформації про людей, що народилися під знаком, найменування якого введене з клавіатури; якщо таких немає, вивести на екран відповідне повідомлення.

Варіант 37.

Описати структуру з ім'ям `Zodiac`, що містить такі поля:

- прізвище, ім'я;
- знак Зодіаку;
- день народження (масив з трьох чисел).

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу `Zodiac`;
- вивід на екран інформації про людей, що народилися в тому місяці, значення якого введене з клавіатури; якщо таких немає, вивести на екран відповідне повідомлення.

Варіант 38.

Описати структуру з ім'ям `Price`, що містить такі поля:

- назва товару;
- назва магазину, в якому продається товар;
- вартість товару в гривнях.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу `Price`;
- вивід на екран інформації про товар, назва якого введена з клавіатури; якщо таких товарів немає, вивести на екран відповідне повідомлення.

Варіант 39.

Описати структуру з ім'ям `Price`, що містить такі поля:

- назва товару;
- назва магазину, в якому продається товар;
- вартість товару в гривнях.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу `Price`;
- вивід на екран інформації про товари, що продаються в магазині, назва якого введена з клавіатури; якщо такого магазину немає, вивести на екран відповідне повідомлення.

Варіант 40.

Описати структуру з ім'ям **Bill**, що містить такі поля:

- розрахунковий рахунок платника;
- розрахунковий рахунок одержувача;
- перерахована сума в гривнях.

Написати програму, що виконує такі дії:

- введення даних з клавіатури у префіксне дерево / закриту / відкриту хеш-таблицю, що складається з елементів типу **Bill**;
- вивід на екран інформації про суму, зняту з розрахункового рахунку платника, введеного з клавіатури; якщо такого розрахункового рахунку немає, вивести на екран відповідне повідомлення.

Предметний покажчик

Ф

FIFO принцип, 12

Л

LIFO принцип, 9

Б

Батьківський вузол, 8, 19, 20

Бінарне дерево, 8, 19, 20

опис елемента, 21

Бінарне дерево пошуку, 19

В

Вироджене дерево, 19

Висота дерева, 8, 19

Вказівник, 5

доступ до змінної за вказівником, 5

розіменування вказівника, 5

Вказівник на функцію, 21

Вузол, 20

Вузол-нащадок, 19

Вузол-предок, 19

Г

Глибина дерева, 8, 19

Д

Двонаправлені списки, 7

Дерево, 7, 18

кореневий вузол, 18

корінь, 18, 20

Дерево, 20

Дочірній вузол, 8, 19, 20

К

Кільцевий однонаправлений список без заголовку, 7

Кільцеві списки, 7

Кореневий вузол, 8, 18, 20

Корінь дерева, 8, 18, 20

Л

Листовий вузол, 8, 19, 20

Листок, 8

Лінійний двонаправлений список без заголовку, 7

Лінійний однонаправлений список

сканування, 16

ітераційний спосіб, 16

рекурсивний спосіб, 17

Лінійний однонаправлений список без заголовку, 7

Лінійний однонаправлений список з заголовком, 7

Лінійні списки, 7

О

Однонаправлені списки, 7

П

Петля, 7, 18

Р

Рівень ієрархії, 18

Рівень ієрархії вузла, 8

С

Списки без елемента-заголовку, 7

Списки з елементом-заголовком, 7

Список, 7

Стек, 9

вершина стеку, 9

вилучення елементів, 11

добавлення елементів, 10

знищення, 11

ініціалізація, 10

опис елементу, 10

структура, 9

схема вилучення елементів, 9

схема добавлення елементів, 9

формування, 10

Ц

Цикл, 7, 18

Ч

Черга, 12

вилучення елементів, 14

добавлення елементів, 14

знищення, 14

ініціалізація, 13

опис елемента, 13

структура, 13

формування, 14

Література

Основна

1. Ковалюк Т.В. Основи програмування. – К. BHV, 2005. – 384 с.
2. Wirth, Niklaus. Algorithms + Data Structures = Programs. Prentice-Hall, 1976. – 366 p.
3. Robinson, Simon. Professional C#. Wrox Press; 2nd edition. 2002. – 1270 p.
4. Stanley Lippman, Josée Lajoie, and Barbara Moo. C++ Primer. Addison-Wesley Professional. 2012. – 976 p.
5. Mark Reed. C++: 2 books in 1 - The Ultimate Beginners Guide to Master C++ Programming Quickly with No Prior Experience. Independently published. 2022. – 254 p.
6. D.S. Malik. C++ Programming: From Problem Analysis to Program Design. Cengage Learning. 2017. – 1488 p.
7. Tony Gaddis. Starting Out with C++ from Control Structures to Objects. Pearson. 2017. – 1344 p.
8. Bjarne Stroustrup. The C++ Programming Language. Addison-Wesley Professional. 2013. – 1376 p.
9. D.S. Malik. C++ Programming: Program Design Including Data Structures. Cengage Learning. 2017. – 1728 p.
10. Marc Gregoire. Professional C++. Wrox. 2021. – 1312 p.
11. Martin Fowler. UML Distilled: A Brief Guide to the Standard Object Modeling Language. 3rd edition. Addison-Wesley, 2004. – 208 p.
12. Grady Booch, James Rumbaugh, and Ivar Jacobson. The Unified Modeling Language User Guide. 2nd Edition. Addison-Wesley Professional, 2005. – 475 p.
13. Herbert Schildt. Java: A Beginner's Guide. 8th Edition. McGraw Hill. 2018. – 720 p.
14. Herbert Schildt. Java: The Complete Reference. 11th Edition. McGraw Hill. 2018. – 1248 p.
15. Kathy Sierra, Bert Bates. Head First Java. 2nd Edition. O'Reilly Media. 2005. – 688 p.

Додаткова

1. Donald Knuth. The Art of Computer Programming, Volumes 1-4A. Addison-Wesley Professional. 2011. – 3168 p.
2. Scott Chacon. Pro Git, 2nd Edition. Apress. 2014. – 458 p.

Інформаційні ресурси

1. [https://learn.microsoft.com/en-us/previous-versions/60k1461a\(v=vs.140\)?redirectedfrom=MSDN](https://learn.microsoft.com/en-us/previous-versions/60k1461a(v=vs.140)?redirectedfrom=MSDN)
2. <http://cppstudio.com/cat/274/>
3. <https://learngitbranching.js.org/?locale=uk>
4. <https://git-scm.com/book/uk/v2/%D0%92%D1%81%D1%82%D1%83%D0%BF-%D0%9F%D1%80%D0%BE-%D1%81%D0%B8%D1%81%D1%82%D0%B5%D0%BC%D1%83-%D0%BA%D0%BE%D0%BD%D1%82%D1%80%D0%BE%D0%BB%D1%8E-%D0%B2%D0%B5%D1%80%D1%81%D1%96%D0%B9>
5. <https://learn.microsoft.com/uk-ua/dotnet/csharp/programming-guide/>
6. [https://msdn.microsoft.com/uk-ua/library/67ef8sbd\(v=vs.120\).aspx](https://msdn.microsoft.com/uk-ua/library/67ef8sbd(v=vs.120).aspx)
7. <http://docs.oracle.com/javase/tutorial/index.html>
8. <https://developer.ibm.com/languages/java/>
9. <http://www.tutorialspoint.com/java/>